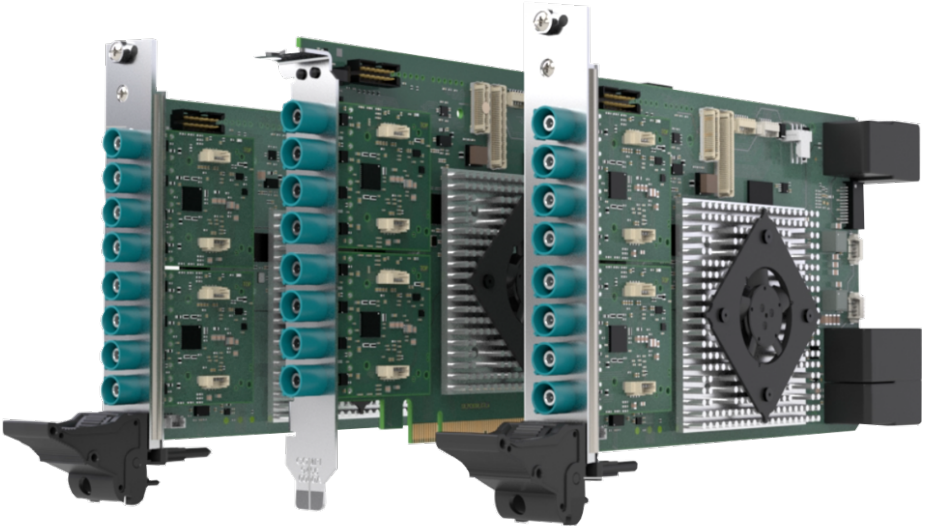


# SOLECTRIX PROFRAME QUICKSTART GUIDE

*Solectrix GmbH*

Author:
S. Herbrecher

February 10, 2025

# Contents


**1 Hardware Setup** **5**
- 1.1 Hardware Assembly . . . 5
  - 1.1.1 Adapter interfaces . . . 6
- 1.2 Mainboard PCIe slot selection . . . 6
- 1.3 Thunderbolt . . . 6
- 1.4 Camera connectors . . . 9

**2 Software Setup** **11**
- 2.1 Linux . . . 11
  - 2.1.1 Unpacking the Software Archive . . . 11
  - 2.1.2 Grabber card detection . . . 11
  - 2.1.3 Building the Linux kernel . . . 11
    - 2.1.3.1 Ubuntu custom kernel . . . 11
  - 2.1.4 Linux Software Driver . . . 12
    - 2.1.4.1 Manual Driver Management . . . 12
      - 2.1.4.1.1 Building the Driver . . . 13
      - 2.1.4.1.2 Loading the driver . . . 13
    - 2.1.4.2 Automatic Driver Management via DKMS . . . 13
      - 2.1.4.2.1 Driver Installation . . . 13
      - 2.1.4.2.2 Setting non-standard Driver Options . . . 14
      - 2.1.4.2.3 Loading and Unloading the installed Driver . . . 15
      - 2.1.4.2.4 NVIDIA GPUDirect RDMA . . . 15
    - 2.1.4.3 Linux Software Applications . . . 16
      - 2.1.4.3.1 Building the Linux applications . . . 16
      - 2.1.4.3.2 Linux applications overview . . . 16
      - 2.1.4.3.3 sxpfapp . . . 17
      - 2.1.4.3.4 sxpfclinit . . . 18
      - 2.1.4.3.5 sxpfcsi2decode . . . 18
      - 2.1.4.3.6 sxpfcsi2encode . . . 18
      - 2.1.4.3.7 sxpfi2c . . . 19
      - 2.1.4.3.8 sxpfinfo . . . 19
      - 2.1.4.3.9 sxpf-init-sequence . . . 19
      - 2.1.4.3.10 sxpfosc . . . 20
      - 2.1.4.3.11 sxpfplay . . . 20
      - 2.1.4.3.12 sxpfrawinfo . . . 21
      - 2.1.4.3.13 sxpfrec . . . 21
      - 2.1.4.3.14 sxpfrw . . . 21
      - 2.1.4.3.15 sxpfscope . . . 22
      - 2.1.4.3.16 sxpfsend . . . 22
      - 2.1.4.3.17 sxpfsendmulti . . . 22
      - 2.1.4.3.18 sxpftrigger . . . 22
      - 2.1.4.3.19 sxpfupdate . . . 23
      - 2.1.4.3.20 sxpfxo2 . . . 24
      - 2.1.4.3.21 cap_ng . . . 24
      - 2.1.4.3.22 cucap . . . 25
- 2.2 Windows . . . 26
  - 2.2.1 Windows Setup . . . 26
  - 2.2.2 Windows driver installation . . . 27
  - 2.2.3 Hardware Interface Service . . . 29
    - 2.2.3.1 Running the Service manually . . . 29
    - 2.2.3.2 Installation as a Windows Service . . . 29
    - 2.2.3.3 Service Options . . . 29
  - 2.2.4 Windows Software Applications . . . 30
    - 2.2.4.1 Building the Windows applications . . . 30
    - 2.2.4.2 Windows applications overview . . . 30

2.3 Graphical User Application - sxpfpygui . . . 31

**3 Update** **32**
3.1 Video grabber card . . . 32
3.1.1 Grabber card firmware version . . . 32
3.1.2 Firmware Update . . . 32
3.2 Camera adapter . . . 33
3.2.1 Camera adapter firmware version . . . 33
3.2.2 Firmware Update . . . 33

**4 Initialization** **35**
4.1 Initialization sequences . . . 35
4.2 Initialization sequence file format . . . 35
4.2.1 Sections . . . 35
4.2.2 Events . . . 37
4.2.3 Commands . . . 37
4.2.4 I2C commands . . . 40
4.2.5 Simple INI Example . . . 40

**5 Camera adapter FPGA - I2C slave interface** **41**
5.1 camAD MAX96705/96706 . . . 43
5.2 camAD MAX9296A and camAD MAX96716A . . . 44
5.3 camAD MAX9295/9296 and camAD MAX96716A/96717 . . . 45
5.4 camAD TI913/914 . . . 49
5.5 camAD TI933/934 . . . 52
5.6 camAD TI953 Dual . . . 53
5.7 camAD TI953/954 . . . 55
5.8 camAD TI954 . . . 60

**6 Power over Coax (POC)** **61**
6.1 proFRAME 2.0 . . . 61
6.2 proFRAME 3.0 . . . 61

**7 Trigger** **62**
7.1 External Trigger Interface . . . 62

**8 Capturing** **64**
8.1 CSI-2 RAW capture mode . . . 65
8.2 CSI-2 capturing different virtual channels . . . 65

**9 Replaying** **67**
9.1 Loopback Tests . . . 67

**10 Software API** **69**

**11 Advanced features** **70**
11.1 GPS timestamping . . . 70

**12 Cameras** **71**
12.1 IMI . . . 71
12.1.1 RDACM24 . . . 71
12.1.1.1 OmniVision OV490 Firmware Merge Tool . . . 71
12.1.1.2 Flash programming . . . 72
12.1.1.3 Host commands . . . 72

**13 FAQ** **73**
13.1 [sxpf0] Could only allocate XX of YY DMA buffers . . . 73
13.2 flash init error . . . 73
13.3 Error - content of `xxx.bit` is no valid bit file . . . 73
13.4 [sxpf0] Plasma command timed out on arg slot . . . 74
13.5 Uninstalling Windows driver . . . 74
13.6 No valid license . . . 74
13.7 uyvy2rgb.vert not found . . . 74
13.8 [sxpf0] pci_iomap of BAR 4 failed . . . 74
13.9 proFRAME 2.0 - LED Status Information . . . 74
13.10 proFRAME 3.0 - LED Status Information . . . 75
13.11 Framesize of RAW files . . . 75
13.12 Open CSI-2 RAW file in ImageJ . . . 76
13.13 I2C bus scan . . . 79
13.14 unrecognized command-line option '-ftrivial-auto-var-init=zero' . . . 79
13.15 Ubuntu 18.04 . . . 79

**14 Appendix** **81**
14.1 proFRAME 2 - FPGA Register Description . . . 81
14.1.1 FPGA System Register . . . 82
14.1.1.1 PROJECT . . . 82
14.1.1.2 DATE . . . 82
14.1.1.3 VERSION . . . 82
14.1.1.4 Video Processing Register . . . 83
14.1.1.4.1 CSI-2 PACKET PARSER CONFIGURATION . . . 84
14.1.1.4.2 REPLAY CONTROL . . . 85
14.1.1.5 PLASMA Control Register . . . 86
14.1.1.5.1 I2C BUS CONTROL - PORT 0...3 . . . 86
14.1.1.5.2 Trigger Module . . . 87
14.1.1.5.3 Trigger Module Architecture . . . 87
14.1.1.5.4 Trigger Module Overview . . . 88
14.1.1.5.5 Trigger Module Modes . . . 88
14.1.1.5.6 Continuous Mode . . . 89
14.1.1.5.7 Manual Mode . . . 90
14.1.1.5.8 External Trigger Mode . . . 91
14.1.1.5.9 External Trigger Mode (modified) . . . 92
14.1.1.5.10 External Trigger Input Frequency Multiplied . . . 93
14.1.1.5.11 External Trigger Output Frequency Divided . . . 94
14.1.1.5.12 External Trigger Input Frequency Multiplied (User Mode) . . . 95
14.1.1.6 Register . . . 96
14.1.1.6.1 TRIGGER CONTROL . . . 97
14.1.1.6.2 TRIGGER PERIOD . . . 98
14.1.1.6.3 TRIGGER PULSE LENGTH . . . 98
14.1.1.6.4 TRIGGER DELAY . . . 99
14.1.1.6.5 EXTERNAL TRIGGER PERIOD . . . 99
14.1.1.6.6 EXTRNAL TRIGGER PULSE LENGTH . . . 99
14.1.1.6.7 TRIG_OUT DIVIDER . . . 100
14.1.1.6.8 EXT_TRIG_IN MULTIPLIER . . . 100
14.1.1.6.9 EXT_TRIG_IN MULTIPLIER USER . . . 101
14.1.1.6.10 EXT_TRIG SOURCE SELECT . . . 102
14.1.1.6.11 TRIGGER TIME - PRIMARY TIMESTAMP . . . 103
14.1.1.6.12 TRIGGER TIME - SECONDARY TIMESTAMP . . . 103
14.2 proFRAME 3 - FPGA Register Description . . . 104
14.2.1 Video Processing Register . . . 105
14.2.1.1 MIPI TRANSMITTER CONTROL . . . 106
14.2.1.2 MIPI RECEIVER CONTROL . . . 107
14.2.1.3 PIXEL FORMATTER CONFIGURATION . . . 107

14.2.1.4 PACKET PARSER CONFIGURATION . . . . . 108
14.2.2 PLASMA Control Register . . . . . 110
14.2.2.1 I2C BUS CONTROL - PORT 0...3 . . . . . 110
14.2.2.2 CAMERA ADAPTER CONTROL . . . . . 111
14.2.2.3 DEBOUNCE COUNTER VALUE . . . . . 111
14.2.2.4 DEBOUNCE CONTROL . . . . . 111
14.2.2.5 BASEBOARD CONTROL . . . . . 112
14.2.2.6 MIPI CONTROL/STATUS . . . . . 112
14.2.2.7 MIPI CONTROL/STATUS SET . . . . . 113
14.2.2.8 MIPI CONTROL/STATUS CLEAR . . . . . 113
14.3 proFRAME V2 dimensions . . . . . 114
14.4 proFRAME V2 component overview . . . . . 114

# 1 Hardware Setup

## 1.1 Hardware Assembly

Usually the Solectrix grabber card is fully assembled when the card is delivered to the customer. In this case the card only has to be assembled into the development PC. If camera adapters should be replaced or additional adapters should be added the following adapter slots are available. When using dual input or dual output camera adapters only slot 0 and slot 3 can be used. When using single input/output camera adapters all slots are available.

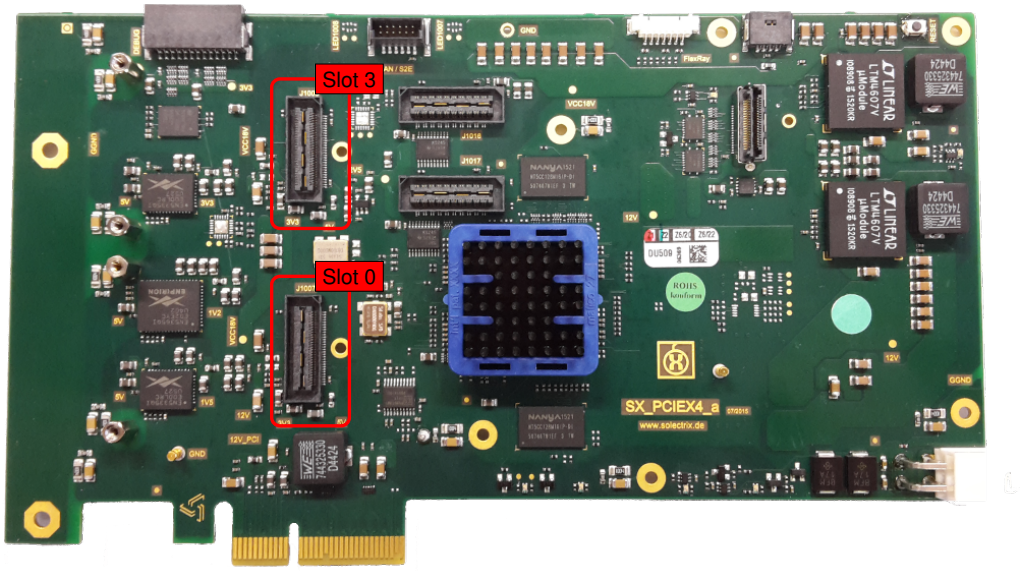



Figure 1: Grabber card top with slot description

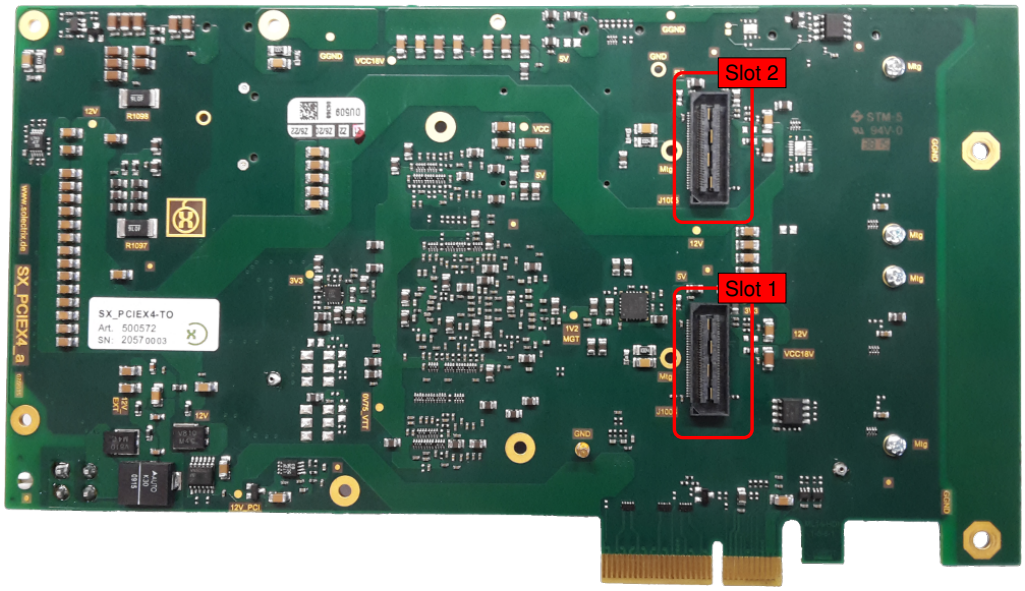



Figure 2: Grabber card bottom with slot description

### 1.1.1 Adapter interfaces

Currently there exist two types of hardware interfaces between the camera adapter and the base board. Both interfaces use the same connector but have a different pinout and signal voltage. Adapters with max. 3,2Gbit/sec datarate usually hava a parallel interface with 3,3V FPGA I/O voltage. Adapters with higher datarates like GMSL2 require a LVDS interface between adapter and base board that works with 2,5V FPGA I/O voltage. This different FPGA I/O voltages require small modifications on the base board. Before assembly of the camera adapters make sure that you have the matching base board variant for your adapter. To double check if the base board is the correct variant check the component placement of the base board (see appendix) against the following table.

| **Components** | **FB5000** | **FB5001** | **FB5002** | **FB5003** | **FB5004** | **FB5005** | **FB5006** | **FB5007** |
|---|---|---|---|---|---|---|---|---|
| | affects J1005 | | affects J1006 | | affects J1007 | | affects J1008 | |
| **Parallel interface** | placed | not placed | not placed | placed | not placed | placed | not placed | placed |
| **LVDS interface** | not placed | placed | placed | not placed | placed | not placed | placed | not placed |

Table 1: Baseboard adapter interface variants

## 1.2 Mainboard PCIe slot selection

When assembling the Solectrix grabber card into the PC the PCIe slot of the mainboard must be carefully selected. The PCIe slot must support at least PCIe x4 Gen1 standard for the SX_PCIEX4 and PCIe x4 Gen2 for the SX_PCIEX4_G2 grabber card. Check the manual of the mainboard for valid PCIe slots. However some mainboards do not support the grabber card in some PCIe slots because of mainboard specific reasons. In this case try if the card is detected in the other available slots.

## 1.3 Thunderbolt

The Sonnet Echo Express SE I combined with a proFRAME grabber card is a smart way to capture automotive cameras with development Laptops. To connect the Sonnet box to your laptop use the Thunderbolt 3 cable coming with the Sonnet box and connect it to your Thunderbolt connector on the Laptop. Take care that you really use the Thunderbolt cable and the Thunderbolt connector. Because USB3 and Thunderbolt 3 use the same USB-Type-C connector but from technical side they are different.
Also take care that you are using the latest software and BIOS firmware and maybe do not use Thunderbolt docking stations that might cause problems. Because Thunderbolt is sometimes not very good supported from the BIOS and operating system. Usually Windows works better than Linux in this case.
The Sonnet box has two Thunderbolt connectors on the backside (see figure 3). Both connectors work because Thunderbolt allows daisy chaining of multiple devices. Also do not forget to connect the external power supply to the Sonnet box. Thunderbolt allows power supply through the Thunderbolt cable but this does not work with the Sonnet box.
To check if the proFRAME device is connected correctly and also attached by the operating system you can check the heardbeat LED. This LED should blink orange/green. From software side you might have to allow the connection of new Thunderbolt devices. Depending on the Thunderbolt software driver you might have a Thunderbolt icon in the tray bar (see figure 4).
When you double-click this icon you should see a dialog like figure 5 that shows the connection status. If this connection status is 'not connected' you can change this by clicking on the status. Now you should see a dialog like in figure 6 where you can allow the connection.
After this steps you should see the heartbeat LED blinking orange/green. If the LED is red please contact your IT service to check what is wrong with your Thunderbolt settings. If the LED is off it might be a problem of the proFRAME grabber card.

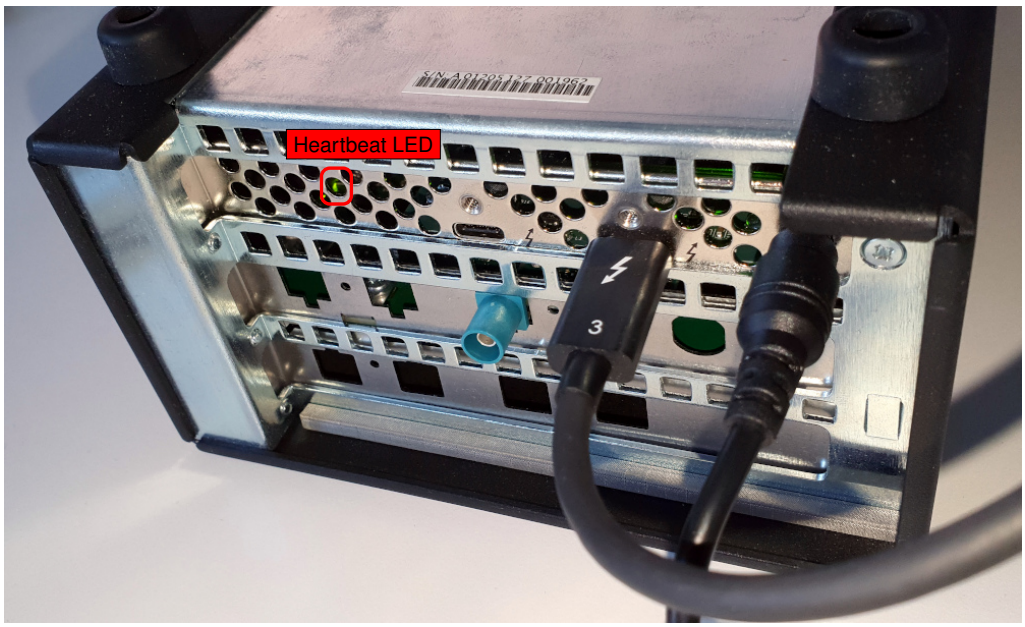



Figure 3: Thunderbolt - Sonnet Echo Express SE I - connection

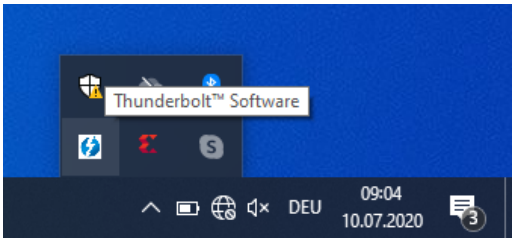



Figure 4: Thunderbolt - Tray Icon

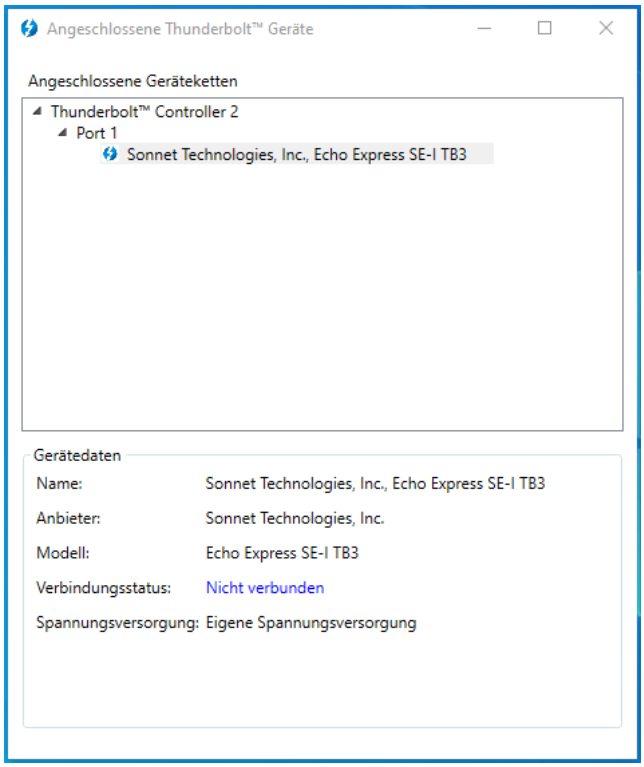



Figure 5: Thunderbolt - Dialog

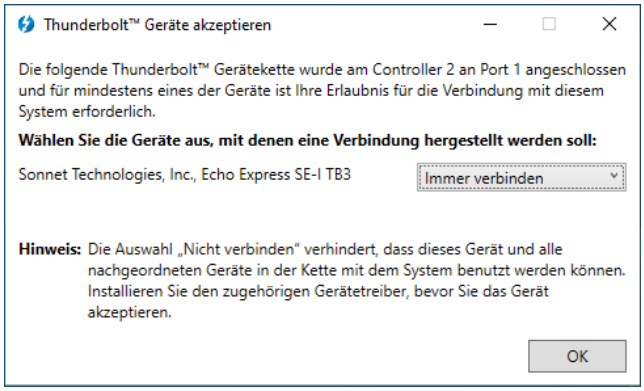



Figure 6: Thunderbolt - Connect Dialog

## 1.4 Camera connectors

Depending on the camera adapter type the camera connectors are mapped to different video channels in the software. When using dual input camera adapters the mapping is described in figure 7. Another example is shown in figure 8 where the mapping is decribed for a grabber card with two single input/output camera adapters.

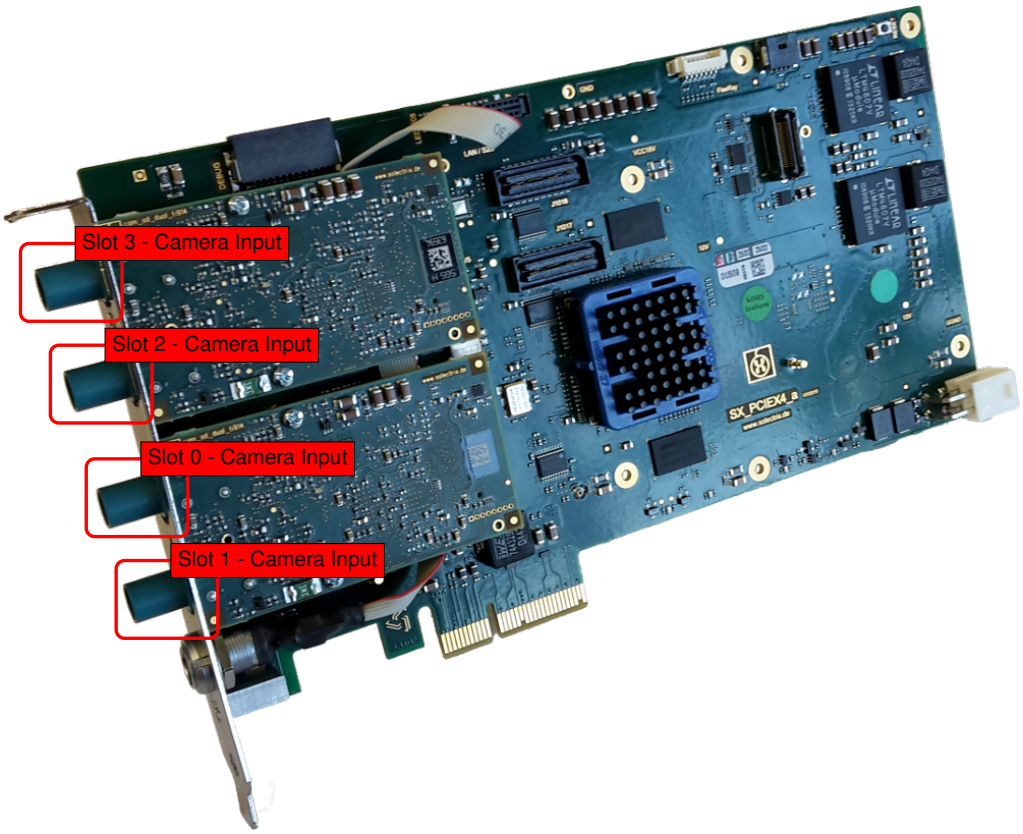



Figure 7: Grabber card with dual input camera adapters

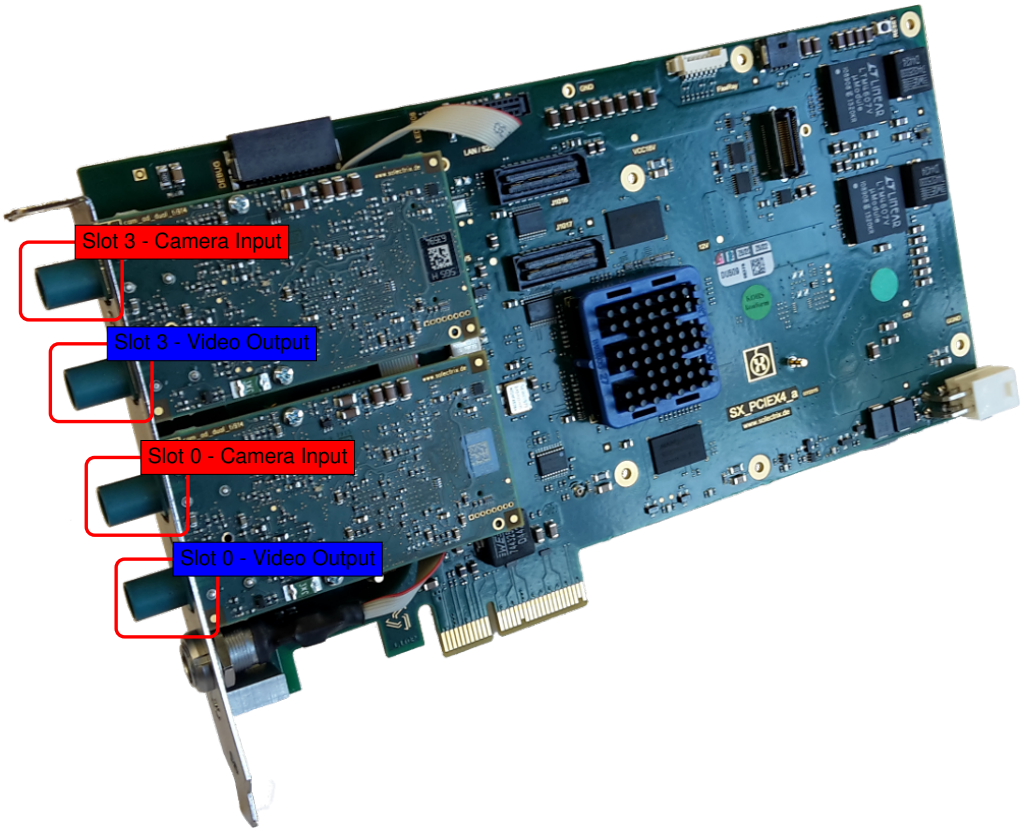



Figure 8: Grabber card with single input/output camera adapters

# 2 Software Setup

## 2.1 Linux

### 2.1.1 Unpacking the Software Archive

The software archive for Linux must be unpacked into a directory, which does **not contain any SPACE, TAB or other white-space characters** itself or in any of its parent directories up to the file system's root.

This is a limitation of the Linux kernel's build system, which we must use for building the `sxpf` kernel driver.

### 2.1.2 Grabber card detection

After assembling the grabber card into the PC check if the card is detected by running lspci. The output should look similar to the following.

```
$ sudo lspci -d:5555 -v
01:00.0 RAM memory: PLDA Device 5555 Subsystem: PLDA Device 5555
       Flags: fast devsel, IRQ 11
       Memory at f7b20000 (32-bit, non-prefetchable) [disabled] [size=64K]
       Memory at f7b34000 (32-bit, non-prefetchable) [disabled] [size=4K]
       Memory at f7b00000 (32-bit, non-prefetchable) [disabled] [size=128K]
       Memory at f7b33000 (32-bit, non-prefetchable) [disabled] [size=4K]
       Memory at f7b30000 (32-bit, non-prefetchable) [disabled] [size=8K]
       Memory at f7b32000 (32-bit, non-prefetchable) [disabled] [size=4K]
       Expansion ROM at f7a00000 [disabled] [size=1M]
       Capabilities: [50] MSI: Enable- Count=1/1 Maskable- 64bit+
       Capabilities: [78] Power Management version 3
       Capabilities: [80] Express Legacy Endpoint, MSI 00
```

### 2.1.3 Building the Linux kernel

> ☞ The following steps are only needed if the proFRAME firmware does not support scatter-gathering feature. This is the case for most of the proFRAME 2.0 replay firmwares. All proFRAME 3.0 firmwares do support scatter-gathering so the following steps are optional and not suggested and a standard kernel can be uses.

To avoid memory management problems it is suggested to build a custom kernel that supports contiguous memory allocation. The grabber card requires contiguous memory blocks in the system memory where the image data is copied. However if the framesize is less than 4MB in most cases the standard Linux kernel will also work. If images larger than 4MB should be captured the customized kernel is necessary.

#### 2.1.3.1 Ubuntu custom kernel

The following section describes how to build a custom Linux kernel under Ubuntu. For the configuration the settings of the currently running kernel are used and extended by the required kernel parameters.
First install all the required packages to build the kernel

```
$ sudo apt install libssl-dev build-essential ncurses-dev xz-utils flex bison libelf-dev
    zstd debhelper
```

Get the version of your running kernel and download the kernel sources from https://www.kernel.org/

```
$ uname -r
```

Extract the kernel source and copy the configuration of the currently running kernel into the kernel source dir

```
$ tar xfv ./linux*.tar.gz
$ cd ./linux*
$ cp /boot/config-$(uname -r) .config
```

Extend the kernel configuration with the required CMA parameters

```
$ sed -ri '/CONFIG_SYSTEM_TRUSTED_KEYS/s/=.+/=""/g' .config
$ sed -ri '/CONFIG_SYSTEM_REVOCATION_KEYS/s/=.+/=""/g' .config
$ echo "CONFIG_CMA=y" >> .config
$ echo "CONFIG_DMA_CMA=y" >> .config
$ echo "CONFIG_CMA_SIZE_MBYTES=512" >> .config
$ echo "CONFIG_CMA_SIZE_SEL_MBYTES=y" >> .config
$ sed -i 's/CONFIG_DEBUG_INFO=y/CONFIG_DEBUG_INFO=n/' .config # optional: faster without
    debug info
$ make olddefconfig
```

Build the kernel image and header packages

```
$ make -j$(nproc) bindeb-pkg LOCALVERSION=-cma
```

☞ If build failes with the following error simply create an empty file (e.g. using touch command) named REPORTING-BUGS in the kernel folder

```
install: cannot stat 'REPORTING-BUGS': No such file or directory
```

☞ If build failes with the error message

```
No rule to make target 'debian/canonical-certs.pem', needed by 'certs/
    x509_certificate_list'. Stop.
```

execute the following command

```
$ echo "CONFIG_SYSTEM_TRUSTED_KEYS=''" >> .config
$ make olddefconfig
```

Install the new kernel

```
$ sudo dpkg -i linux-image-*.deb
$ sudo dpkg -i linux-headers-*.deb
```

Enable CMA support by modifing the Grub configuration file /etc/default/grub. E.g.

```
GRUB_CMDLINE_LINUX_DEFAULT="coherent_pool=512M quiet splash"
```

Update Grub and reboot using new kernel

```
$ sudo update-grub
$ sudo reboot
```

### 2.1.4 Linux Software Driver

The kernel module is the basic driver that is needed to interact with the grabber card.
Two alternative ways of managing the driver installation are supported: Manually, and via DKMS.

#### 2.1.4.1 Manual Driver Management

#### 2.1.4.1.1 Building the Driver

To use the grabber card it is necessary to build the sxpf Linux kernel module.
The driver module is normally built as part of the build process, when running `make` in the `./src/` directory.
But it can also be built individually. For this go into the base or driver directory and making the target `driver`.
The output should look like the following:

```
$ cd ./src/
$ make driver
  CC [M] /home/devel/sxpf/trunk/src/libs/sxpf_ll/lin64/driver/sxpf_module.o
  CC [M] /home/devel/sxpf/trunk/src/libs/sxpf_ll/lin64/driver/sxpf_cmd_fifo.o
  LD [M] /home/devel/sxpf/trunk/src/libs/sxpf_ll/lin64/driver/sxpf.o
  Building modules, stage 2.
  MODPOST 1 modules
  CC /home/devel/sxpf/trunk/src/libs/sxpf_ll/lin64/driver/sxpf.mod.o
  LD [M] /home/devel/sxpf/trunk/src/libs/sxpf_ll/lin64/driver/sxpf.ko
[copy] libs/sxpf_ll/lin64/driver/sxpf.ko
[copy] libs/sxpf_ll/lin64/driver/load_sxpf.sh
```

The last two line of the above output show that the kernel module and the load script are copied to the `release` directory.

#### 2.1.4.1.2 Loading the driver

The kernel module is loaded using the load_sxpf.sh script. The output should look similar to the following. Warnings about the firmware file sxpf.fw can be ignored.

```
$ sudo ./src/release/load_sxpf.sh
$ dmesg | grep sxpf
[50694.743591] [sxpf] Driver revision r2461 initializing.
[50694.748940] [sxpf0] created sxpf device 0x0f200000
[50694.748995] [sxpf0] Playback FPGA version 9.10.6, built on 2019-07-19
[50694.753698] [sxpf1] created sxpf device 0x0f200001
[50694.753744] [sxpf1] Record FPGA version 3.10.9, built on 2019-04-18
```

Table 2 shows the kernel module parameters. For example use these parameters to set the frame buffer size to 5MB:

```
$ sudo ./src/release/load_sxpf.sh framesize=$((0x500000))
```

### 2.1.4.2 Automatic Driver Management via DKMS

#### 2.1.4.2.1 Driver Installation

Using DKMS is the most convenient method of managing the driver installation, because DKMS automatically re-installs the driver for each updated or newly installed Linux kernel.

```
$ cd ./src/
$ sudo make dkms_install
```

The installation status of the driver can be verified with the following command:

```
$ dkms status
nvidia, 430.50, 4.15.0-66-generic, x86_64: installed
nvidia, 430.50, 5.3.8-cma, x86_64: installed
sxpf, 2476, 5.3.8-cma, x86_64: installed
```

If the driver should be installed for a kernel it is currently not active for, use the following command:

```
$ sudo dkms install -m sxpf/2476 -k 4.15.0-66-generic
```

| Parameter | Example | Description |
|---|---|---|
| framesize | framesize=$((0x400000)) | This parameter sets the size that is allocated in system memory for one frame. The example sets the frame buffer size to 4MB. The current frame buffer size can be checked using the sxpfinfo tool. |
| buffers | buffers=10 | This parameter sets the number of frame buffers pre-allocated by the driver in system memory. For proFRAME 2.0 each camera should have at least 2 frame buffers. Because proFRAME 3.0 does not have hardware frame buffers at least 3 frame buffers should be used per camera. The current number of frame buffers can be checked using the sxpfinfo tool. |
| user_buffers | user_buffers=24 | Set the number of buffers that can be provided by user programs. If the sum of user_buffers and buffers exceeds 31, user buffers take precedence over driver-allocated buffers. |
| debug | debug=4 | This parameter sets the debug level. Only enable this option when requesting support from Solectrix |
| debug_irq | debug_irq=4 | This parameter sets the interrupt debug level. Only enable this option when requesting support from Solectrix |
| trigint | trigint=1 | Enable generation of trigger interrupts. This option is normally disabled so that kernel logs captured with debug=5 don't contain too much register access noise. |
| map_cached | map_cached=0 | Mainly for ARM64: Allow switching off caching for DMA buffers, if the CPU/SoC doesn't have a bus snooping unit for the caches. Has no effect on x86.[1] |

[1] Generally, setting this to false is needed on ARM, as hardware snooping is not required by the architecture. But the NVIDIA Xavier AGX is an example that has coherent caches, so this setting can be left enabled.

Table 2: Kernel module parameters

To remove the driver from DKMS for all kernels it is currently active in, issue the following command (subsituting the driver revision 2476 with the output from dkms status):

```
$ sudo dkms remove sxpf/2476 --all
```

To remove the driver from DKMS for only one specific kernel, issue the following command (subsituting the driver revision 2476 with the output from dkms status and the kernel version accordingly):

```
$ sudo dkms remove sxpf/2476 -k 4.15.0-66-generic
```

#### 2.1.4.2.2 Setting non-standard Driver Options

Because after installing the driver via DKMS it is loaded automatically when the system starts, module parameters must be set using a config file in the directory /etc/modprobe.d.
For example, to set the frame buffer size used by the driver to 8MByte, create a file /etc/modprobe.d/sxpf-options.conf as root with the following content:

```
options sxpf framesize=8388608
```

Other options, like debug logging, can be appended to this line to further customize the driver operation.

☞ Numeric values must be entered in decimal in the options file /etc/modprobe.d/sxpf-options.conf. Expressions like $((0x800000)) are *not supported*.

#### 2.1.4.2.3 Loading and Unloading the installed Driver

If the driver is loaded automatically, it can still be unloaded and loaded again manually — also with different parameter settings, if needed.
Unloading the installed driver is possible with the command:

```
$ sudo rmmod sxpf
```

If the automatic loading of the driver is not desired, create the file `/etc/modprobe.d/blacklist-sxpf.conf` and add the following line to it:

```
blacklist sxpf
```

The driver, if istalled with DKMS, can be loaded (again) with:

```
$ sudo modprobe sxpf [optional parameters...]
```

#### 2.1.4.2.4 NVIDIA GPUDirect RDMA

The Linux driver supports linking to the NVIDIA proprietary kernel module to enable NVIDIA GPUDirect RDMA for sending grabbed image data directly to GPU device memory without using up the PC's main RAM bandwidth.
The following preconditions must be met to use this feature:

- For compiling the proFRAME driver and running applications:
  - NVIDIA GPU with support for GPUDirect RDMA. Generally, all professional graphics cards with a product number of 4000 or higher are supported (e.g., RTX A4000, Quadro RTX 4000, Quadro P4000,...)
  - Installed NVIDIA driver. NVIDIA's Closed-Source and GPL-licensed variants both work. See below for restrictions imposed by the closed-source driver.
  - Setting the option `SXPF_NV_P2P=y` when building the proFRAME driver (see below).
  - Loading the driver with enabled user-space buffer allocation by setting the `user_buffers` module parameter to a non-zero value. See table 2 in section 2.1.4.1.2 for a description of the kernel module parameters.
- For compiling user applications that make use of the RDMA feature:
  - Installed CUDA Toolkit. Any recent CUDA version will do (we tested with 10.0 up to 12.2), either from your Linux distribution's package repository [1] or NVIDIA's installer.

☞ The RDMA feature must be enabled by setting the variable `SXPF_NV_P2P=y` on the `make` command line when building the driver:

```
$ cd ./src/
$ make driver SXPF_NV_P2P=y
```

or in the environment when installing it via DKMS:

```
$ cd ./src/
$ SXPF_NV_P2P=y sudo make dkms_install
```

Enabling this feature with the closed-source NVIDIA driver requires that the license of the proFRAME kernel module changes from "GPL" to "Solectrix". This prevents it from using a number of kernel functions that are only available to GPL-licensed modules, which has the following consequences, sorted from most to least severe:

[1] For Ubuntu: sudo apt install nvidia-cuda-toolkit

- The driver can't create the character device under `/dev` on it's own, anymore.
  This must now be done by running the `mknod` command. The script `load_sxpf.sh` detects the need for this and does it automatically. When using DKMS for installation no such automatism is implemented, yet, so it is suggested to use `load_sxpf.sh` instead of DKMS when activating NVRDMA.
- The clock source `SXPF_CLOCK_MONOTONIC` when calling `sxpf_timestamp_sync()` is not available anymore and will result in an error return value when used.
  The use of this clock is not recommended, anyway, because it is subject to jumps introduced by NTP (due to leap seconds and switching daylight saving time). Rather use `SXPF_CLOCK_MONOTONIC_RAW`.
- PCIe Advanced Error Reporting is disabled.
- The time duration logged in certain debug outputs of the driver has lower resolution.

If the `Makefile` detects the availablity of NVRDMA in the installed NVIDIA driver it may automtically enable it (for the GPL-licensed driver, for instance on the Jetson platform). If you don't want this to happen you should set the `SXPF_NV_P2P` environment variable to an empty value:

```
$ cd ./src/
$ make driver SXPF_NV_P2P=""
```

or

```
$ cd ./src/
$ SXPF_NV_P2P="" sudo make dkms_install
```

#### 2.1.4.3 Linux Software Applications

The Linux library contains various applications to control the grabber system. This tools can also be used as sample code to integrate similar functions into the customer software tools.

##### 2.1.4.3.1 Building the Linux applications

Before building the sample applications install the required dependencies using the systems package manager

Listing 1: Install applicaion dependencies

```
$ sudo apt install libsdl2-ttf-dev libgl-dev libglm-dev libavcodec-dev libavformat-dev
    libavutil-dev libjpeg-turbo8-dev libpng-dev libswscale-dev libopencv-dev libsdl2-dev
```

To build the Linux applications execute the following commands

Listing 2: Building the Linux applications

```
$ cd ./src/
$ make
```

After a successful build all tools can be found in the directory ./src/release/.

##### 2.1.4.3.2 Linux applications overview

#### 2.1.4.3.3 sxpfapp

sample application

Listing 3: sxpfapp

```
$ ./sxpfapp -h
The sxpfapp sample application allows capturing image frames from the grabber card.
The commandline output shows the received frame size and frame rate.

Usage: sxpfapp [options]

Options:
       -a num Abort after number of images
       -A num Abort after number of seconds if no image is received
       -b Dump binary (default: CSV)
       -c image Compare received images against reference image
       -C10|-C12|-C14|-C16
                    Assume CSI-2 format with 10, 12, 14 or 16 bits per component
       -d dt CSI-2 datatype/virtual channel to parse RAW CSI-2 data format
                    Bit 7:6 = virtual channel
                    Bit 5:0 = datatype
                    Do not use together with -C parameter
       -e name File name of saved error images (default: err_%010d.raw)
       -D image Input image for debugging sxpfapp image pipeline
       -f Start fullscreen
       -g WxH[@XxY] Set window geometry, e.g. -g 1280x720
                    Optionally add window position, e.g. -g 1280x720@0x0
       -G WxH Force image geometry and disregard geometry of received data
       -h/--help Show this help screen
       -i Input is 12bpp packed raw DOL2 from SONY IMX290
       -l num Shift input pixels left by this amount (default: 6)
       -n name File name of saved images (default: img_%010d.raw)
       -o Input is RGB24 in 32 bit per pixel OpenLDI-scrambled
       -p 0..3 Select 1 out of 4 possible Bayer patterns
       -q Do not show SDL window
       -r RAW input: de-bayer received images
       -R24|-R32 Select RGB input (24bit RGB or 32bit RGBA)
       -s shader Override shader file(s): filename without extension
                    Built-in shaders: bgr, bw, debayer, oldi, rgb, uyvy, vyuy, yuv420, yuyv
                        , yvyu
       -t Enable pixel test
       -w num Write number of images to disk (-1 = save all)
       -W num Write number of error images to disk (-1 = save all)
       -v Store video into binary video.raw file
       -V Verify CSI-2 data (check for CRC errors)
       -8 Input is compacted to 8 bit per component
       --card Select card to capture (0...cards-1)
       --channel Select channel to capture (0...7)
       --drop x x >= 2: Drop every x frame
                    x <= -2: Show every x frame
                    In this mode the displayed frame rate will be a calculated mean value
       --newline Output \n instead of \r when printing frame informations
       --show-ts Show timestamp informations
       --borderless Show SDL window in borderless mode
       --bg 0xrrggbb Set background color for SDL window (default: 0x4c4cff)
       --gamma x Set gamma value (0.5-2.0, default: 1.2)
       --saturation x Set saturation value (0.0-2.0, default: 1.0)
       --vflip Flip image vertically
```

☞ sxpfapp does not work with Microsoft RemoteDesktop because it only provides OpenGL Version 1.1 but sxpfapp requires OpenGL Version >=4.3. When using RemoteDesktop the windowless quiet mode (parmeter -q) must be used.

#### 2.1.4.3.4 sxpfclinit

sample application

Listing 4: sxpfclinit

```
$ ./sxpfclinit
The sxpfclinit sample application shows how to program the Lattice CrossLink FPGA
when using a camera adapter like MAX9295/9296

Usage: sxpfclinit <device num> [options] <algo file> <data file>

Options:
 -b, --bus <number> I2C bus number (default: 0).
 -B, --baud <rate> Set baudrate of I2C bus (default: 100000).
 -c, --configure Configure the Port-Enables automatically.
 -d, --device <ID> CrossLink I2C device ID (default: 0x40).
 -v, --verbose Print verbose messages. Repeat for mor detail.

Operations:

  CrossLink util version: 18101401
```

#### 2.1.4.3.5 sxpfcsi2decode

sample application

Listing 5: sxpfcsi2decode

```
$ ./sxpfcsi2decode
The sxpfcsi2decode sample applications decodes RAW CSI-2 based frame data into 16bit arrays
    and stores them into binary files
Usage: sxpfcsi2decode <raw_file>
```

#### 2.1.4.3.6 sxpfcsi2encode

sample application

Listing 6: sxpfcsi2encode

```
$ ./sxpfcsi2encode
The sxpfcsi2encode sample applications encodes RAW CSI-2 based frame data into a out.raw
    file
from 16bit arrays stored in binary files
The names of the binary files must match the format out0-0xAA-BBBBxCCCC.bin
like sxpfcsi2decode creates it to decode the relevant meta data
Usage: sxpfcsi2encode <out0-0xAA-BBBBxCCCC.bin> <out1-0xDD-EEEExFFFF.bin> ...
```

#### 2.1.4.3.7 sxpfi2c

sample application

Listing 7: sxpfi2c

```
$ ./sxpfi2c
The sxpfi2c sample applications shows how to read and write data over I2C

Usage: sxpfi2c [options] <device num> <chan> <slave ID> <read len> [<wbytes>...]
       sxpfi2c [options] <device num> <chan> <command file>
Options:
       -f <I2C flags>: 0 = DO_START_STOP
                       1 = DO_START_ONLY
                       2 = DO_NO_START_STOP
                       3 = DO_STOP_ONLY
```

#### 2.1.4.3.8 sxpfinfo

sample application

Listing 8: sxpfinfo

```
$ ./sxpfinfo
system information: Linux 6.0.0-murcle #1 SMP PREEMPT_DYNAMIC Mon Oct 3 11:48:21 CEST 2022
    x86_64
sxpf library revision: r5713 (v3.4.6)
sxpf kernel module revision: r5713 (v3.4.6)

Found 1 SX proFRAME card in the system:
#0 -> SX proFRAME 3.0 PCIe, CAPTURE card, Firmware version 4.2.10 (Scatter/Gather)
     build date 2022-07-21
     Plasma version 2022-06-28/1
     max. video channels = 8
     8 pre-allocated image buffers, max. frame size = 4194304
     0 image buffers available for user allocation
     I2C capture supported with 31 buffers of size 4096
     Available bandwidth at slot 6400MByte/s (PCIe x8 Gen 3)
     Chip temperature 54.9°C
       port #0 -> SX camAD3 TI954 dual (or TI936 dual)
              Adapter type 23, version 3
              MachXO creation date 2020-12-03/2
              EEPROM INI data capacity: 1016 bytes
              EEPROM is blank
       port #1 -> no adapter
```

#### 2.1.4.3.9 sxpf-init-sequence

sample application

Listing 9: sxpf-init-sequence

```
$ ./sxpf-init-sequence

The sxpf-init-sequence sample applications shows how to write INI files
to initialize the camera adapters and connected cameras

Usage:
  program eeprom:
       sxpf-init-sequence <device> -port <n> -ini <input.ini> --write
       sxpf-init-sequence <device> -port <n> -bin <input.bin> --write
  execute directly (don't touch eeprom):
```

```
      sxpf-init-sequence <device> -port <n> -ini <input.ini> --execute <event-id>
      sxpf-init-sequence <device> -port <n> -bin <input.bin> --execute <event-id>
  generate binary setup file:
      sxpf-init-sequence -ini <input.ini> -bin <output.bin>
  dump setup sequence:
      sxpf-init-sequence -ini <input.ini> --dump
      sxpf-init-sequence -bin <input.bin> --dump
      sxpf-init-sequence <device> -port <n> --dump
  decode and write setup-sequence to new ini-file:
      sxpf-init-sequence -ini <input.ini> -out <output.ini>
      sxpf-init-sequence -bin <input.bin> -out <output.ini>
```

#### 2.1.4.3.10 sxpfosc

sample application

Listing 10: sxpfosc

```
Usage: sxpfosc <device num> <chan> <freq_hz>
```

#### 2.1.4.3.11 sxpfplay

sample application

Listing 11: sxpfplay

```
$ .sxpfplay -h
The sxpfplay sample application shows how to replay image frames
through the serializer to a connected ECU

Usage: sxpfplay [options]

Options:
  -h|--help This help
  -C|--card <num> Select card to send on. (0...cards-1)
  -c|--channel <num> Select channel to send on. (0...3)
  -D|--datatype <num> CSI-2 RAW datatype to encode to
                          0 for no CSI-2 encoding (default: 0)
     --csi2-febd <num> Front EBD lines count (default: 0)
     --csi2-rebd <num> Rear EBD lines count (default: 0)
  -f|--frame-rate <freq> Set frame rate of synthetic images (default: 30.0 fps)
  -g|--geometry <WxH[@XxY]> Set image geometry
                          e.g. - g 1280x720
                               - g 1280x720@10x10
     --loop Loop video sequence
     --bpp <num> Force bpp value
     --ifg <num> Inter-Frame Gap
     --ilg <num> Inter-Line Gap
  -x|--max-frames <n> Send n frames (default: 100), -1 for 'no limit'.
     --source-dir <name> Provide playback frames from files in
                          directory <name>.
     --video <name> Provide playback frames from video.raw
     --ffmpeg <filepath> Provide video from FFmpeg supported formats
     --plugin <name> Provide plugin name
     --value <value> Set <value> as a fixed pixel value for frame generator
     --screencapture Captures screen
     --sync Use time stamps from recording, instead of
                          replaying with fixed frame spacing

Debug options:
```

```
 -b|--buffers <num> Number of used buffers (default: 2)
    --csi2-crc Compute CSI-2 checksums in software
                        (requires --datatype, default: off)
 -d|--dump Dump sent images
 -i|--force-counter <pos> Force incrementing of frame counter
                        and write 32bit counter to payload at byte <pos>
                        <pos> = -1 disables writing counter to payload
    --pixclk <hz> Set pixel clock frequency to <hz>
 -T <seconds> Timeout to wait for a free buffer (default: 1s)
 -t|--thread Use extra thread to recycle buffers
```

☞ Please do not use sxpfplay as a reference implementation for your own replay software. Instead use sxpfsend or sxpfsendmulti as a reference

#### 2.1.4.3.12 sxpfrawinfo

sample application

Listing 12: sxpfrawinfo

```
$ ./sxpfrawinfo
The sxpfrawinfo sample applications shows how to analyze the Solectrix image header and CSI
    -2-formatted frame data

Usage: sxpfrawinfo <raw_file>
```

#### 2.1.4.3.13 sxpfrec

sample application

Listing 13: sxpfrec

```
$ ./sxpfrec
The sxpfrec sample application shows how easy it is to capture a frame from the grabber card
    .
```

#### 2.1.4.3.14 sxpfrw

sample application

Listing 14: sxpfrw

```
$ ./sxpfrw
The sxpfrw sample application shows how to read and write register values
in the grabber card FPGA.

Usage: sxpfrw <device number> <offset> [<write value>]
        <offset> = BAR * 0x100000 + BAR offset (BAR = base address register)
                   BAR = 0 when accessing FPGA logic registers
                   BAR = 1 when accessing Plasma microcontroller registers
```

#### 2.1.4.3.15 sxpfscope

sample application

Listing 15: sxpfscope

```
$ ./sxpfscope -h
The sxpfscope sample application shows if frame data is received by
the grabber card based on the frame valid and line valid signals

Usage: sxpfscope <device number> <channel> [characters per line]
        <channel> = 0 ... 7
        hit Ctrl-C to close application
Output:
        '_' = FrameValid = LOW and LineValid = LOW (inter frame gap)
        '-' = FrameValid = HIGH and LineValid = LOW (inter line gap)
        'x' = FrameValid = HIGH and LineValid = HIGH (valid data)
        'o' = FrameValid = LOW and LineValid = HIGH (invalid lines received maybe frame sync
             pulse received instead of frame valid signal)
```

#### 2.1.4.3.16 sxpfsend

sample application

Listing 16: sxpfsend

```
$ ./sxpfsend
The sxpfsend sample applications shows how easy it is to replay a frame to the grabber card.
Usage: sxpfsend <card> <channel> <raw_file>
```

#### 2.1.4.3.17 sxpfsendmulti

sample application

Listing 17: sxpfsendmulti

```
$ ./sxpfsendmulti
The sxpfsendmulti sample applications shows how easy it is to replay a frame to multiple
    grabber cards and multiple camera adapters.
Usage: sxpfsendmulti <raw_file> <card0> <channel0> <card1> <channel1> ...
```

#### 2.1.4.3.18 sxpftrigger

sample application

Listing 18: sxpftrigger

```
$ ./sxpftrigger
The sxpftrigger sample application shows how to configure the trigger generator
to send frame synchronization signals to the connected cameras

Usage: sxpftrigger <device number> [options]

Device number:
       0,1,2... Open captue device <number>

Options:
       -c <mask> Channel mask value
                                  For selecting cam adapter, set the corresponding bit, e.g
                                      .
                                  -c 0x1 = 0b0001 -> camera adapter port #0
                                  -c 0x2 = 0b0010 -> camera adapter port #1
                                  -c 0x4 = 0b0100 -> camera adapter port #2
```

```
                                -c 0x8 = 0b1000 -> camera adapter port #3
                                -c 0xf = 0b1111 -> camera adapter port #0 #1 #2 #3
      -d Dump current config
      -D <delay> Capture trigger delay in ms
      -e <time> Camera exposure time for config in ms
      -E <time> Manual trigger with exposure time in ms
      -f <freq> Frame rate in Hz
      -i <mask> Enable secondary external trigger input mask (0..15)
                                If bit for adapter is 0 primary external trigger is used
                                If bit for adapter is 1 secondary external trigger is
                                    used
      -l <time> Output trigger length in ms
      -m <mult> Trigger input frequency multiplier
      -O|--timestamp-offset <offset> Trigger polarity: H = active high, L = active low
      -p H|L Trigger polarity: H = active high, L = active low
      -P <source> External PPS source:
                                0=selected input of -s parameter (default)
                                1=primary input of J1009
      -q <div> Trigger output frequency divider
      -r <source adapter> Specify trigger routing from one adapter to another:
                                source can be 0,1,2,3 or i ('internal', default)
      -R|--timestamp-receiver Enable timestamp receiver
      -S|--timestamp-sender Enable timestamp sender
      -s <source> External trigger source:
                                0=J1009 (external trigger connector)
                                1=J4000 (PC internal trigger connector)
      -t <trig> Trigger type: cont, man, ext, extmod, extmul, outdiv, dis (disabled)
      -T <trigger channel> Start new section of settings for trigger channel 0..3 (default
          : 0)
      -w <number> Watch timestamp against different card
      -W <number> Watch saved timestamp against different card
      -h Show this help screen
```

#### 2.1.4.3.19 sxpfupdate

sample application

Listing 19: sxpfupdate

```
$ ./sxpfupdate
The sxpfupdate sample application shows how to update the FPGA on the grabber card

Usage:
   sxpfupdate <bitfile> <device> -- write <bitfile> to FPGA-device
   sxpfupdate <device> <bitfile> -- read <bitfile> back from FPGA-device
Optional arguments:
   sxpfupdate ... -offset <o> -- start offset to read/write
   sxpfupdate ... -size <n> -- number of bytes to read/write
   sxpfupdate ... -bpix16 <0|1> -- convert bitfile to binary-format BPIx16 before flashing
   sxpfupdate ... -raw -- do not check for bitfile, write only allowed for offsets >= 0
       xF00000
Example:
   sxpfupdate update.bit /dev/sxpf0 -- update FPGA firmware
   sxpfupdate /dev/sxpf0 readback.bin -size 4220280 -- read FPGA firmware back
   sxpfupdate README /dev/sxpf0 -offset 0xF00000 -raw -- write file README to 15MB offset
```

#### 2.1.4.3.20 sxpfxo2

sample application

Listing 20: sxpfxo2

```
$ ./sxpfxo2
The sxpfxo2 sample application shows how to update the Lattice MachXO FPGA
on the camera adapter.

Usage: sxpfxo2 <device number> [options] <operation(s)> [<file name>]

Options:
 -b, --bus <number> I2C bus number (default: 0).
 -B, --baud <rate> Set baudrate of I2C bus (default: 100000).
 -d, --device <ID> MachoXO/CrossLink I2C device ID (default: 0x40).

Operations:
 -A, --set-adr <ID> Set I2C Slave Address.
 -J, --jtag-disable <0/1> Enable (0) Disable (1) JTAG Interface.
 -e, --erase Erase the config Flash.
 -h, --help Print this help and exit.
 -i, --initsram Init CrossLink SRAM with given bit file.
 -p, --program Program the MachXO2 with given bit file.
 -r, --refresh Reload FPGA config from Flash as last step.
 -v, --verify Compare the config Flash contents to a bit file.
 -f, --fr-read Read Feature Row.
 -c, --configure Configure the Port-Enables, if not correctly.
 -t, --traceid Read TraceID.
 -w, --write-ini <file name> Write an INI file instead of performing a hardware access. (
     Needs -i ...)
 -y, --retry Troubleshoot by retrying failed commands.
 -Y, --retrycount <n> Number of retries before giving up (default 3).

The options -i, -p and -v require that the file name is given as the
last command line argument.

  MachXO2-Utils Version: 19072201
```

#### 2.1.4.3.21 cap_ng

sample application

Listing 21: cap_ng

```
$ ./cap_ng -h

The cap_ng sample application shows the new proFRAME capture buffer setup API.

options:
  -h|--help Show this help
  -c|--card <num> proFRAME card index (default: 0)
  -j|--jobfile <file> Read JSON file with channel setup
  --dvp Use NVIDIA GPUDirect for Video
  --amd-pinned Use AMD_pinned_memory GL extension
  -s|--shader <shader> Override shader file(s): filename without extension
                          Built-in shaders: bgr, bw, rgb
```

#### 2.1.4.3.22 cucap

sample application

Listing 22: cucap

```
$ ./cucap -h

The cucap sample application shows the new proFRAME capture buffer setup API.

options:
  -c|--channel <num> Capture the given channel (0..7)
  -h|--help Show this help
  --nvrdma Use NVIDIA GPUDirect RDMA
  -s|--shader <shader> Override shader file(s): filename without extension
                          Built-in shaders: bgr, bw, rgb
```

## 2.2 Windows

### 2.2.1 Windows Setup

To install the proFRAME drivers and software API under Windows 7 or Windows 10 the Setup.exe can be used. This setup will install all relevant files including pre build binary tools into the selected folder and also installs the required drivers and the SxpfService.
When updating older installations in some cases the drivers are locked and can not be updated automatically. In this case manually uninstall all Solectrix drivers (see 10), if needed reboot the PC and start the setup again. This setup uses default configurations that match for most systems. If special configurations are needed, every installation step can also be done manually by following the next chapters of this document.

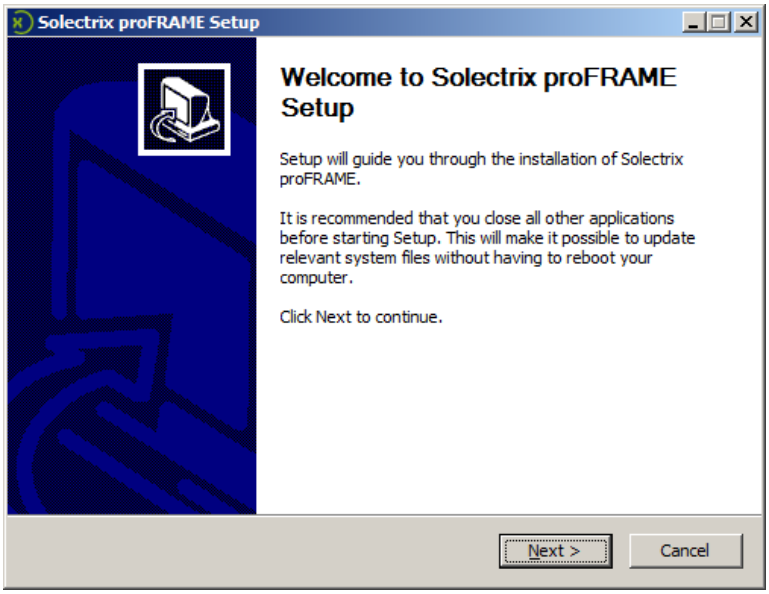



Figure 9: Solectrix Windows Setup

☞ If you are using Windows 10 you must disable the 'fast startup' feature of Windows 10 because this will cause undefined behaviour for the Solectrix proFRAME service. To disable this feature press Windows Key + R and execute powercfg.cpl. Click 'Choose what the power buttons do' and uncheck 'Turn on fast startup'

### 2.2.2 Windows driver installation

To install the windows driver on Windows 7 64-Bit or Windows 10 64-Bit, execute the following steps.
Open a windows command shell, enter the folder redist containing the driver files and execute the following command.

```
> cd .\src\libs\sxpf_ll\win64\redist\
> wdreg.exe -inf driver\sx_pciex4_driver.inf install
```

In Windows 7 a dialog appears asking for confirmation to install unsigned software. Allow installation to continue. The printed output on the shell should look like the following.

```
WDREG utility v12.2.1. Build Jun 16 2016 10:01:27

Processing HWID sx_pciex4
Installing a non-signed driver package for sx_pciex4
LOG ok: 1, ENTER: DriverPackageInstallA
LOG ok: 1, ENTER: DriverPackageInstallW
LOG ok: 2, DRIVER_PACKAGE_LEGACY_MODE flag set but not supported on Plug and Play driver on
    VISTA. Flag will be ignored.
LOG ok: 1, Looking for Model Section [DeviceList.NTamd64]...
LOG ok: 1, Installing INF file 'E:\HIL\sw\win64\redist\sx_pciex4_driver.inf' (Plug and Play)
    .
LOG ok: 1, Looking for Model Section [DeviceList.NTamd64]...
LOG ok: 1, Installing devices with Id "sx_pciex4" using INF "C:\Windows\System32\DriverStore
    \FileRepository\sx_pciex4_driver.inf_amd64_neutral_d05598c28f6bba96\sx_pciex4_driver.inf
    ".
LOG ok: 1, ENTER UpdateDriverForPlugAndPlayDevices...
LOG ok: 0, RETURN UpdateDriverForPlugAndPlayDevices.
LOG ok: 1, Installation was successful.
LOG ok: 0, Install completed
LOG ok: 1, RETURN: DriverPackageInstallW (0x0)
LOG ok: 1, RETURN: DriverPackageInstallA (0x0)
 difx_install_preinstall_inf: err 0, last event 0, last error 0. SUCCESS
install: completed successfully
```

Now install the second part by executing:

```
> cd .\src\libs\sxpf_ll\win64\redist\
> wdreg.exe -inf library\sx_pciex4_device.inf install
```

Again, in Windows 7, in the confirmation dialog the installation should be allowed to continue, resulting in output on the shell akin to the following

```
WDREG utility v12.2.1. Build Jun 16 2016 10:01:27

Processing HWID PCI\VEN_1556&DEV_5555&SUBSYS_55551556&REV_00
Installing a non-signed driver package for PCI\VEN_1556&DEV_5555&SUBSYS_55551556&REV_00
Device node (hwid:PCI\VEN_1556&DEV_5555&SUBSYS_55551556&REV_00): exists and is configured.
    Installing.
CM_Get_DevNode_Status ret value 0, status 180200a, problem 0
LOG ok: 1, ENTER: DriverPackageInstallA
LOG ok: 1, ENTER: DriverPackageInstallW
LOG ok: 2, DRIVER_PACKAGE_LEGACY_MODE flag set but not supported on Plug and Play driver on
    VISTA. Flag will be ignored.
```

```
LOG ok: 1, Looking for Model Section [DeviceList.NTamd64]...
LOG ok: 1, Installing INF file 'E:\HIL\sw\win64\redist\sx_pciex4_device.inf' (Plug and Play)
    .
LOG ok: 1, Looking for Model Section [DeviceList.NTamd64]...
LOG ok: 1, Installing devices with Id "PCI\VEN_1556&DEV_5555&SUBSYS_55551556&REV_00" using
    INF "C:\Windows\System32\DriverStore\FileRepository\sx_pciex4_device.
    inf_amd64_neutral_746f3cd3f32befed\sx_pciex4_device.inf".
LOG ok: 1, ENTER UpdateDriverForPlugAndPlayDevices...
LOG ok: 0, RETURN UpdateDriverForPlugAndPlayDevices.
LOG ok: 1, Installation was successful.
LOG ok: 1, Marked Phantom Device with Hardware/Compatible Id 'PCI\VEN_1556&DEV_5555&
    SUBSYS_55551556&REV_00' for reinstall on next plug-in.
LOG ok: 1, Marked Phantom Device with Hardware/Compatible Id 'PCI\VEN_1556&DEV_5555&
    SUBSYS_55551556&REV_00' for reinstall on next plug-in.
LOG ok: 1, Marked Phantom Device with Hardware/Compatible Id 'PCI\VEN_1556&DEV_5555&
    SUBSYS_55551556&REV_00' for reinstall on next plug-in.
LOG ok: 1, Marked Phantom Device with Hardware/Compatible Id 'PCI\VEN_1556&DEV_5555&
    SUBSYS_55551556&REV_00' for reinstall on next plug-in.
LOG ok: 1, Marked Phantom Device with Hardware/Compatible Id 'PCI\VEN_1556&DEV_5555&
    SUBSYS_55551556&REV_00' for reinstall on next plug-in.
LOG ok: 0, Install completed
LOG ok: 1, RETURN: DriverPackageInstallW (0x0)
LOG ok: 1, RETURN: DriverPackageInstallA (0x0)
 difx_install_preinstall_inf: err 0, last event 0, last error 0. SUCCESS
install: completed successfully
```

In order for the grabber cards to be ready to use the PC system must be restarted. After that the grabber card should be displayed in the Windows device manager (see 10)

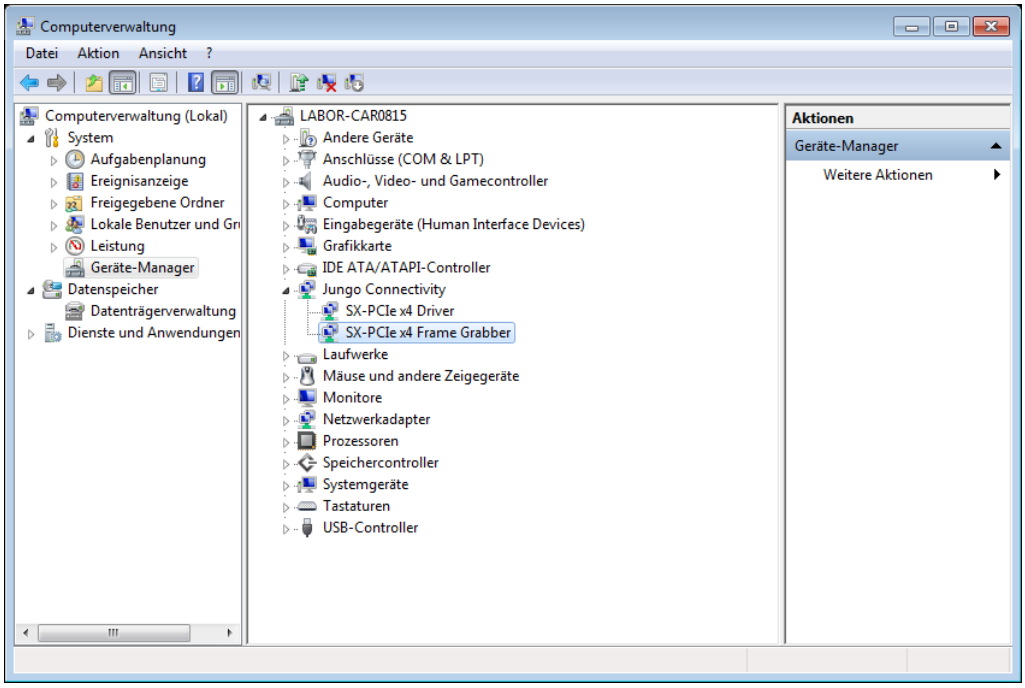



Figure 10: Windows device manager

If the grabber card is not detected it might be shown under System devices as PCI standard RAM Controller. If this occues change the driver of this device to SX-PCIe x4 Frame Grabber.

### 2.2.3 Hardware Interface Service

The grabber hardware is managed by a dedicated Windows Service executable. It is responsible for allocating video frame buffers, setting up DMA transfers, handling interrupts and processing client application requests.

The service executable is installed together with the driver in the Windows system directory, but it can also be found in the redistributable folder, which also contains the Windows kernel driver.

#### 2.2.3.1 Running the Service manually

To start the service, open a windows command shell with administrator privileges[2] and execute the following command.

```
> SxpfService run
```

To stop the hardware service, just press the `ENTER` key in its console window.

#### 2.2.3.2 Installation as a Windows Service

The hardware service can also be installed as a Windows service, which runs automatically when Windows starts:

```
> SxpfService install
> sc qc sxpf
```

To start or stop the service manually (e.g., directly after installation without having rebooted), use the Windows service manager GUI or execute the following commands:

```
> sc start sxpf
> sc stop sxpf
```

To uninstall the service from the system, execute to following:

```
> SxpfService remove
```

#### 2.2.3.3 Service Options

The hardware service understands a few options to modify its behaviour:

- `--debug 0..5`
  Set debug output verbosity.
- `--debug_irq 0..5`
  Set interrupt handler's debug output verbosity.
- `--framesize <size in bytes>`
  Buffer size allocated for a single video frame. The total frame buffer spans 128MBytes. It is divided into at most 31 individual frames. The default frame size is 8MB. For proFRAME 2.0 each camera should have at least 2 frame buffers. Because proFRAME 3.0 does not have hardware frame buffers at least 3 frame buffers should be used. The current number of frame buffers can be checked using the sxpfinfo tool.
- `--log <file name>`
  Set the log file name. The default is `"D:\sxpf.log"`.

These options are applicable to both manual and Windows service modes of operation, i.e., they can be used together with the `run` and `install` sub-commands.
The service options must be added in the following format.

```
> SxpfService install --framesize 16777216
```

[2] Press START, type cmd, hit SHIFT-CTRL-ENTER.

### 2.2.4 Windows Software Applications

The Windows library contains various applications to control the grabber system. This tools can also be used as sample code to integrate similar functions into the customer software tools.

#### 2.2.4.1 Building the Windows applications

To build the Windows applications simply open the file .\src\win64\sxpf-lib.sln with Microsoft Visual Studio 2013 or later. After that simply build the required utils (see 11).

> ☞ The Solectrix proFRAME API only works with C++11 or later. So Microsoft Visual Studio 2013 or later must be used!

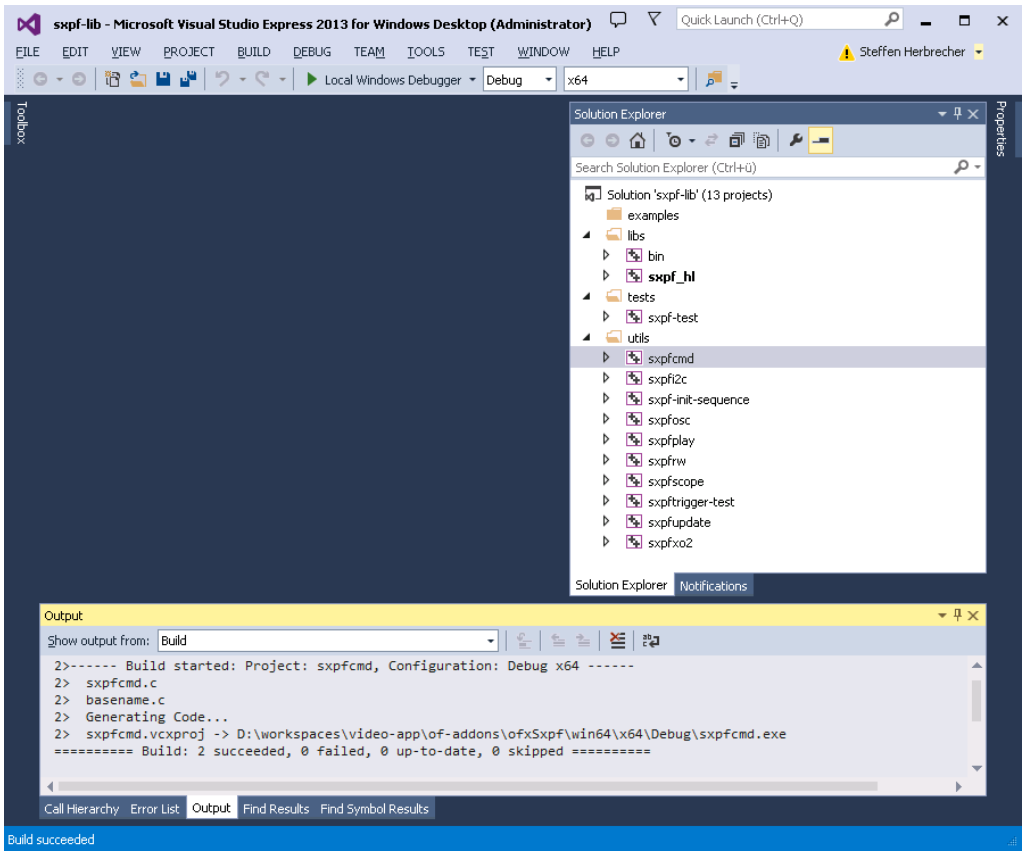



Figure 11: Microsoft Visual Studio 2013 - sxpf-lib.sln

#### 2.2.4.2 Windows applications overview

see Linux applications overview

## 2.3 Graphical User Application - sxpfpygui

sxpfpygui is a simple Python based graphical user application to control the proFRAME system. It supports a subset of the existing command line applications and provides the user a graphical way to configure the application parameters. Also it provides the user an easy way to add project specific tabs to realize customized functions. The sxpfpygui is included in each software release.

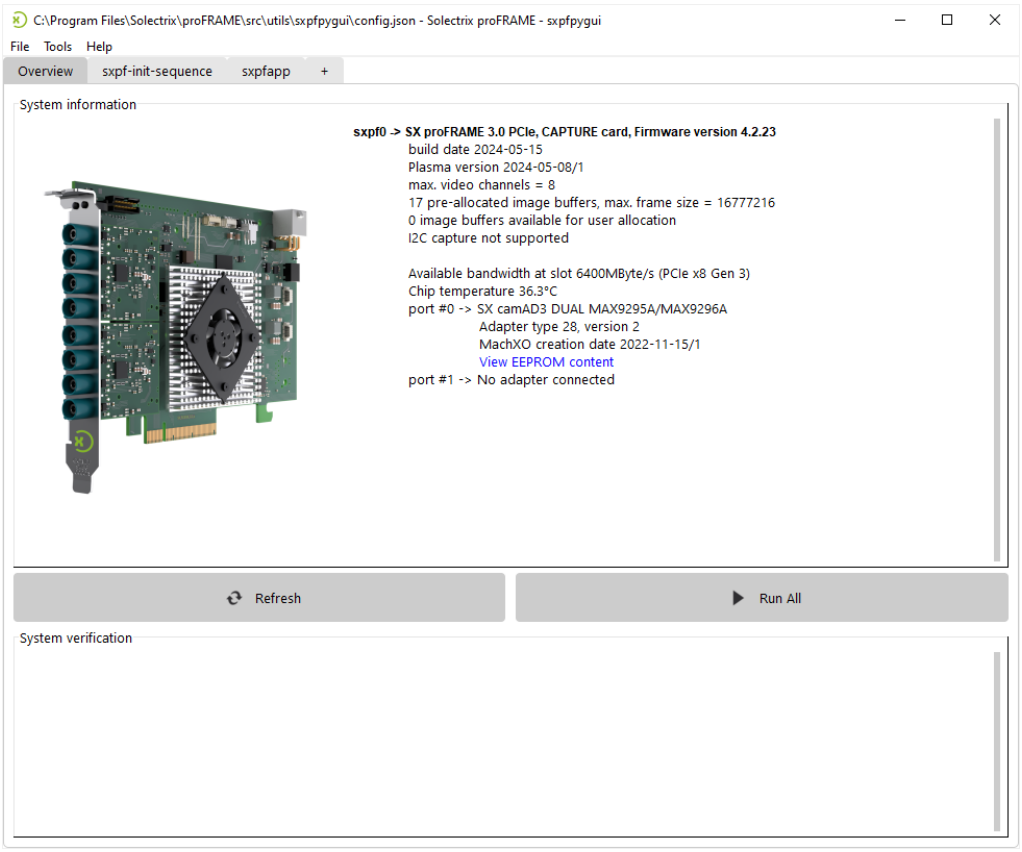



Figure 12: Graphical User Application - sxpfpygui

# 3 Update

To update the complete grabber system to the latest version the following steps are required.

## 3.1 Video grabber card

### 3.1.1 Grabber card firmware version

To check the current firmware of the grabber card execute the sxpfinfo application.

### 3.1.2 Firmware Update

The firmware of the video grabber card can be updated using the sxpfupdate tool. Use the following command sequence to update the video card. The output should look similar to this example. If any errors occure during update DO NOT power off the PC until the problem is resolved. If the update finishes successful shut down the PC and power up again.

Listing 23: Linux Firmware Update

```
$ sudo ./src/release/load_sxpf.sh
$ ./src/release/sxpfupdate CARD_UPDATE.[bin|bit] /dev/sxpf0

flash-id : 0x2019bb
flash-size : 67108864
  sector-size : 131072
  page-size : 256
file-size : 7044168
bit-file : no
ZU4 bin-file : yes
flash to address : 0x0

wrote 7044168 of 7044168 bytes


!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
!!! Update finished !!!
!!! Now power off and power on your system !!!
!!! Reboot is not enough !!!
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
```

There are different firmware branches available to support various use cases. The first number indicates if the card should be used for capturing or replaying. The second number must match to the use case and camera adapters you have.

## 3.2 Camera adapter

### 3.2.1 Camera adapter firmware version

To check the current firmware of the camera adapter execute the sxpfinfo application.

### 3.2.2 Firmware Update

The firmware of the camera adapters can be updated using the sxpfxo2 tool. Before updating the firmware check the binary files using md5sum. After that load the kernel drivers and write the binary file to the camera adapter. Use the following command sequence to update camera adapter 0. Change the bus parameter -b to 0, 1, 2 or 3 to update another camera adapter. The output should look similar to this example. If any errors occure during update DO NOT power off the PC until the problem is resolved. If the update finishes successful shut down the PC and power up again.

```
$ md5sum -c ADAPTER_UPDATE.md5sum
ADAPTER_UPDATE.bit: OK
$ sudo ./src/release/load_sxpf.sh
$ ./src/release/sxpfxo2 0 -b 0 -c -p ADAPTER_UPDATE.bit

  File name: ADAPTER_UPDATE.bit
  File Size: 37964

  Lattice Semiconductor Corporation Bitstream
  Version: Diamond (64-bit) 3.7.1.502
  Bitstream Status: Final Version 1.6
  Design name: diamond_lcmxo3_4300_edif.ncd
  Architecture: xo3c00f
  Part: LCMXO3LF-4300C-6CABGA324
  Date: Mon Jun 20 16:17:19 2016
  Rows: 623
  Cols: 1560
  Bits: 971880
  Readback: Off
  Security: Off
  Bitstream CRC: 0x4D19

  Header-size: 345

setup Port-Enables
Read Device ID: 61 2b c0 43
Enable config access: OK
Erase config Flash: OK
Reset Flash address: OK
Write Flash Page: OK
rep. count: 2372
Program DONE bit: OK
Disable config access: OK
restore Port-Enables
Done
```

☞ For camera adapters containing a Lattice CrossLink the programming device ID must be manually changed to 0x70 to update the Lattice MachXO3 FPGA.

```
$ ./src/release/sxpfxo2 0 -d 0x70 -b 0 -c -p ADAPTER_UPDATE.bit
```

| Firmware Version Branch | Description |
|---|---|
| 3.1.x | proFRAME 2.0 Capture firmware for adapters with 16bit parallel interface (e.g. TI913/914) |
| 3.5.x | proFRAME 2.0 Capture firmware for dual adapters with 16bit parallel interface (e.g. TI914 Dual) |
| 3.7.x | proFRAME 2.0 Capture firmware for LVDS adapters with 12 differential lanes (e.g. MAX9295/9296) |
| 3.8.x | proFRAME 2.0 Capture firmware for LVDS adapters with 12 differential lanes (e.g. MAX9295/9296). Only one LVDS adapter allowed on slot 0. The FPGA maps the CSI-2 packages based on the virtual channel from 0-4 to the image streams 0-4. |
| 3.9.x | proFRAME 2.0 Capture firmware for LVDS adapters with 12 differential lanes (e.g. MAX9295/9296). Only two LVDS adapters allowed on slot 0 and 3. The FPGA maps the CSI-2 packages based on the virtual channel from slot 0 to image streams 0 or 1 and from slot 3 to image stream 2 or 3. Mapping can be configured based on FPGA register 0x468. |
| 3.10.x | proFRAME 2.0 Capture firmware for adapters with 32bit parallel interface (e.g. TI953/954) |
| 3.11.x | proFRAME 2.0 Capture firmware for dual adapters with 32bit parallel interface (e.g. TI954) |
| 9.1.x | proFRAME 2.0 Replay firmware for adapters with 16bit parallel interface (e.g. TI913/914) |
| 9.5.x | proFRAME 2.0 Replay firmware for dual adapters with 16bit parallel interface (e.g. TI953 Dual) |
| 9.7.x | proFRAME 2.0 Replay firmware for LVDS adapters with 12 differential lanes (e.g. MAX9295/9296) |
| 9.10.x | proFRAME 2.0 Replay firmware for adapters with 32bit parallel interface (e.g. TI953/954) |
| 4.1.x | proFRAME 3.0 cPCI-S Capture firmware for CSI-2 based adapters (e.g. camAD3 MAX9296A dual) |
| 4.2.x | proFRAME 3.0 PCIe Capture firmware for CSI-2 based adapters (e.g. camAD3 MAX9296A dual) |
| 4.3.x | proFRAME 3.0 PCIe Capture firmware for oLDI based adapters (e.g. camAD3 MAX96878 dual) |
| 4.4.x | proFRAME 3.0 PCIe TAP firmware for CSI-2 based adapters (e.g. camAD3 MAX9295/9296A dual) |
| 4.5.x | proFRAME 3.0 PCIe Replication firmware for CSI-2 based adapters (e.g. camAD3 MAX9295/9296A dual). The data received on CSI-2 interface 0 is forwarded to all CSI-2 output interfaces. This allows e.g. sending the video data from one camera to multiple ECUs. |
| 4.9.x | proFRAME 3.0 PCIe Capture firmware for CSI-2 based adapters (e.g. camAD3 TI954 dual). For adapters with only 2 instead of 4 CSI-2 interfaces. Both CSI-2 interfaces from the adapter are each mapped to two FPGA video channels. Using virtual channel filters it is possible to seperated different virtual channels into different FPGA video channels |
| 4.10.x | proFRAME 3.0 cPCI-S Capture firmware for CSI-2 based adapters (e.g. camAD3 TI954 dual). For adapters with only 2 instead of 4 CSI-2 interfaces. Both CSI-2 interfaces from the adapter are each mapped to two FPGA video channels. Using virtual channel filters it is possible to seperated different virtual channels into different FPGA video channels |
| 4.14.x | proFRAME 3.0 PCIe TAP firmware for CSI-2 based adapters (e.g. camAD3 MAX9295/9296A dual). The TAP forwarding is done from adapter 0 to adapter 1 and vise versa. So it is possible to do an interface conversion e.g. from GMSL2 to FPD-Link III |
| 10.1.x | proFRAME 3.0 cPCI-S Replay firmware for CSI-2 based adapters (e.g. camAD3 MAX9295/9296A dual) |
| 10.2.x | proFRAME 3.0 PCIe Replay firmware for CSI-2 based adapters (e.g. camAD3 MAX9295/9296A dual) |

Table 3: Video grabber card firmware version overview

# 4 Initialization

## 4.1 Initialization sequences

The initialization sequence files contain the required configuration data to setup the capture system and the cameras. These files can be manually executed by running the sxpf-init-sequence application in the driver directory. E.g.

```
./src/release/sxpf-init-sequence /dev/sxpf0 -port [CAMERA_ADAPTER] -ini EXAMPLE.ini --
    execute [SEQUENCE-NUMBER]
```

Alternatively the complete initialization sequence can be written into the EEPROM of the camera adapter. This allows the video card to automatically execute an initialization sequence on power up, FPGA reset or other predefined events. The sequence can be written by the following command.

```
./src/release/sxpf-init-sequence /dev/sxpf0 -port [CAMERA_ADAPTER] -ini EXAMPLE.ini --write
```

## 4.2 Initialization sequence file format

The initialization sequence files are standard text files in the INI format. They contain different predefined sections and commands described in the following. By writing a # the rest of the current line will be interpreted as a comment text.

### 4.2.1 Sections

| Section | Description |
|---|---|
| [aliases] | In this section custom names for event numbers, command number and I2C slave ids can be defined. Also names for command parameter values can be defined. This section simply helps to make the sequence file more readable. The names will automatically be replaced by the corresponding number when executing the sxpf-init-sequence application. Example:<br><br>`[aliases]`<br>`# The predefined power on reset event has id 0x00.`<br>`# In the INI file the name EVT_POR will be used.`<br>`EVT_POR = 0x00`<br><br>`# The predefined command for waiting a specific time`<br>`# is 0x02.`<br>`# In the INI file the name CMD_WAIT_MILLIS will be`<br>`# used.`<br>`CMD_WAIT_MILLIS = 0x02`<br><br>`# The serializer in the camera has the I2C id 0xB0.`<br>`# In the INI file the name ADR_SERIALIZER will be used.`<br>`ADR_SERIALIZER = 0xB0`<br><br>The following extract should be used as default aliases.<br><br>`# i2c address of camera adapter FPGA`<br>`ADR_MACHXO2 = 0x84` |

```
# command (arg: 0 bytes) --> next 128 bytes cached
CMD_INVALIDATE_CACHE = 0x01
# command (arg: 1,2,3 or 4 bytes)
CMD_WAIT_MILLIS = 0x02
# command (arg: 8 bytes)
# -> [addr + (ch*ch_shift)] = value
CMD_WRITE_REGISTER = 0x03
# command (arg: 1,2,3 or 4 bytes)
CMD_I2C_BAUDRATE = 0x04
# command (arg: 1 byte)
CMD_I2C_RETRYCOUNT = 0x05
# command (arg: 1 byte)
CMD_READ_XO2_EVENTS = 0x06
# command (arg: 1 byte)
CMD_CLEAR_XO2_EVENTS = 0x07
# command (arg: 0 byte)
CMD_SEND_EVENT2HOST = 0x08
# command (arg: 0 byte)
CMD_SEND_EVENT2CPU = 0x09
# command (arg: 0 byte)
CMD_ENABLE_XO2_EVENTS= 0x0C

# event-id: PowerOnReset
EVT_POR = 0x00
# event-id: SoftWareReset
EVT_SWR = 0x01
# event-id: MachXO2 detects something -> must read out
EVT_EXTERNAL_XO2_IRQ = 0x02
# virtual-event-id: PowerDetectECU
EVT_PWR_ECU = 0x32
# virtual-event-id: PowerDetectCamera
EVT_PWR_CAM = 0x33
# virtual-event-id: PowerDeserializerParity
EVT_DES_PARITY = 0x34
# virtual-event-id: PowerDeserializerLocked
EVT_DES_LOCK = 0x35
```

[event-list]

This section defines which command lists will be executed when an event occurs. It is possible to define multiple command lists for a single event. In this case the command lists will be executed in the given order. Example:

```
[event-list]
# On power up the video card executes the command list
# [init] and after that the command list [setup_camera]
EVT_POR = [init]
EVT_POR = [setup_camera]
# On an soft reset (FPGA reset) the video card executes
# only the command list [setup_camera]
EVT_SWR = [setup_camera]
```

| | |
|---|---|
| [COMMAND_LIST_XYZ] | Additionally to the predefined sections described above multiple command lists can be defined that contain a list of commands that will be executed in order. Usually different command lists for e.g. camera adapter/deserializer initialization e.g. [init] and camera serializer/sensor initialization e.g. [setup_camera] should be defined that contain all required commands and I2C messages to initialize the capture system. |

### 4.2.2 Events

| Id | Default alias | Description |
|---|---|---|
| 0x00 | EVT_POR | Power on reset event<br><br>This event is executed when the video card is powered on. |
| 0x01 | EVT_SWR | Software reset event<br><br>This event is executed when the FPGA on the video card receives a reset command from the software API. This also occurs when loading the Linux driver when running the command sudo ./load_sxpf.sh |
| 0x02 | EVT_EXTERNAL_XO2_IRQ | Camera adapter interrupt<br><br>Unknown interrupt on the camera adapter occurred. In this case the interrupt reason must be read from the FPGA on the camera adapter to get more detailed informations. |
| 0x32 | EVT_PWR_ECU | ECU power detected interrupt<br><br>This interrupt indicates that a POC ECU has been connected to the serializer of the camera adapter. |
| 0x33 | EVT_PWR_CAM | Camera power detected interrupt<br><br>When powering the camera via POC this interrupt indicates that the voltage has been enabled and there is no short circuit on the deserializer of the camera adapter. |
| 0x34 | EVT_DES_PARITY | Deserializer parity interrupt<br><br>This interrupt indicates that a parity error occurred on the deserializer |
| 0x35 | EVT_DES_LOCK | Deserializer lock interrupt<br><br>This interrupt indicates that the deserializer has locked or lost connection to a connected serializer. |

### 4.2.3 Commands

In the I2C protocol specification the addresses 0x00 to 0x0F are reserved for special use. For the initialization sequence files this reserved id are used for custom commands. Command ids from 0x10 will be automatically interpreted as an I2C write command to an I2C slave. A command generally has the following format:

```
COMMAND_ID PARAMETER_SIZE_IN_BYTES COMMAND_PARAMETERS
```

Example:

```
# command 0x02 with the parameter 1000. The parameter size is 3 Bytes.
0x02 3 1000
```

| Id | Default alias | Parameter | Description |
|---|---|---|---|
| 0x01 | CMD_INVALIDATE_CACHE | | Invalidate cache command<br><br>When configuring e.g. the imager in the camera the video card must change the I2C channel and the I2C EEPROM on the camera adapter is no longer readable because it is connected to a different I2C channel. To allow configuring the imager nevertheless the video card caches 128 Bytes of the command list. If more than 128 Bytes are needed to configure the imager the video card must switch back to the I2C channel of the EEPROM. Run an invalidate cache command to read the next 128 Bytes and continue the imager configuration. This command is only necessary when storing the initialization sequence in the EEPROM. If the INI file is only executed by the sxpf-init-sequence application cache management is not needed. Example:<br><br>`# invalidate cache command has no parameters`<br>`0x01 0` |
| 0x02 | CMD_WAIT_MILLIS | Sleep time | Wait x milliseconds command<br><br>The sleep time can be entered as an integer number. The parameter size must match to the sleep time. E.g. max 256 milliseconds fit into 1 Byte. Max 65536 milliseconds fit into 2 Bytes. Example:<br><br>`# wait 1 second`<br>`0x02 2 1000` |
| 0x03 | CMD_WRITE_REGISTER | Address<br>Value<br>[Offset] | Write FPGA register command<br><br>This command writes the value into the register address of the FPGA on the video card. Optionally an offset can be defined that is added depending on the adapter number. Example:<br><br>`# write value 0x12345678 to register 0x8000000`<br>`0x03 8 0x8000000 0x12345678`<br>`# write value 0x12345678 to`<br>`# register 0x8000000 for camera adapter 0`<br>`# register 0x8000004 for camera adapter 1`<br>`# register 0x8000008 for camera adapter 2`<br>`# register 0x800000c for camera adapter 3`<br>`0x03 12 0x8000000 0x12345678 0x0000004` |
| 0x04 | CMD_I2C_BAUDRATE | Baudrate | Set I2C baudrate command |

| | | | |
|---|---|---|---|
| | | | Example:<br>`# set I2C baudrate to 10 kBaud`<br>`0x04 2 10000` |
| 0x05 | CMD_I2C_RETRYCOUNT | Retry count | Set I2C retry count command<br><br>When configured, the video card retries the I2C write command up to the given number, if an error occurs during communication. Example:<br>`# retry max 3 times on error`<br>`0x05 1 3` |
| 0x06 | CMD_READ_XO2_EVENTS | I2C address | Read adapter FPGA interrupt reason<br><br>This command reads the interrupt reasons of the camera adapter FPGA. The parameter is the I2C address of the FPGA on the camera adapter. Example:<br>`# read interrupt reason from camera adapter.`<br>`# I2C slave address is 0x84`<br>`0x06 1 0x84` |
| 0x07 | CMD_CLEAR_XO2_EVENTS | I2C address | Clear adapter FPGA interrupt reason<br><br>This command clears the interrupt reasons of the camera adapter FPGA. The parameter is the I2C address of the FPGA on the camera adapter. Example:<br>`# clear interrupt reason from camera adapter.`<br>`# I2C slave address is 0x84`<br>`0x07 1 0x84` |
| 0x08 | CMD_SEND_EVENT2HOST | | Send interrupt notification to host pc<br><br>This command sends an interrupt notification with some details to the host pc. Example:<br>`# no parameters needed`<br>`0x08 0` |
| 0x09 | CMD_SEND_EVENT2CPU | | Send interrupt notification to next interrupt handler |

| | | | |
|---|---|---|---|
| | | | This command sends an interrupt notification to the next interrupt handler in the FPGA softcore for more detailed interrupt handling. Example:<br>`# no parameters needed`<br>`0x09 0` |
| 0x0C | CMD_ENABLE_XO2_EVENTS | | Enable event handling on adapter FPGA<br><br>This command enables the event handling in the adapter FPGA. This has to be done at the first initialization and at the end of the interrupt handler section. Example:<br>`# no parameters needed`<br>`0x0C 0` |

### 4.2.4 I2C commands

The most important part of the INI files are the I2C commands. By using I2C commands the deserializer on the camera adapter and the serializer and image sensor in the camera can be configured. A I2C command generally has the following format:

```
I2C_SLAVE_ID NUMBER_OF_BYTES_TO_WRITE I2C_DATA_BYTES
```

Example:

```
# write value 0x7B to register 0x00 of I2C slave with ID 0x60.
# Number of bytes to write is 2.
0x60 2 0x00 0x7B # set id to 0x7A

# alternatively if an alias for this I2C slave ID is defined the following can be used.
ADR_DESERIALIZER 2 0x00 0x7B # set id to 0x7A
```

### 4.2.5 Simple INI Example

The following shows a simple example of an INI file where the image sensor is automatically configured by an EEPROM in the camera. So there is no need to manually initialize the sensor using the INI.

Listing 24: Simple INI example

```
#! sxproframe-init-sequence, version=1.0
#
# (c) 2015 SX
# This file contains the init sequence of the periphery.
#
# Note:
# -Numbers can be written in Hex (0x..), Bin (0b..) and Dec.
# -Numbers can be 8Bit, 16Bit, 24Bit and 32Bit (etc. 0x11223344)
# and are interpreted as byte-sequence in little-endian (0x44 0x33 0x22 0x11)
# -Integers > 0xFFFFFF interpreted as 4 bytes
# -Integers > 0xFFFF interpreted as 3 bytes
# -Integers > 0xFF interpreted as 2 bytes
# -Warn: Aliases support only single-byte values! etc. 0x1234 is not possible!
#
[aliases]
EVT_POR = 0x00 # event-id
```

```
EVT_SWR = 0x01 # event-id

CMD_WAIT_MILLIS = 0x02 # command (arg: 1,2,3 or 4 bytes)
CMD_WRITE_REGISTER = 0x03 # command (arg: 8 bytes)
CMD_I2C_BAUDRATE = 0x04 # command (arg: 1,2,3 or 4 bytes)

ADR_MACHXO = 0x84 # address: machxo slave
ADR_POTI = 0x5E # address: poti
ADR_SERIALIZER = 0x80 # address: serializer
ADR_DESERIALIZER = 0x90 # address: deserializer

[event-list]
EVT_POR = [init]
EVT_SWR = [init]

[init]
CMD_I2C_BAUDRATE 3 100000 # 100.000 Baud
CMD_WRITE_REGISTER 12 0x00000020 0x00000002 0x00000004 # enable internal i2c channel
ADR_POTI 2 0x02 0x70 # set poti to 5.4V (with camera load 5.1V)

CMD_WRITE_REGISTER 12 0x00000020 0x00000004 0x00000004 # enable machxo i2c channel
ADR_MACHXO 3 0x01 0x00 0x00 # disable deserializer on i2c hub
CMD_WAIT_MILLIS 2 500
ADR_MACHXO 3 0x01 0x00 0x0D # enable deserializer on i2c hub
ADR_MACHXO 3 0x01 0x06 0x01 # invert vsync
CMD_WAIT_MILLIS 2 500

CMD_WRITE_REGISTER 12 0x00000020 0x00000001 0x00000004 # enable external i2c channel
CMD_I2C_BAUDRATE 3 320000 # 320.000 Baud
ADR_SERIALIZER 2 0x04 0x43 # enable configuration link
CMD_WAIT_MILLIS 2 500 # Wait 500ms
ADR_SERIALIZER 2 0x07 0x84 # double input mode
CMD_WAIT_MILLIS 2 500 # Wait 500ms
ADR_DESERIALIZER 2 0x07 0x84 # double input mode
CMD_WAIT_MILLIS 2 500 # Wait 500ms
ADR_SERIALIZER 2 0x04 0x83 # enable serial link
```

# 5 Camera adapter FPGA - I2C slave interface

The configuration registers of the camera adapter FPGA can be accessed via I2C.

The following I2C message can be used to write the value VAL to register REG.

```
0x84 0x01 0xREG 0xVAL
```

The first byte is the I2C Id of the I2C slave. The second byte is the command type. In this case 0x01 defines a write register command. The third byte must be replaced by a valid register address. The register address width is 8bit and there are 4 write registers available. So 0xREG must be replaced by a valid register address 0x00, 0x01, 0x02 or 0x03. The last byte is the value that should be written.

The following I2C message can be used to read the current value at register REG.

```
0x84 0x05 0xREG
```

The first byte is the I2C ID of the I2C slave. The second byte is the command type. In this case 0x05 defines a read register command. The third byte must be replaced by a valid register address. The register address width is 8bit and there are 4 read registers available. So 0xREG must be replaced by a valid register address

0x00, 0x01, 0x02 or 0x03.

The following extract shows a short example of writing and reading registers in the initialization sequence.

```
# enable adapter FPGA i2c channel
# when using newer camera adapters like MAX9295/9296
# the MachXO FPGA is connected to I2C bus 0x2
# in this case use CMD_WRITE_REGISTER 12 0x00000020 0x00000002 0x00000004
CMD_WRITE_REGISTER 12 0x00000020 0x00000004 0x00000004
# write value 0x0F into adapter FPGA register 0x00
ADR_MACHXO 3 0x01 0x00 0x0F
# write value 0x00 into adapter FPGA register 0x01
ADR_MACHXO 3 0x01 0x01 0x00
# enable internal i2c channel
CMD_WRITE_REGISTER 12 0x00000020 0x00000002 0x00000004
```

The next example shows how to read and write the camera adapter registers with the command line tools.

```
# enable adapter FPGA i2c channel for camera adapter 0
# when using newer camera adapters like MAX9295/9296
# the MachXO FPGA is connected to I2C bus 0x2
# in this case use $ ./sxpfrw 0 0x100020 2
$ ./sxpfrw 0 0x100020 4
# write value 0x0F into adapter FPGA register 0x00 for camera adapter 0
$ ./sxpfi2c 0 0 0x84 0 0x01 0x00 0x0F
# enable adapter FPGA i2c channel for camera adapter 1
$ ./sxpfrw 0 0x100024 4
# read value from adapter FPGA register 0x00 for camera adapter 1
$ ./sxpfi2c 0 1 0x84 1 0x05 0x00
```

The following I2C message can be used to write the value VAL to memory block MEM.

```
0x84 0x02 0xMEM 0xVAL
```

The first byte is the I2C Id of the I2C slave. The second byte is the command type. In this case 0x02 defines a write memory block command. The third byte must be replaced by a valid memory address. The memory address width is 8bit. The last byte is the value that should be written.

The following I2C message can be used to read the current value at memory block MEM.

```
0x84 0x0B 0xMEM
```

The first byte is the I2C Id of the I2C slave. The second byte is the command type. In this case 0x0B defines a read memory block command. The third byte must be replaced by a valid memory address. The register address width is 8bit.

☞ When using a camera adapter with a Lattice CrossLink like MAX9295/9296 also the CrossLink can be accessed using I2C. The communication protocol to read/write registers and memory is equal to the MachXO communication. Only the I2C slave ID has to be changed to 0x86. E.g.

```
0x86 0x01 0xREG 0xVAL
```

## 5.1 camAD MAX96705/96706

| Addr (hex) | Bit(s) | Type | Default | Description |
|---|---|---|---|---|
| 0x00 | 0 | RW | 1 | Deserializer I2C Enable |
| | 1 | RW | 1 | Serializer I2C Enable |
| | 2 | RW | 0 | Serializer Enable |
| | 3 | RW | 0 | Deserializer + Camera Power Enable |
| | 4 | RW | 0 | Camera Power Enable if power from ECU is detected |
| | 5 | R | 0 | ECU power deteced |
| | 6 | R | 0 | Deserializer Parity |
| | 7 | R | 0 | Deserializer Lock |
| 0x01 | 2 | RW | | Deserializer GPI value |
| | 4 | RW | | Serializer GPIO1 value |
| | 5 | RW | | Serializer GPO value |
| 0x02 | 2 | RW | 0 | FPGA Pin to Deserializer GPI is Input (0) / Output (1) |
| | 4 | RW | 0 | FPGA Pin to Serializer GPIO1 is Input (0) / Output (1) |
| | 5 | RW | 0 | FPGA Pin to Serializer GPO is Input (0) / Output (1) |
| 0x04 | 2 | RW | 0 | Route Base Board Trigger Signal to Deserializer GPI |
| | 4 | RW | 0 | Route Base Board Trigger Signal to Serializer GPIO1 |
| | 5 | RW | 0 | Route Base Board Trigger Signal to Serializer GPO |
| 0x06 | 0 | RW | 0 | Invert VSync Signal in capture path |
| | 4 | RW | 0 | Invert VSync Signal in replay path |
| 0x07 | 2 | RW | 0 | Enable trigger signal to proFRAME from Deserializer GPI |
| | 4 | RW | 0 | Enable trigger signal to proFRAME from Serializer GPIO1 |
| | 5 | RW | 0 | Enable trigger signal to proFRAME from Serializer GPO |
| 0x0A | 4 | RW | 0 | Forward Serializer GPIO1 to Deserializer GPI |
| | 5 | RW | 0 | Forward Serializer GPO to Deserializer GPI |
| 0x0C | 2 | RW | 0 | Forward Deserializer GPI to Serializer GPIO1 |
| | 5 | RW | 0 | Forward Serializer GPO to Serializer GPIO1 |
| 0x0D | 2 | RW | 0 | Forward Deserializer GPI to Serializer GPO |
| | 4 | RW | 0 | Forward Serializer GPIO1 to Serializer GPO |

Table 7: MAX96705/96706 camera adapter FPGA I2C slave registers

| Addr (hex) | Bit(s) | Type | Default | Description |
|---|---|---|---|---|
| 0x80 | 7-0 | RW | | Replay delay HSync and data low byte |
| 0x81 | 1-0 | RW | | Replay delay HSync and data high byte |
| 0x82 | 7-0 | RW | | Replay VSync pulse width low bypte |
| 0x83 | 7-0 | RW | | Replay VSync pulse width high byte |
| 0x84 | 0 | RW | 0 | Use data enable pin as hsync |
| 0x85 | 0 | RW | 0 | Drop every second frame |

Table 8: MAX96705/96706 camera adapter FPGA I2C slave memory

MAX96705 Serializer datasheet:
`https://datasheets.maximintegrated.com/en/ds/MAX96705.pdf`
MAX96706 Deserializer datasheet:
`https://datasheets.maximintegrated.com/en/ds/MAX96706.pdf`

## 5.2 camAD MAX9296A and camAD MAX96716A

For register description see MAX9295/9296 adapter

## 5.3 camAD MAX9295/9296 and camAD MAX96716A/96717

| Addr (hex) | Bit(s) | Type | Default | Description |
|---|---|---|---|---|
| 0x00 | 0 | RW | 1 | Deserializer I2C Enable |
| | 1 | RW | 1 | Serializer I2C Enable |
| | 2 | RW | 0 | Serializer Enable |
| | 3 | RW | 0 | Deserializer and Camera Power Enable |
| | 4 | RW | 0 | Camera Power Enable if ECU PoC is detected. If set Bit 3 only enables Deserializer |
| 0x01 | 0 | RW | | Deserializer MFP7 Value |
| | 1 | RW | | Deserializer MFP8 Value |
| | 2 | RW | | Deserializer MFP9 Value |
| | 3 | RW | | Deserializer MFP10 Value |
| | 4 | RW | | Serializer MFP0 Value |
| | 5 | RW | | Serializer MFP3 Value |
| | 6 | RW | | Serializer MFP4 Value |
| | 7 | RW | | Serializer MFP5 Value |
| 0x02 | 0 | RW | 0 | FPGA Pin to Deserializer MFP7 is Input (0) / Output (1) |
| | 1 | RW | 0 | FPGA Pin to Deserializer MFP8 is Input (0) / Output (1) |
| | 2 | RW | 0 | FPGA Pin to Deserializer MFP9 is Input (0) / Output (1) |
| | 3 | RW | 0 | FPGA Pin to Deserializer MFP10 is Input (0) / Output (1) |
| | 4 | RW | 0 | FPGA Pin to Serializer MFP0 is Input (0) / Output (1) |
| | 5 | RW | 0 | FPGA Pin to Serializer MFP3 is Input (0) / Output (1) |
| | 6 | RW | 0 | FPGA Pin to Serializer MFP4 is Input (0) / Output (1) |
| | 7 | RW | 0 | FPGA Pin to Serializer MFP5 is Input (0) / Output (1) |
| 0x04 | 0 | RW | 0 | Route Base Board Trigger Signal to Deserializer MFP7 |
| | 1 | RW | 0 | Route Base Board Trigger Signal to Deserializer MFP8 |
| | 2 | RW | 0 | Route Base Board Trigger Signal to Deserializer MFP9 |
| | 3 | RW | 0 | Route Base Board Trigger Signal to Deserializer MFP10 |
| | 4 | RW | 0 | Route Base Board Trigger Signal to Serializer MFP0 |
| | 5 | RW | 0 | Route Base Board Trigger Signal to Serializer MFP3 |
| | 6 | RW | 0 | Route Base Board Trigger Signal to Serializer MFP4 |
| | 7 | RW | 0 | Route Base Board Trigger Signal to Serializer MFP5 |
| 0x07 | 0 | RW | 0 | Route Signal from Deserializer MFP7 to Base Board |
| | 1 | RW | 0 | Route Signal from Deserializer MFP8 to Base Board |
| | 2 | RW | 0 | Route Signal from Deserializer MFP9 to Base Board |
| | 3 | RW | 0 | Route Signal from Deserializer MFP10 to Base Board |
| | 4 | RW | 0 | Route Signal from Serializer MFP0 to Base Board |
| | 5 | RW | 0 | Route Signal from Serializer MFP3 to Base Board |
| | 6 | RW | 0 | Route Signal from Serializer MFP4 to Base Board |
| | 7 | RW | 0 | Route Signal from Serializer MFP5 to Base Board |

| Addr | Bit(s) | Type | Default | Description |
|---|---|---|---|---|
| 0x08 | 1 | RW | 0 | Forward Deserializer MFP8 to Deserializer MFP7 |
| | 2 | RW | 0 | Forward Deserializer MFP9 to Deserializer MFP7 |
| | 3 | RW | 0 | Forward Deserializer MFP10 to Deserializer MFP7 |
| | 4 | RW | 0 | Forward Serializer MFP0 to Deserializer MFP7 |
| | 5 | RW | 0 | Forward Serializer MFP3 to Deserializer MFP7 |
| | 6 | RW | 0 | Forward Serializer MFP4 to Deserializer MFP7 |
| | 7 | RW | 0 | Forward Serializer MFP5 to Deserializer MFP7 |
| 0x09 | 0 | RW | 0 | Forward Deserializer MFP7 to Deserializer MFP8 |
| | 2 | RW | 0 | Forward Deserializer MFP9 to Deserializer MFP8 |
| | 3 | RW | 0 | Forward Deserializer MFP10 to Deserializer MFP8 |
| | 4 | RW | 0 | Forward Serializer MFP0 to Deserializer MFP8 |
| | 5 | RW | 0 | Forward Serializer MFP3 to Deserializer MFP8 |
| | 6 | RW | 0 | Forward Serializer MFP4 to Deserializer MFP8 |
| | 7 | RW | 0 | Forward Serializer MFP5 to Deserializer MFP8 |
| 0x0A | 0 | RW | 0 | Forward Deserializer MFP7 to Deserializer MFP9 |
| | 1 | RW | 0 | Forward Deserializer MFP8 to Deserializer MFP9 |
| | 3 | RW | 0 | Forward Deserializer MFP10 to Deserializer MFP9 |
| | 4 | RW | 0 | Forward Serializer MFP0 to Deserializer MFP9 |
| | 5 | RW | 0 | Forward Serializer MFP3 to Deserializer MFP9 |
| | 6 | RW | 0 | Forward Serializer MFP4 to Deserializer MFP9 |
| | 7 | RW | 0 | Forward Serializer MFP5 to Deserializer MFP9 |
| 0x0B | 0 | RW | 0 | Forward Deserializer MFP7 to Deserializer MFP10 |
| | 1 | RW | 0 | Forward Deserializer MFP8 to Deserializer MFP10 |
| | 2 | RW | 0 | Forward Deserializer MFP9 to Deserializer MFP10 |
| | 4 | RW | 0 | Forward Serializer MFP0 to Deserializer MFP10 |
| | 5 | RW | 0 | Forward Serializer MFP3 to Deserializer MFP10 |
| | 6 | RW | 0 | Forward Serializer MFP4 to Deserializer MFP10 |
| | 7 | RW | 0 | Forward Serializer MFP5 to Deserializer MFP10 |
| 0x0C | 0 | RW | 0 | Forward Deserializer MFP7 to Serializer MFP0 |
| | 1 | RW | 0 | Forward Deserializer MFP8 to Serializer MFP0 |
| | 2 | RW | 0 | Forward Deserializer MFP9 to Serializer MFP0 |
| | 3 | RW | 0 | Forward Deserializer MFP10 to Serializer MFP0 |
| | 5 | RW | 0 | Forward Serializer MFP3 to Serializer MFP0 |
| | 6 | RW | 0 | Forward Serializer MFP4 to Serializer MFP0 |
| | 7 | RW | 0 | Forward Serializer MFP5 to Serializer MFP0 |
| 0x0D | 0 | RW | 0 | Forward Deserializer MFP7 to Serializer MFP3 |
| | 1 | RW | 0 | Forward Deserializer MFP8 to Serializer MFP3 |
| | 2 | RW | 0 | Forward Deserializer MFP9 to Serializer MFP3 |
| | 3 | RW | 0 | Forward Deserializer MFP10 to Serializer MFP3 |
| | 4 | RW | 0 | Forward Serializer MFP0 to Serializer MFP3 |
| | 6 | RW | 0 | Forward Serializer MFP4 to Serializer MFP3 |
| | 7 | RW | 0 | Forward Serializer MFP5 to Serializer MFP3 |
| 0x0E | 0 | RW | 0 | Forward Deserializer MFP7 to Serializer MFP4 |
| | 1 | RW | 0 | Forward Deserializer MFP8 to Serializer MFP4 |
| | 2 | RW | 0 | Forward Deserializer MFP9 to Serializer MFP4 |
| | 3 | RW | 0 | Forward Deserializer MFP10 to Serializer MFP4 |
| | 4 | RW | 0 | Forward Serializer MFP0 to Serializer MFP4 |
| | 5 | RW | 0 | Forward Serializer MFP3 to Serializer MFP4 |
| | 7 | RW | 0 | Forward Serializer MFP5 to Serializer MFP4 |
| 0x0F | 0 | RW | 0 | Forward Deserializer MFP7 to Serializer MFP5 |
| | 1 | RW | 0 | Forward Deserializer MFP8 to Serializer MFP5 |
| | 2 | RW | 0 | Forward Deserializer MFP9 to Serializer MFP5 |
| | 3 | RW | 0 | Forward Deserializer MFP10 to Serializer MFP5 |
| | 4 | RW | 0 | Forward Serializer MFP0 to Serializer MFP5 |
| | 5 | RW | 0 | Forward Serializer MFP3 to Serializer MFP5 |
| | 6 | RW | 0 | Forward Serializer MFP4 to Serializer MFP5 |

Table 9: MAX9295/9296 camera adapter FPGA I2C slave registers - MachXO3

| Addr (hex) | Bit(s) | Type | Default | Description |
|---|---|---|---|---|
| 0x82 | 0 | RW | 0 | I2C Dummy Slave Enable. The I2C Dummy Slave simply acknowledges I2C requests and can be used to emulate an image sensor on a trivial level. |
| | 7 | RW | | I2C Dummy Slave Reset |
| 0x83 | 7-0 | RW | | I2C Dummy Slave Address |
| 0x88 | 1-0 | RW | | Forward deserializer GPIO to proFRAME (0=MFP7, 1=MFP8, 2=MFP9, 3=MFP10) |
| 0x89 | 0 | RW | 0 | Use MFP2 instead of MFP7 for all registers <0x80 and 0x88 |
| | 1 | RW | 0 | Use MFP3 instead of MFP8 for all registers <0x80 and 0x88 |
| | 2 | RW | 0 | Use MFP4 instead of MFP9 for all registers <0x80 and 0x88 |
| 0x90 | 7-0 | R | | MAX9296 MFP0 rising edge counter (read or write to clear) |
| 0x91 | 7-0 | R | | MAX9296 MFP0 falling edge counter (read or write to clear) |
| 0x92 | 7-0 | R | | MAX9296 MFP1 rising edge counter (read or write to clear) |
| 0x93 | 7-0 | R | | MAX9296 MFP1 falling edge counter (read or write to clear) |
| 0x94 | 7-0 | R | | MAX9296 MFP2 rising edge counter (read or write to clear) |
| 0x95 | 7-0 | R | | MAX9296 MFP2 falling edge counter (read or write to clear) |
| 0x96 | 7-0 | R | | MAX9296 MFP3 rising edge counter (read or write to clear) |
| 0x97 | 7-0 | R | | MAX9296 MFP3 falling edge counter (read or write to clear) |
| 0x98 | 7-0 | R | | MAX9296 MFP4 rising edge counter (read or write to clear) |
| 0x99 | 7-0 | R | | MAX9296 MFP4 falling edge counter (read or write to clear) |
| 0x9a | 7-0 | R | | MAX9296 MFP5 rising edge counter (read or write to clear) |
| 0x9b | 7-0 | R | | MAX9296 MFP5 falling edge counter (read or write to clear) |
| 0x9c | 7-0 | R | | MAX9296 MFP6 rising edge counter (read or write to clear) |
| 0x9d | 7-0 | R | | MAX9296 MFP6 falling edge counter (read or write to clear) |
| 0x9e | 7-0 | R | | MAX9296 MFP7 rising edge counter (read or write to clear) |
| 0x9f | 7-0 | R | | MAX9296 MFP7 falling edge counter (read or write to clear) |
| 0xa0 | 7-0 | R | | MAX9296 MFP8 rising edge counter (read or write to clear) |
| 0xa1 | 7-0 | R | | MAX9296 MFP8 falling edge counter (read or write to clear) |
| 0xa2 | 7-0 | R | | MAX9296 MFP9 rising edge counter (read or write to clear) |
| 0xa3 | 7-0 | R | | MAX9296 MFP9 falling edge counter (read or write to clear) |
| 0xa4 | 7-0 | R | | MAX9296 MFP10 rising edge counter (read or write to clear) |
| 0xa5 | 7-0 | R | | MAX9296 MFP10 falling edge counter (read or write to clear) |

Table 10: MAX9295/9296 camera adapter FPGA I2C slave memory - MachXO3

| Addr (hex) | Bit(s) | Type | Default | Description |
|---|---|---|---|---|

Table 11: MAX9295/9296 camera adapter FPGA I2C slave registers - CrossLink

To check the status of the camera link the following registers of the MAX9295 serializer

- VIDOE_TX2 (0x102, 0x10A, 0x112, 0x11A)
- MIPI_RX17 (0x341)

and the following registers of the MAX9296 deserializer

- VIDOE_RX8 (0x108, 0x11A, 0x12C, 0x13E)
- BACKTOP11 (0x312)

are useful. In the default configuration the Y-Pipe is transfered through CSI-2 Interface #0. In this case register 0x11A should be 0x6X.

| Addr (hex) | Bit(s) | Type | Default | Description |
|---|---|---|---|---|
| 0x83 | 0 | R | | Reserved |
| | 1 | R | | RX LVDS Ready |
| | 2 | R | | RX FIFO Full Sticky |
| | 3 | R | | RX FIFO Empty Flag |
| 0x88 | 0 | R | | TX D-Phy PLL Lock |
| | 1 | R | | TX LVDS Ready |
| | 4 | R | | TX FIFO Empty Flag |
| | 5 | R | | TX FIFO Full Sticky |
| 0x90 | 7-0 | R | | CSI-2 Replay TX Datarate [Mbps per lane] low byte |
| 0x91 | 7-0 | R | | CSI-2 Replay TX Datarate [Mbps per lane] high byte |
| 0xF1 | 0 | R | | D-Phy Reset A disable |
| | 1 | R | | D-Phy Reset B disable |
| | 2 | R | | D-Phy Reset C disable |
| | 3 | R | | D-Phy Reset D disable |
| | 4 | R | | D-Phy Reset E disable |

Table 12: MAX9295/9296 camera adapter FPGA I2C slave memory - CrossLink

MAX9295 Serializer datasheet:
Datasheet is under NDA. Please contact MAXIM for datasheet.
MAX9296 Deserializer datasheet:
Datasheet is under NDA. Please contact MAXIM for datasheet.

## 5.4 camAD TI913/914

| Addr (hex) | Bit(s) | Type | Default | Description |
|---|---|---|---|---|
| 0x00 | 0 | RW | 1 | Deserializer I2C Enable |
| | 1 | RW | 1 | Serializer I2C Enable |
| | 2 | RW | 0 | Camera Power Enable |
| | 3 | RW | 0 | Deserializer Enable |
| | 4 | RW | 0 | Camera Power Enable if power from ECU is detected |
| 0x01 | 0 | RW | | Deserializer GPIO0 value |
| | 1 | RW | | Deserializer GPIO1 value |
| | 2 | RW | | Deserializer GPIO2 value |
| | 3 | RW | | Deserializer GPIO3 value |
| | 4 | RW | | Serializer GPIO0 value |
| | 5 | RW | | Serializer GPIO1 value |
| | 6 | RW | | Serializer GPIO2 value |
| | 7 | RW | | Serializer GPIO3 value |
| 0x02 | 0 | RW | 0 | FPGA Pin to Deserializer GPIO0 is Input (0) / Output (1) |
| | 1 | RW | 0 | FPGA Pin to Deserializer GPIO1 is Input (0) / Output (1) |
| | 2 | RW | 0 | FPGA Pin to Deserializer GPIO2 is Input (0) / Output (1) |
| | 3 | RW | 0 | FPGA Pin to Deserializer GPIO3 is Input (0) / Output (1) |
| | 4 | RW | 0 | FPGA Pin to Serializer GPIO0 is Input (0) / Output (1) |
| | 5 | RW | 0 | FPGA Pin to Serializer GPIO1 is Input (0) / Output (1) |
| | 6 | RW | 0 | FPGA Pin to Serializer GPIO2 is Input (0) / Output (1) |
| | 7 | RW | 0 | FPGA Pin to Serializer GPIO3 is Input (0) / Output (1) |
| 0x04 | 0 | RW | 0 | Route Base Board Trigger Signal to Deserializer GPIO0 |
| | 1 | RW | 0 | Route Base Board Trigger Signal to Deserializer GPIO1 |
| | 2 | RW | 0 | Route Base Board Trigger Signal to Deserializer GPIO2 |
| | 3 | RW | 0 | Route Base Board Trigger Signal to Deserializer GPIO3 |
| | 4 | RW | 0 | Route Base Board Trigger Signal to Serializer GPIO0 |
| | 5 | RW | 0 | Route Base Board Trigger Signal to Serializer GPIO1 |
| | 6 | RW | 0 | Route Base Board Trigger Signal to Serializer GPIO2 |
| | 7 | RW | 0 | Route Base Board Trigger Signal to Serializer GPIO3 |
| 0x06 | 0 | RW | 0 | Invert VSync Signal in capture path |
| | 4 | RW | 0 | Invert VSync Signal in replay path |
| 0x07 | 0 | RW | 0 | Route Signal from Deserializer GPIO0 to Base Board |
| | 1 | RW | 0 | Route Signal from Deserializer GPIO1 to Base Board |
| | 2 | RW | 0 | Route Signal from Deserializer GPIO2 to Base Board |
| | 3 | RW | 0 | Route Signal from Deserializer GPIO3 to Base Board |
| | 4 | RW | 0 | Route Signal from Serializer GPIO0 to Base Board |
| | 5 | RW | 0 | Route Signal from Serializer GPIO1 to Base Board |
| | 6 | RW | 0 | Route Signal from Serializer GPIO2 to Base Board |
| | 7 | RW | 0 | Route Signal from Serializer GPIO3 to Base Board |

| Addr | Bit(s) | Type | Default | Description |
|---|---|---|---|---|
| 0x08 | 1 | RW | 0 | Forward Deserializer GPIO1 to Deserializer GPIO0 |
| | 2 | RW | 0 | Forward Deserializer GPIO2 to Deserializer GPIO0 |
| | 3 | RW | 0 | Forward Deserializer GPIO3 to Deserializer GPIO0 |
| | 4 | RW | 0 | Forward Serializer GPIO0 to Deserializer GPIO0 |
| | 5 | RW | 0 | Forward Serializer GPIO1 to Deserializer GPIO0 |
| | 6 | RW | 0 | Forward Serializer GPIO2 to Deserializer GPIO0 |
| | 7 | RW | 0 | Forward Serializer GPIO3 to Deserializer GPIO0 |
| 0x09 | 0 | RW | 0 | Forward Deserializer GPIO0 to Deserializer GPIO1 |
| | 2 | RW | 0 | Forward Deserializer GPIO2 to Deserializer GPIO1 |
| | 3 | RW | 0 | Forward Deserializer GPIO3 to Deserializer GPIO1 |
| | 4 | RW | 0 | Forward Serializer GPIO0 to Deserializer GPIO1 |
| | 5 | RW | 0 | Forward Serializer GPIO1 to Deserializer GPIO1 |
| | 6 | RW | 0 | Forward Serializer GPIO2 to Deserializer GPIO1 |
| | 7 | RW | 0 | Forward Serializer GPIO3 to Deserializer GPIO1 |
| 0x0A | 0 | RW | 0 | Forward Deserializer GPIO0 to Deserializer GPIO2 |
| | 1 | RW | 0 | Forward Deserializer GPIO1 to Deserializer GPIO2 |
| | 3 | RW | 0 | Forward Deserializer GPIO3 to Deserializer GPIO2 |
| | 4 | RW | 0 | Forward Serializer GPIO0 to Deserializer GPIO2 |
| | 5 | RW | 0 | Forward Serializer GPIO1 to Deserializer GPIO2 |
| | 6 | RW | 0 | Forward Serializer GPIO2 to Deserializer GPIO2 |
| | 7 | RW | 0 | Forward Serializer GPIO3 to Deserializer GPIO2 |
| 0x0B | 0 | RW | 0 | Forward Deserializer GPIO0 to Deserializer GPIO3 |
| | 1 | RW | 0 | Forward Deserializer GPIO1 to Deserializer GPIO3 |
| | 2 | RW | 0 | Forward Deserializer GPIO2 to Deserializer GPIO3 |
| | 4 | RW | 0 | Forward Serializer GPIO0 to Deserializer GPIO3 |
| | 5 | RW | 0 | Forward Serializer GPIO1 to Deserializer GPIO3 |
| | 6 | RW | 0 | Forward Serializer GPIO2 to Deserializer GPIO3 |
| | 7 | RW | 0 | Forward Serializer GPIO3 to Deserializer GPIO3 |
| 0x0C | 0 | RW | 0 | Forward Deserializer GPIO0 to Serializer GPIO0 |
| | 1 | RW | 0 | Forward Deserializer GPIO1 to Serializer GPIO0 |
| | 2 | RW | 0 | Forward Deserializer GPIO2 to Serializer GPIO0 |
| | 3 | RW | 0 | Forward Deserializer GPIO3 to Serializer GPIO0 |
| | 5 | RW | 0 | Forward Serializer GPIO1 to Serializer GPIO0 |
| | 6 | RW | 0 | Forward Serializer GPIO2 to Serializer GPIO0 |
| | 7 | RW | 0 | Forward Serializer GPIO3 to Serializer GPIO0 |
| 0x0D | 0 | RW | 0 | Forward Deserializer GPIO0 to Serializer GPIO1 |
| | 1 | RW | 0 | Forward Deserializer GPIO1 to Serializer GPIO1 |
| | 2 | RW | 0 | Forward Deserializer GPIO2 to Serializer GPIO1 |
| | 3 | RW | 0 | Forward Deserializer GPIO3 to Serializer GPIO1 |
| | 4 | RW | 0 | Forward Serializer GPIO0 to Serializer GPIO1 |
| | 6 | RW | 0 | Forward Serializer GPIO2 to Serializer GPIO1 |
| | 7 | RW | 0 | Forward Serializer GPIO3 to Serializer GPIO1 |
| 0x0E | 0 | RW | 0 | Forward Deserializer GPIO0 to Serializer GPIO2 |
| | 1 | RW | 0 | Forward Deserializer GPIO1 to Serializer GPIO2 |
| | 2 | RW | 0 | Forward Deserializer GPIO2 to Serializer GPIO2 |
| | 3 | RW | 0 | Forward Deserializer GPIO3 to Serializer GPIO2 |
| | 4 | RW | 0 | Forward Serializer GPIO0 to Serializer GPIO2 |
| | 5 | RW | 0 | Forward Serializer GPIO1 to Serializer GPIO2 |
| | 7 | RW | 0 | Forward Serializer GPIO3 to Serializer GPIO2 |
| 0x0F | 0 | RW | 0 | Forward Deserializer GPIO0 to Serializer GPIO3 |
| | 1 | RW | 0 | Forward Deserializer GPIO1 to Serializer GPIO3 |
| | 2 | RW | 0 | Forward Deserializer GPIO2 to Serializer GPIO3 |
| | 3 | RW | 0 | Forward Deserializer GPIO3 to Serializer GPIO3 |
| | 4 | RW | 0 | Forward Serializer GPIO0 to Serializer GPIO3 |
| | 5 | RW | 0 | Forward Serializer GPIO1 to Serializer GPIO3 |
| | 6 | RW | 0 | Forward Serializer GPIO2 to Serializer GPIO3 |

Table 13: TI913/914 camera adapter FPGA I2C slave registers

| Addr (hex) | Bit(s) | Type | Default | Description |
|---|---|---|---|---|
| 0x80 | 7-0 | RW | 0 | Replay delay HSync and data low byte |
| 0x81 | 1-0 | RW | 0 | Replay delay HSync and data high byte |
| 0x82 | 7-0 | RW | 0 | Replay VSync pulse width low bypte |
| 0x83 | 7-0 | RW | 0 | Replay VSync pulse width high byte |

Table 14: TI913/914 camera adapter FPGA I2C slave memory

DS90UB913 Serializer datasheet:
`http://www.ti.com/lit/ds/symlink/ds90ub913a-q1.pdf`
DS90UB914 Deserializer datasheet:
`http://www.ti.com/lit/ds/symlink/ds90ub914a-q1.pdf`

## 5.5 camAD TI933/934

| Addr (hex) | Bit(s) | Type | Default | Description |
|---|---|---|---|---|
| 0x00 | 0 | RW | 1 | Deserializer I2C Enable |
| | 1 | RW | 1 | Serializer I2C Enable |
| | 2 | RW | 0 | Camera Power Enable |
| | 3 | RW | 0 | Deserializer Enable |
| | 4 | RW | 0 | Camera Power Enable if power from ECU is detected |
| 0x01 | 0 | RW | | Deserializer GPIO0 |
| | 1 | RW | | Deserializer GPIO1 |
| | 2 | RW | | Deserializer GPIO2 |
| | 3 | RW | | Deserializer GPIO3 |
| | 4 | RW | | Serializer GPIO0 |
| | 5 | RW | | Serializer GPIO1 |
| | 6 | RW | | Serializer GPIO2 |
| | 7 | RW | | Serializer GPIO3 |
| 0x02 | 0 | RW | | Deserializer GPIO0 Output Enable |
| | 1 | RW | | Deserializer GPIO1 Output Enable |
| | 2 | RW | | Deserializer GPIO2 Output Enable |
| | 3 | RW | | Deserializer GPIO3 Output Enable |
| | 4 | RW | | Serializer GPIO0 Output Enable |
| | 5 | RW | | Serializer GPIO1 Output Enable |
| | 6 | RW | | Serializer GPIO2 Output Enable |
| | 7 | RW | | Serializer GPIO3 Output Enable |
| 0x03 | 3 | RW | | Serializer Enable |
| 0x04 | 0 | RW | | Enable Trigger Signal on Deserializer GPIO0 |
| | 1 | RW | | Enable Trigger Signal on Deserializer GPIO1 |
| | 2 | RW | | Enable Trigger Signal on Deserializer GPIO2 |
| | 3 | RW | | Enable Trigger Signal on Deserializer GPIO3 |
| | 4 | RW | | Enable Trigger Signal on Serializer GPIO0 |
| | 5 | RW | | Enable Trigger Signal on Serializer GPIO1 |
| | 6 | RW | | Enable Trigger Signal on Serializer GPIO2 |
| | 7 | RW | | Enable Trigger Signal on Serializer GPIO3 |

Table 15: TI933/934 camera adapter FPGA I2C slave registers

DS90UB933 Serializer datasheet:
`http://www.ti.com/lit/ds/symlink/ds90ub933-q1.pdf`
DS90UB934 Deserializer datasheet:
`http://www.ti.com/lit/ds/symlink/ds90ub934-q1.pdf`

## 5.6 camAD TI953 Dual

Feature overview

- ICs
  - 2x DS90UB953 serializer
    Serializer 1 default I2C ID 0x30 (8bit)
    Serializer 2 default I2C ID 0x32 (8bit)
    Can be set to power down mode via software
  - 8kBit EEPROM for storing adapter, serdes and camera configurations
- Connectors
  - 1x FAKRA 59S2AQ-40MT5-Z_1 connected to DOUT0 of Serializer 2 (DS90UB953)
  - 1x FAKRA 59S2AQ-40MT5-Z_1 connected to DOUT0 of Serializer 2 (DS90UB953)
- PoC
  - 47Ohm PoC load resistors
  - PoC input voltage detector on serializers. Max. 24V. Threadhold around 1,226V
- I2C
  - 1x I2C bus (0x1) from proFRAME to external devices on serializer 1 (DS90UB953 and ECU)
  - 1x I2C bus (0x2) from proFRAME to internal devices (EEPROM, PoC, MachXO programming)
  - 1x I2C bus (0x4) from proFRAME to MachXO register interface
  - 1x I2C bus (0x8) from proFRAME to external devices on serializer 2 (DS90UB953 and ECU)
- Image capturing and replay
  - 4 lane + CLK CSI-2 from MachXO3 to serializer 1 (max. 800Mbps per lane)
  - 4 lane + CLK CSI-2 from MachXO3 to serializer 2 (max. 800Mbps per lane)
  - Replay of CSI-2 data (RAW mode or with CSI-2 framing generated by adapter)
  - 32bit replay mode where both serializer get the same data
  - 2x16bit replay mode where both serializer get different data
- Side Signals
  - Trigger forwarding from serializer GPIO to proFRAME
- Others
  - 32bit or 2x16bit parallel Interface to SX proFRAME with max 100MHz

| Addr (hex) | Bit(s) | Type | Default | Description |
|---|---|---|---|---|
| 0x00 | 2 | RW | 0 | Serializer 1 and 2 Enable |
| 0x01 | 0 | RW | | Serializer 1 GPIO0 value |
| | 1 | RW | | Serializer 1 GPIO1 value |
| | 2 | RW | | Serializer 1 GPIO2 value |
| | 3 | RW | | Serializer 1 GPIO3 value |
| | 4 | RW | | Serializer 2 GPIO0 value |
| | 5 | RW | | Serializer 2 GPIO1 value |
| | 6 | RW | | Serializer 2 GPIO2 value |
| | 7 | RW | | Serializer 2 GPIO3 value |
| 0x02 | 0 | RW | 0 | Serializer 1 GPIO0 Output Enable (0=Input, 1=Output) |
| | 1 | RW | 0 | Serializer 1 GPIO1 Output Enable (0=Input, 1=Output) |
| | 2 | RW | 0 | Serializer 1 GPIO2 Output Enable (0=Input, 1=Output) |
| | 3 | RW | 0 | Serializer 1 GPIO3 Output Enable (0=Input, 1=Output) |
| | 4 | RW | 0 | Serializer 2 GPIO0 Output Enable (0=Input, 1=Output) |
| | 5 | RW | 0 | Serializer 2 GPIO1 Output Enable (0=Input, 1=Output) |
| | 6 | RW | 0 | Serializer 2 GPIO2 Output Enable (0=Input, 1=Output) |
| | 7 | RW | 0 | Serializer 2 GPIO3 Output Enable (0=Input, 1=Output) |

Table 16: TI953 Dual camera adapter FPGA I2C slave registers

| Addr (hex) | Bit(s) | Type | Default | Description |
|---|---|---|---|---|
| 0x82 | 0 | RW | 0 | I2C Dummy Slave Enable |
| | 7 | RW | | I2C Dummy Slave Reset |
| 0x83 | 7-0 | RW | | I2C Dummy Slave Address |
| 0x87 | 0 | RW | 0 | Enable RAW CSI-2 replay on port 0 |
| | 4 | RW | 0 | Enable RAW CSI-2 replay on port 1 |
| 0x88 | 5-0 | RW | | Replay CSI-2 Datatype on port 0 |
| | 7-6 | RW | | Replay CSI-2 Virtual Channel on port 0 |
| 0x89 | 7-0 | RW | | Replay CSI-2 Word Count Low Byte on port 0 |
| 0x8A | 7-0 | RW | | Replay CSI-2 Word Count High Byte on port 0 |
| 0x8B | 5-0 | RW | | Replay CSI-2 Datatype on port 1 |
| | 7-6 | RW | | Replay CSI-2 Virtual Channel on port 1 |
| 0x8C | 7-0 | RW | | Replay CSI-2 Word Count Low Byte on port 1 |
| 0x8D | 7-0 | RW | | Replay CSI-2 Word Count High Byte on port 1 |
| 0x8E | 7-0 | RW | | Data delay in 10nsec steps to allow sending CSI-2 SOF package |
| 0xA0 | 0 | RW | 0 | 0 = Single Replay of one 32bit stream to both TI953<br>1 = Dual Replay of two different 16bit streams to each TI953 |

Table 17: TI953 Dual camera adapter FPGA I2C slave memory

DS90UB953 Serializer datasheet:
`http://www.ti.com/lit/ds/symlink/ds90ub953-q1.pdf`

## 5.7 camAD TI953/954

Feature overview

- ICs
  - 1x DS90UB954 deserializer
    Default I2C ID 0x60 (8bit)
    Can be set to power down mode via software
  - 1x DS90UB953 serializer
    Default I2C ID 0x30 (8bit)
    Can be set to power down mode via software
  - 8kBit EEPROM for storing adapter, serdes and camera configurations
- Connectors
  - 1x FAKRA 59S2AQ-40MT5-Z_1 connected to RIN0 of DS90UB954
  - 1x FAKRA 59S2AQ-40MT5-Z_1 connected to DOUT0 of DS90UB953
- PoC
  - Software configurable PoC output voltage on DS90UB954 deserializer from 5V to 10V. Max. 200mA.
  - PoC input voltage detector on DS90UB953 serializer max. 24V. Threadhold around 1,226V
- I2C
  - 1x I2C bus (0x1) from proFRAME to external devices (DS90UB953, DS90UB954, camera and ECU)
  - 1x I2C bus (0x2) from proFRAME to internal devices (EEPROM, PoC, MachXO programming)
  - 1x I2C bus (0x4) from proFRAME to MachXO register interface
  - I2C hub to control/connect external I2C to DS90UB953, DS90UB954, camera and ECU. This also allows bypassing of I2C from ECU through camera adapter to the camera.
- Image capturing and replay
  - 4 lane + CLK CSI-2 from DS90UB954 to MachXO3 (max. 800Mbps per lane)
  - 4 lane + CLK CSI-2 from MachXO3 to DS90UB953 (max. 800Mbps per lane)
  - Bypassing of CSI-2 data from DS90UB954 to DS90UB954
  - Capturing of RAW CSI-2 data (CSI-2 header, footer and short packages)
  - Replay of CSI-2 data (RAW mode or with CSI-2 framing generated by adapter)
- Side Signals
  - Configurable GPIO trigger signal routing to any GPIO port
  - Configurable GPIO forwarding from DS90UB953 to DS90UB954 and vise versa
- Others
  - 32bit parallel Interface to SX proFRAME with max 100MHz
  - 25MHz DS90UB954 reference clock (can be enabled/disabled by software)

| Addr (hex) | Bit(s) | Type | Default | Description |
|---|---|---|---|---|
| 0x00 | 0 | RW | 1 | Deserializer I2C Enable |
| | 1 | RW | 1 | Serializer I2C Enable |
| | 2 | RW | 0 | Serializer Enable |
| | 3 | RW | 0 | Deserializer and Camera Power Enable |
| | 4 | RW | 0 | (W) Camera Power Enable if power from ECU is detected<br>(R) Detect if PoC voltage is greater than 4,4V<br>(only for internal PoC supply) |
| 0x01 | 0 | RW | | Deserializer GPIO0 Value |
| | 1 | RW | | Deserializer GPIO1 Value |
| | 2 | RW | | Deserializer GPIO2 Value |
| | 3 | RW | | Deserializer GPIO3 Value |
| | 4 | RW | | Serializer GPIO0 Value |
| | 5 | RW | | Serializer GPIO1 Value |
| | 6 | RW | | Serializer GPIO2 Value |
| | 7 | RW | | Serializer GPIO3 Value |
| 0x02 | 0 | RW | 0 | FPGA Pin to Deserializer GPIO0 is Input (0) / Output (1) |
| | 1 | RW | 0 | FPGA Pin to Deserializer GPIO1 is Input (0) / Output (1) |
| | 2 | RW | 0 | FPGA Pin to Deserializer GPIO2 is Input (0) / Output (1) |
| | 3 | RW | 0 | FPGA Pin to Deserializer GPIO3 is Input (0) / Output (1) |
| | 4 | RW | 0 | FPGA Pin to Serializer GPIO0 is Input (0) / Output (1) |
| | 5 | RW | 0 | FPGA Pin to Serializer GPIO1 is Input (0) / Output (1) |
| | 6 | RW | 0 | FPGA Pin to Serializer GPIO2 is Input (0) / Output (1) |
| | 7 | RW | 0 | FPGA Pin to Serializer GPIO3 is Input (0) / Output (1) |
| 0x04 | 0 | RW | 0 | Route Base Board Trigger Signal to Deserializer GPIO0 |
| | 1 | RW | 0 | Route Base Board Trigger Signal to Deserializer GPIO1 |
| | 2 | RW | 0 | Route Base Board Trigger Signal to Deserializer GPIO2 |
| | 3 | RW | 0 | Route Base Board Trigger Signal to Deserializer GPIO3 |
| | 4 | RW | 0 | Route Base Board Trigger Signal to Serializer GPIO0 |
| | 5 | RW | 0 | Route Base Board Trigger Signal to Serializer GPIO1 |
| | 6 | RW | 0 | Route Base Board Trigger Signal to Serializer GPIO2 |
| | 7 | RW | 0 | Route Base Board Trigger Signal to Serializer GPIO3 |
| 0x07 | 0 | RW | 0 | Route Signal from Deserializer GPIO0 to Base Board |
| | 1 | RW | 0 | Route Signal from Deserializer GPIO1 to Base Board |
| | 2 | RW | 0 | Route Signal from Deserializer GPIO2 to Base Board |
| | 3 | RW | 0 | Route Signal from Deserializer GPIO3 to Base Board |
| | 4 | RW | 0 | Route Signal from Serializer GPIO0 to Base Board |
| | 5 | RW | 0 | Route Signal from Serializer GPIO1 to Base Board |
| | 6 | RW | 0 | Route Signal from Serializer GPIO2 to Base Board |
| | 7 | RW | 0 | Route Signal from Serializer GPIO3 to Base Board |

| Addr (hex) | Bit(s) | Type | Default | Description |
|---|---|---|---|---|
| 0x08 | 4 | RW | 0 | Forward Serializer GPIO0 to Deserializer GPIO0 |
| | 5 | RW | 0 | Forward Serializer GPIO1 to Deserializer GPIO0 |
| | 6 | RW | 0 | Forward Serializer GPIO2 to Deserializer GPIO0 |
| | 7 | RW | 0 | Forward Serializer GPIO3 to Deserializer GPIO0 |
| 0x09 | 4 | RW | 0 | Forward Serializer GPIO0 to Deserializer GPIO1 |
| | 5 | RW | 0 | Forward Serializer GPIO1 to Deserializer GPIO1 |
| | 6 | RW | 0 | Forward Serializer GPIO2 to Deserializer GPIO1 |
| | 7 | RW | 0 | Forward Serializer GPIO3 to Deserializer GPIO1 |
| 0x0A | 4 | RW | 0 | Forward Serializer GPIO0 to Deserializer GPIO2 |
| | 5 | RW | 0 | Forward Serializer GPIO1 to Deserializer GPIO2 |
| | 6 | RW | 0 | Forward Serializer GPIO2 to Deserializer GPIO2 |
| | 7 | RW | 0 | Forward Serializer GPIO3 to Deserializer GPIO2 |
| 0x0B | 4 | RW | 0 | Forward Serializer GPIO0 to Deserializer GPIO3 |
| | 5 | RW | 0 | Forward Serializer GPIO1 to Deserializer GPIO3 |
| | 6 | RW | 0 | Forward Serializer GPIO2 to Deserializer GPIO3 |
| | 7 | RW | 0 | Forward Serializer GPIO3 to Deserializer GPIO3 |
| 0x0C | 0 | RW | 0 | Forward Deserializer GPIO0 to Serializer GPIO0 |
| | 1 | RW | 0 | Forward Deserializer GPIO1 to Serializer GPIO0 |
| | 2 | RW | 0 | Forward Deserializer GPIO2 to Serializer GPIO0 |
| | 3 | RW | 0 | Forward Deserializer GPIO3 to Serializer GPIO0 |
| 0x0D | 0 | RW | 0 | Forward Deserializer GPIO0 to Serializer GPIO1 |
| | 1 | RW | 0 | Forward Deserializer GPIO1 to Serializer GPIO1 |
| | 2 | RW | 0 | Forward Deserializer GPIO2 to Serializer GPIO1 |
| | 3 | RW | 0 | Forward Deserializer GPIO3 to Serializer GPIO1 |
| 0x0E | 0 | RW | 0 | Forward Deserializer GPIO0 to Serializer GPIO2 |
| | 1 | RW | 0 | Forward Deserializer GPIO1 to Serializer GPIO2 |
| | 2 | RW | 0 | Forward Deserializer GPIO2 to Serializer GPIO2 |
| | 3 | RW | 0 | Forward Deserializer GPIO3 to Serializer GPIO2 |
| 0x0F | 0 | RW | 0 | Forward Deserializer GPIO0 to Serializer GPIO3 |
| | 1 | RW | 0 | Forward Deserializer GPIO1 to Serializer GPIO3 |
| | 2 | RW | 0 | Forward Deserializer GPIO2 to Serializer GPIO3 |
| | 3 | RW | 0 | Forward Deserializer GPIO3 to Serializer GPIO3 |

Table 18: TI953/954 camera adapter FPGA I2C slave registers

| Addr (hex) | Bit(s) | Type | Default | Description |
|---|---|---|---|---|
| 0x82 | 0<br><br><br><br>7 | RW<br><br><br><br>RW | 0 | I2C Dummy Slave Enable. The I2C Dummy Slave simply acknowledges I2C requests and can be used to emulate an image sensor on a trivial level.<br>I2C Dummy Slave Reset |
| 0x83 | 7-0 | RW | | I2C Dummy Slave Address |
| 0x87 | 0 | RW | 0 | Enable capturing/replaying of D-Phy Start-of-Transmission (0xB8), CSI-2 header and CSI-2 footer data. This mode is required if the image sensor sends different CSI-2 datatypes or virtual channels. If this bit is not set there is no information which image line had which datatype. This bit is also required when replaying images with differente datatypes or virtual channels. In this case the software must deliver the CSI-2 header/footer data by sending the data captured with this mode. If this bit is not set the Lattice MachXO3 FPGA creates the CSI-2 header/footer data based on register 0x88 and 0x89/0x8A |
| | 1 | RW | 0 | Capture any CSI-2 datatype and do not use filters. Usually this bit should be set together with bit 0. |
| | 2 | RW | 0 | Capture short packages (e.g. Start-of-Frame, End-of-Frame). Currently this bit causes that the line counter in the image header is not correct. But the data can be decoded using the libcsi-2 library. |
| 0x88 | 5-0<br>7-6 | RW<br>RW | 0x2C | Replay CSI-2 Datatype<br>Replay CSI-2 Virtual Channel |
| 0x89 | 7-0 | RW | 0x80 | Replay CSI-2 Word Count Low Byte |
| 0x8A | 7-0 | RW | 0x16 | Replay CSI-2 Word Count High Byte |
| 0x8E | 7-0 | RW | 0x4B | Replay Data Delay Low Byte |
| 0x8F | 7-0 | RW | 0x00 | Replay Data Delay High Byte |
| 0x90 | 0 | RW | 1 | Enable Deserializer Reference Clock |
| 0xA0 | 7-0 | RW | 0x2C | Capture CSI-2 Datatype filter 1 |
| 0xA1 | 7-0 | RW | 0x3F | Capture Second CSI-2 Datatype filter 2 |
| 0xA8 | 7-0 | R | 0x3F | Last captured CSI-2 Datatype |
| 0xA9 | 7-0 | R | 0x3F | Second last captured CSI-2 Datatype |
| 0xAA | 7-0 | R | 0x3F | Third last captured CSI-2 Datatype |
| 0xAB | 7-0 | R | 0x3F | Fourth last captured CSI-2 Datatype |
| 0xAC | 7-0 | R | 0x3F | Last captured CSI-2 Datatype latched at end of frame (should be datatype 0x01 = end of frame) |
| 0xAD | 7-0 | R | 0x3F | Second last captured CSI-2 Datatype latched at end of frame |
| 0xAE | 7-0 | R | 0x3F | Third last captured CSI-2 Datatype latched at end of frame |
| 0xAF | 7-0 | R | 0x3F | Fourth last captured CSI-2 Datatype latched at end of frame |
| 0xB0 | 7-0 | R | | CSI-2 line count low byte |
| 0xB1 | 7-0 | R | | CSI-2 line count high byte |
| 0xB2 | 7-0 | R | | CSI-2 CRC Error Counter |
| 0xB3 | 0 | W | | Reset CSI-2 CRC Error Counter |

Table 19: TI953/954 camera adapter FPGA I2C slave memory

☞ The TI953/954 adapter requires CSI-2 continious clock mode enabled in the TI954 deserializer (register 0x33 bit 1) to work correctly. Depending on the frame timing it is possible that capturing images work without continious clock but bypass to the serializer does not work. But even if capturing works the interline- and interframe-gap data is not stored correctly into the image header. It is recommended to enable continious clock mode in any case.

☞ The TI953/954 camera adapter only supports 4 lane mode. Please take care that it is set correctly (see register 0x33 bits 5:4 for TI954 and register 0x02 bits 5:4 for TI953). On camera or ECU side the number of lanes can be different but on the adapter only 4 lanes are supported.

☞ Take care that the receiver datarate is equal or larger then the sender datarate. In case of replaying data using the proFRAME by default the data will be sent to the serializer with 800Mbps per CSI-2 lane and 4 lanes will be used (total 3,2Gbps). It is not possible to change the number of CSI-2 lanes but the CSI-2 sender datarate can be reduced using the sxpfosc tool. If the receiver is using e.g. only 2 lanes with each 800Mbps the sender datarate should be reduced to 400Mbps per lane.

☞ The CSI-2 datarate sent out by the deserializer is independent to the CSI-2 datarate the serializer receives. The deserializer uses the datarate configured in register 0x1F. As mentioned above the receiver datarate must be equal or larger then the sender datarate else a BUFFER_ERROR in register 0x4E will be detected. If the sender datarate is lower then the receiver datarate there will be larger gaps between the CSI-2 packages.

DS90UB953 Serializer datasheet:
`http://www.ti.com/lit/ds/symlink/ds90ub953-q1.pdf`
DS90UB954 Deserializer datasheet:
`http://www.ti.com/lit/ds/symlink/ds90ub954-q1.pdf`

## 5.8 camAD TI954

DS90UB954 Deserializer datasheet:
http://www.ti.com/lit/ds/symlink/ds90ub954-q1.pdf

# 6 Power over Coax (POC)

## 6.1 proFRAME 2.0

The camera adapters contain a power supply unit to supply the camera connected to the deserializer via power over coax. The voltage level can be configured through a programmable I2C potentiometer (AD5110) in a range from around 5V to 10V at 250mA (max.). The I2C Id of the potentiometer is 0x5E (8bit) or 0x2F (7bit). The potentiometer contains a non-volatile memory to store the configuraton permanently. The camera adapter FPGA can enable or disable the supply voltage. This can be controlled by writing the camera adapter FPGA I2C slave registers.
The voltage level can be configured by the following I2C command.

```
0x5F 0x02 0xVAL
```

The first byte is the I2C Id of the programmable potentiometer. The second byte is the register address where the value should be stored. The third byte is the configuration value. This value can be in a range from 0x00 to 0x80. The output voltage can be calculated by the following equation.

$$V_{out} = 0.8V * \left(1 + \frac{90.9}{7.5 + (0.078125 * 0xVAL)}\right) \tag{1}$$

$$0xVAL = \frac{72.72V - 7.5 * (V_{out} - 0.8V)}{0.078125 * (V_{out} - 0.8V)} \tag{2}$$

Examples:

$$V_{outMin} = 0.8V * \left(1 + \frac{90.9}{7.5 + (0.078125 * 0xFF)}\right) = 3.45V \tag{3}$$

$$V_{outMax} = 0.8V * \left(1 + \frac{90.9}{7.5 + (0.078125 * 0x00)}\right) = 10.50V \tag{4}$$

The configured voltage level can be permanently stored in the potentiometer with the following I2C command.

```
0x5F 0x01 0xFF
```

Voltages higher than 10V are possible but require hardware modifications. For 12V PoC a simple hardware patch on the camera adapter allows to give the 12V PCIe voltage directly to the PoC filter. For voltages higher than 12V an additional boost regulator on the base board is needed that is not placed on all base board variants. Newer camera adapters allows the user to inject a custom PoC voltage.

## 6.2 proFRAME 3.0

On the proFRAME 3.0 the PoC voltage is created on the base board and no longer on the camera adapters. Each camera adapter slot has his own PoC voltage which can be controlled independently.
The PoC voltage can be calculated by the following equation.

$$V_{out} = 1V * \left(1 + \frac{10^6}{39000 + (390 * 0xVAL)}\right) \tag{5}$$

$$0xVAL = \frac{10^6V - 39000 * (V_{out} - 1V)}{390 * (V_{out} - 1V)} \tag{6}$$

Examples:

$$V_{outMin} = 1V * \left(1 + \frac{10^6}{39000 + (390 * 0xFF)}\right) = 8.22V \tag{7}$$

$$V_{outMax} = 1V * \left(1 + \frac{10^6}{39000 + (390 * 0x00)}\right) = 26.64V \tag{8}$$

The following INI lines can be used to set and enable the PoC voltage.

```
CMD_WRITE_REGISTER 12 0x00000020 0x00000004 0x00000004 # enable internal i2c channel
ADR_POTI 2 0x11 0xff # set poc for slot 0: 0xff for 8V
ADR_POTI 2 0x12 0xff # set poc for slot 1: 0xff for 8V

CMD_WRITE_REGISTER 12 0x00000030 0x00000002 0x00000004 # enable PoC
```

# 7 Trigger

The FPD3-Link uses a single coax cable to connect the camera with the capturing device. Over this coax cable the image data, the bidirectional back-channel and optionally the power supply is transfered. The back-channel allows also sending frame synchronization signals from the captre device to the image sensor.
The video grabber card contains a trigger module that allows generating various trigger signals and send it through the back-channel to the camera. The trigger module can be configured using the trigger-test application. The following examples show how to set a continious trigger for all camera adapter slots with 30fps and a trigger pulse length of 1msec. Additionally the camera adapter must be configured correctly to output the trigger signal on the correct GPIO on the deserializer. This configuration depends on the hardware design of the used camera.
See appendix for additional information of available trigger modes and configuration.

```
sudo ./src/release/load_sxpf.sh
./src/release/sxpftrigger 0 -c 15 -i 0 -e 1 -f 30 -t cont
```

## 7.1 External Trigger Interface

To synchronize the Solectrix grabber card with external hardware the external *SX proFRAME SYNC* modules can be used. It provides galvanic isolation of each trigger input / output channel by means of dedicated opto-couplers. The *SX proFRAME SYNC* modules are available in different input / output configurations as depicted in Figure 13 to 16.

The connector on the sync module is a LEMO ERD.0S.304.CLL and e.g. the connector LEMO FFA.0S.304.CLAC50 can be used to connect external hardware.

For trigger input the voltage must be between 3,3V and 12V. A constant delay of 3,2µs must be considered during input sampling in the proFRAME FPGA due to input debouncing.

For trigger output an external voltage between 3,3V and 12V must be supplied at the optocoupler output and a pull down to ground e.g. with 1kOhm must be added at pin 4 for proFRAME 2.0 and at pin 2 for proFRAME 3.0.

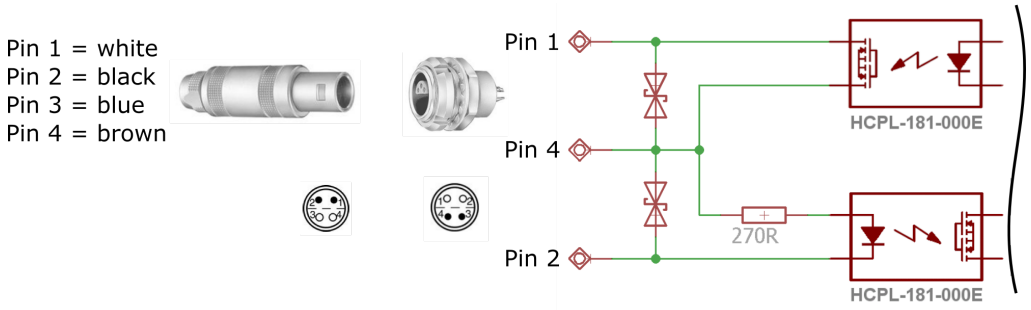



Figure 13: Schematic - SX proFRAME 2.0 SYNC IN/OUT

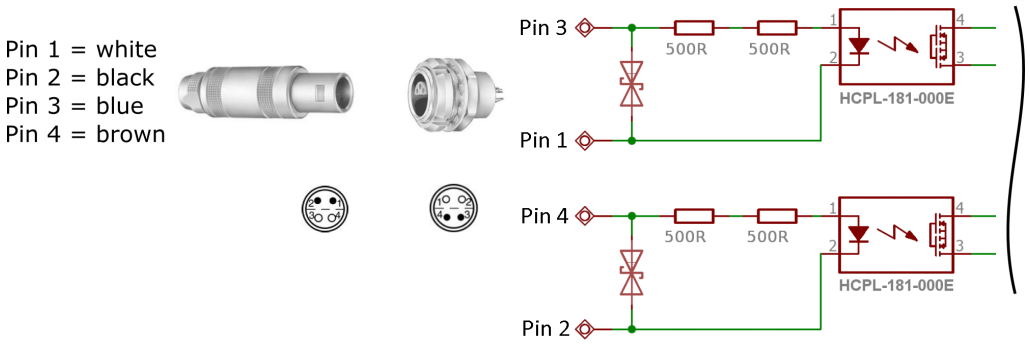



Figure 14: Schematic - SX proFRAME 2.0 SYNC DUAL IN

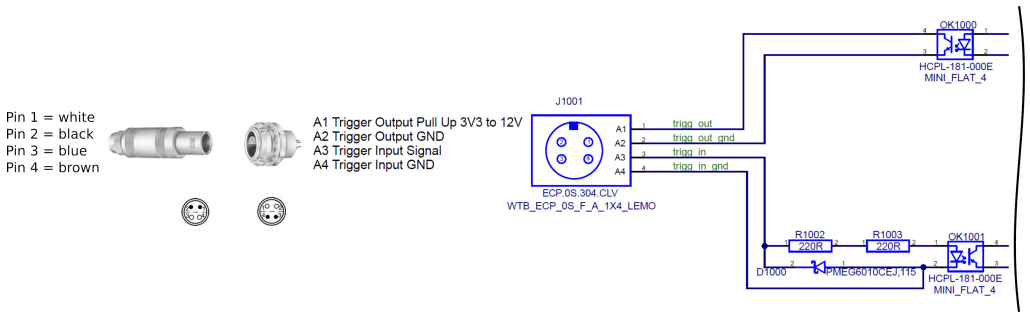



Figure 15: Schematic - SX proFRAME 3.0 SYNC IN/OUT

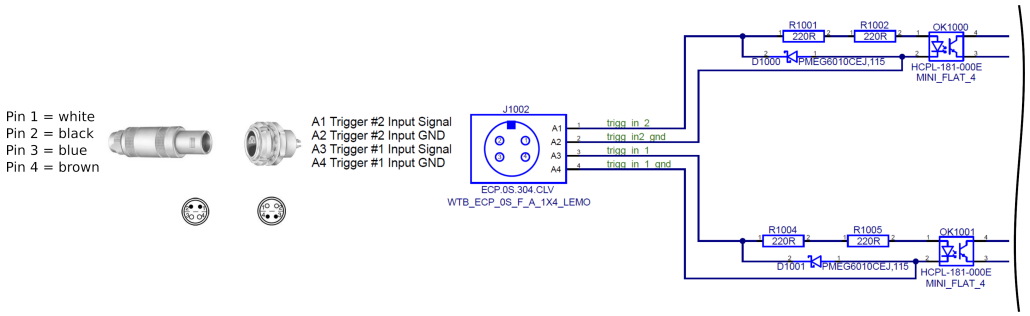



Figure 16: Schematic - SX proFRAME 3.0 SYNC DUAL IN

# 8 Capturing

After initializing the camera adapters and the connected cameras the image data can be captured using the sxpfapp sample application. If no data is received the sxpfscope application might help to debug this issue. Additionally the status registers of the serializers and deserializers can give useful informations at which point the image data gets lost. This registers can be accessed using the sxpfi2c application.
The data received from the grabber card contains a 64byte image header followed by the captured image payload. The definition of the image header is shown below. The payload depends on the camera adapter type and configuration.

Listing 25: sxpf_image_header_s

```
struct sxpf_image_header_s
{
    // information fields that are delivered in this layout for all frame types
    __u32 frame_size; /**< total size including header */
    __u32 version: 8; /**< header version, curreently: 0 */
    __u32 type: 8; /**< frame type: video=1 */
    __u32 cam_id: 4; /**< channel number: 0..3 */
    __u32 card_id: 4; /**< card ID, fixed at 0 */
    __u32 reserved: 8; /**< reserved for future extensions */

    // the following fields are frame type specific
    __u16 payload_offset; /**< actual header size */
    __u8 bpp; /**< bits occupied per pixel */
    __u8 sample_size; /**< actual number of active pixel bits */
    __u16 rows; /**< number of rows in image frame */
    __u16 columns; /**< number of columns in image frame */
    __u32 ts_start_lo; /**< bits[31:0] of start timestamp */
    __u32 ts_start_hi; /**< bits[63:32] of start timestamp */
    __u32 ts_end_lo; /**< bits[31:0] of end timestamp */
    __u32 ts_end_hi; /**< bits[63:32] of end timestamp */
    __u32 frame_counter; /**< frame counter provided by FPGA */
    __u32 ilg; /**< inter-line gap */
    __u32 ifg; /**< inter-frame gap */
    __u32 reserved2; /**< reserved for future extensions */
    __u32 ts_second_start_lo; /**< bits[31:0] of start timestamp */
    __u32 ts_second_start_hi; /**< bits[63:32] of start timestamp */
    __u32 ts_second_end_lo; /**< bits[31:0] of end timestamp */
    __u32 ts_second_end_hi; /**< bits[63:32] of end timestamp */

    // the payload data follows in memory directly after the preceeding
    // header fields, as indicated by the payload_offset member

} PACKED;
```

When capturing CSI-2 image data the columns information must be recalculated based on the CSI-2 datatype to get the real image width. The ts_start_lo and ts_start_hi information contains the hardware timestamp when the capturing of this frame started (frame valid signal = high or CSI-2 Start-of-Frame Package received). ts_end_lo and ts_end_hi the timestamp when the capturing of this frame has ended (frame valid signal = low or CSI-2 End-of-Frame Package received). This hardware timestamp is a 40MHz internal counter that starts when the grabber card is powered. To map this hardware timestamp to a real time value the timestamp could be read periodically out of the grabber card by software or synchronized e.g. to a GPS time source.

## 8.1 CSI-2 RAW capture mode

When using the CSI-2 RAW capture mode e.g. with a TI953/954 camera adapter. The following data format will be received.

```
$ hexdump img_0000000000.raw | grep -A 2 -B 4 b8b8 | head -15
0000000 ba4c 0015 0100 0000 0040 0c20 02e1 01e3
0000010 1861 5520 0001 0000 0c0f 5537 0001 0000
0000020 0043 0000 04d7 0000 aad4 0008 0000 0000
0000030 79eb 57ce 0001 0000 6d99 57e5 0001 0000
0000040 b8b8 b8b8 8012 0307 aa0a 2055 55a5 5a00
0000050 0055 555a 5a00 0355 555a 5ad5 0055 555a
0000060 5a00 0055 555a 5a00 0055 555a 5a08 0055
--
0000780 5a00 0055 555a 5a0c 0055 555a 5a00 0055
0000790 555a 5a00 0055 555a 5a00 0055 555a 5a00
00007a0 0055 555a 5a00 0855 555a 5a04 0655 555a
00007b0 5a14 0055 555a 5a00 ff55 555a 5aff 0155
00007c0 555a 5a00 0055 555a 5624 ffff b8b8 b8b8
00007d0 8012 0307 aa0a 3155 55a5 5a76 0055 555a
00007e0 5a00 0055 555a 5a00 0055 555a 5a00 0055

# Solectrix 64byte header
# D-PHY Start of Transmission Sync sequence (each CSI-2 lane receives a 0xb8)
# CSI-2 Datatype (in this example 0x12 = Embedded 8-bit non Image Data)
# CSI-2 Word Count (bytes of long package,in this example 0x0780 = 1920)
# CSI-2 ECC (in this example 0x03)
# CSI-2 Payload
# CSI-2 Footer CRC (in this example 0x2456)
# Solectrix line padding based on the bpp value in the Solectrix 64byte header
```

In folder src/libs/csi-2 there are helper functions to encode and decode this CSI-2 data.

## 8.2 CSI-2 capturing different virtual channels

CSI-2 supports virtual channels. This means different images can be send together (interleaved or sequencially) through a single MIPI CSI-2 interface. The SX proFRAME supports virtual channel capturing but it depends on the timing/ordering of the received data.

- Two cameras captured using one deserializer (e.g. SX camAD TI954)
  E.g. the SX camAD TI954 contains two FAKRA connectors routed to one deserializer. The TI954 is a dual deserializer and can capture two cameras in parallel. Both image streams will be configured to have different virtual channels and the proFRAME system will analyze virtual channels and take care that camera 1 is captured independently of camera 2. In software there will be 2 different image streams. One stream for each camera. Multiple virtual channels per camera are not supported in this case.

- One camera in RAW HDR mode with multiple aligned virtual channels captured with one deserializer (e.g. SX camAD TI953/954)
  E.g. the SX camAD TI953/954 firmware is usually used to capture image streams from one single camera. The FPGA will capture all CSI-2 packages between the Start-of-Frame and the End-of-Frame package. In most cases if an image sensor is configured for RAW HDR mode it sends different exposures line interleaved with different virtual channels. But all packages are inbetween the Start-of-Frame and End-of-Frame packages. The SX proFRAME system is able to capture this type of data.

- One ECU with multiple unaligned virtual channels captured using one deserializer
  When an ECU sends images to a display the images can be completely independent with different frame-sizes and framerates. Also this data can be transfered through one CSI-2 interface but the SX proFRAME system is currently in most cases not able to capture this data because the different image streams are not aligned. One exception is the firmware for the new SX camAD MAX9295/9296 that allows capturing

of up to four completely independent virtual channels. In this case max. 4 different image streams are supported by one SX proFRAME system.

# 9 Replaying

The previously captured image data can be replayed using the sxpfsend (simple) or sxpfplay (more complex) sample applications. Replaying requires a different grabber card firmware. It is not possible to capture and replay with one single grabber card. For replaying two different timing modes are available that can be selected depending on the values set in the image header.

- Asynchronous replay mode
  If all bits of ts_start_lo/ts_start_hi are set to 1 the asynchronous replay mode is used by the hardware. In this case the hardware start replaying immediately after a complete frame was transfered to the frame-buffer of the hardware. In case of camera adapters with a parallel interface the frame valid signal will be set to high. In case of a CSI-2 based camera adapter a SOF (Start-of-Frame) short package will be generated. For the line timing the ilg (inter-line-gap) value in the image header is used for each inter-line-gap. After the last line one additonal ilg time is waited before stopping the current frame. In case of a camera adapter with a parallel interface the frame valid signal will be set to low. In case of a CSI-2 based camera adapter a EOF (End-of-Frame) short package will be generated. After frame end the ifg (inter-frame-gap) value in the image header is used to wait before replaying the next frame. If the next frame is already in the framebuffer the hardware start replay of this frame. If not it waits until the next frame was completely received.

- Synchronous replay mode
  If ts_start_lo/ts_start_hi contain a timestamp value the synchronous replay mode is used. In this case replaying of the frame starts (set frame valid to high or send SOF package) if the internal timestamp counter matches the timestamp in ts_start_lo/ts_start_hi. If the internal timestamp is already larger then the timestamp in the image header an error interrupt is generated and the replaying of this frame start immediately. For the line timing the same strategy is used like in asynchronous replay mode. After the last line the hardware waits until the ts_end_lo/ts_end_hi value is reached before stopping (set frame valid to low or send EOF package) the current frame. If the internal timestamp is already larger then ts_end_lo/ts_end_hi value an error interrupt is generated and the frame is stopped immediately after the last line was sent. After that the next frame of the frame buffer is analyzed.

  Writing incorrect values to ts_start_lo/ts_start_hi can cause the hardware to wait a very long time before the replaying start. So take care to set correct value. The sxpfplay source code shows how to set the timestamps to useful values.

- Trigger based replay mode
  Also it is possible to use the GPIO based trigger signal from an ECU to replay the frames. It is possible to select the rising or falling edge of the trigger signal and also add a delay from receiving trigger signal to start sending frame. In this case set all ts_start_lo/ts_start_hi bits to 1 like in asynchronous replay mode and set ilg to a useful value that the replaying of one frame is finished before the next trigger pulse is received.

## 9.1 Loopback Tests

An easy way to test the replay mode is to setup a loopback system using two proFRAME boards and connecting the serializer (video outout) and deserializer (camera input) using a coax cable (see 17). The sending card must contain a replay firmware and the receiving card a capture firmware. Using the sxpfplay and sxpfapp sample applications frames can be transfered. In the tests folder there are many loopback examples available. In many cases this is an easy first step before sending frames to an ECU because usually the ECUs require a correct image format, a good frame timing etc.. If one of this points does not match the requirements, the ECU might crash or does not deliver valuable debug data. Using the proFRAME to emulate the ECU can help to better understand the problem.

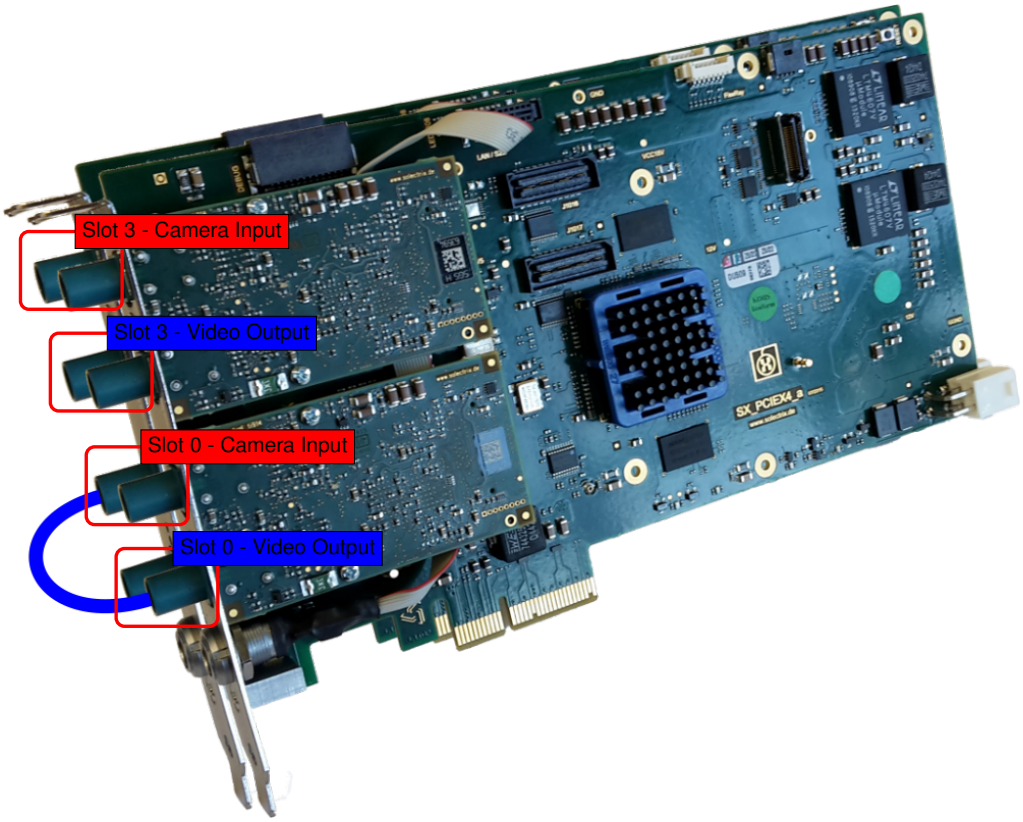



Figure 17: Two grabber cards for loopback test

# 10 Software API

All the delivered applications are not intended to be used in the final customer application. They are only examples that show how to use the functions of the grabber card. The customer must integrate the sxpf API into his application framework to realize a full featured image capture, decode, storage and replay solution.
The most important functions and definitions of the API can be found in the source code files sxpf.h and sxpftypes.h. A detailed function description can be found in sxpf.cpp.
A complete API documentation can be created in the following way

Listing 26: Building Doxygen API Documentation

```
$ cd ./src/
$ make doc
```

The sxpfrec code sample gives a first impression how to receive an image frame from the grabber card.

# 11 Advanced features

## 11.1 GPS timestamping

The grabber card by default uses an internal 40MHz counter to timestamp the recorded image data. This counter is free running and starts when the card is powered. However it is possible to use a GPS based PPS/NMEA timesource for timestamping.
The PPS signal must be connected to the trigger input of the grabber card. The NMEA sequence that contains date and time information is received by the PC via e.g. UART.
In the grabber card there is a 64bit register that contains the GPS timestamp. The higher 32bits are a seconds counter. This counter is incremented by one on each PPS pulse. This register is writable by the software and can be set to e.g. a UNIX timestamp (seconds since 01.01.1970). The example applications sxpf-nmea-sync shows how to parse the NMEA string and set the seconds counter in the grabber card. The lower 32bit contain a 40MHz counter. This counter is resetted to zero on each received PPS pulse. So usually this counter should increment from 0 to 40000000. However based on clock variations this counter could be a bit smaller or greater. If no PPS pulse is received the counter is not resetted and the software is able to detect the missing PPS pulse.

> ☞ The FPGA reacts on the rising edge of the PPS pulse. When using the SX proFRAME Sync module the pulse is inverted because of the opto-isolator so the FPGA will react on the falling edge of an external pulse.

If no NMEA source is available it is also possible to use a NTP synchronized PC date/time to timestamp the images. In this case just make sure that the time value is written to the 32bit as soon as possible after a new seconds value is reached.
For a simple example see /tests/proFRAME_V2/others/gps_synchronisation

# 12 Cameras

## 12.1 IMI

### 12.1.1 RDACM24

The RDACM24 camera from IMI contains an OV10640 image sensor, a OV490 companion ISP processor and a DS90UB913 serializer. Additionally there is a SPI flash that is connected to the OV490. After power up the OV490 reads the SPI flash and configures the OV10640. The I2C pins of the DS90UB913 are connected to the OV490 only. This means that the image sensor can not be configured by the FPD-3 link back channel directly. Instead it is needed to reprogram the SPI flash through the OV490.

#### 12.1.1.1 OmniVision OV490 Firmware Merge Tool

To build a bin file that can be programmed into the camera SPI flash the tool *OV490 Firmware Merge Tool* from OmniVision can be used. To do that run the *OV490_GeneralFW_Tool(v1.3)_v317.exe* and press the *Open* button. Select the directory containing the configuration txt files for the camera. After that enable the checkbox *with SPI Header* and create the bin file by pressing the *Export* button.

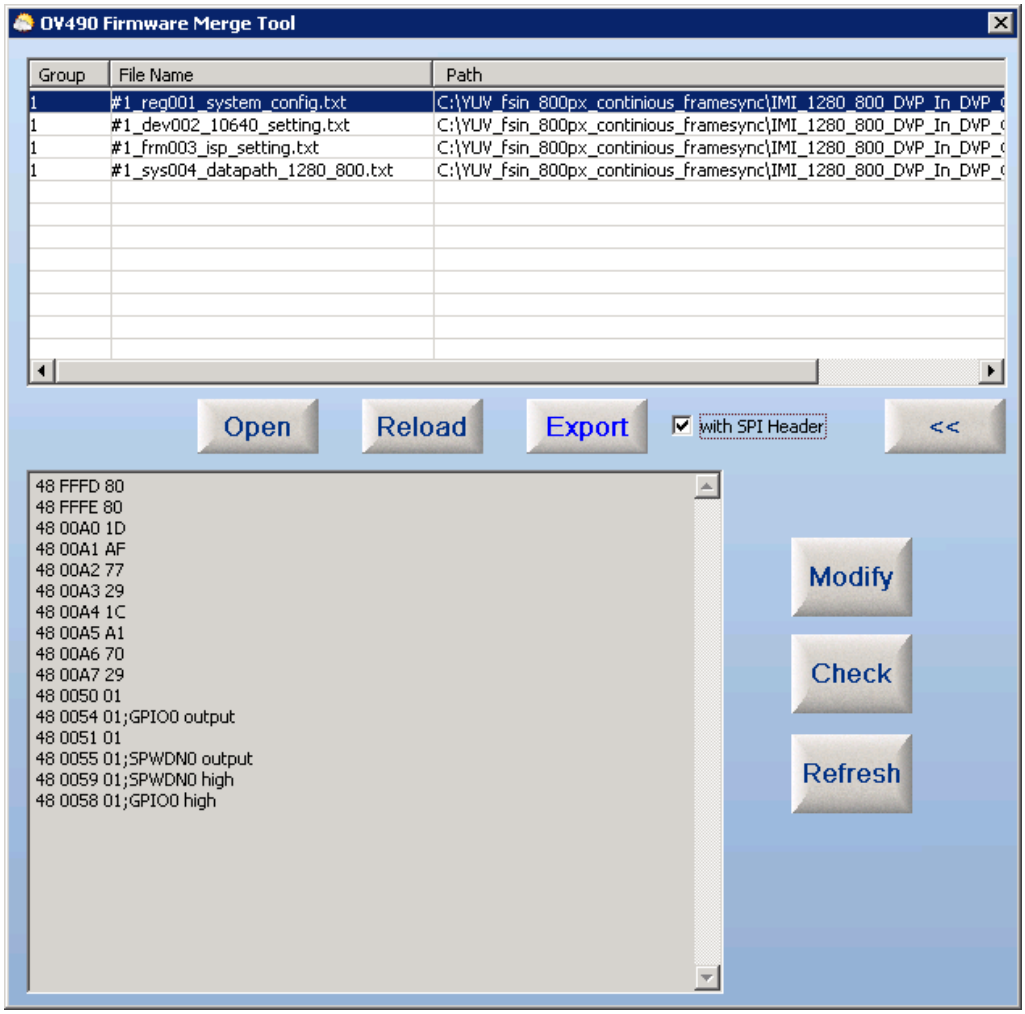



Figure 18: OV490 Firmware Merge Tool

#### 12.1.1.2 Flash programming

To program the SPI flash of the RDACM24 camera the *sxpf-IMI-RDACM24-Control* tool can be used. First use the OmniVision toolchain to create a bin file. After that load the kernel driver and initialize the camera link by running a correct INI file.

```
$ sudo ./src/release/load_sxpf.sh
$ ./src/release/sxpf-init-sequence /dev/sxpf0 -port 0 -ini RDACM24-01_FSIN_enable_RAW12.ini
    --execute 10
```

Now the green link LED on the camera adapter should indicate a link to the camera. Run the *sxpf-IMI-RDACM24-Control* tool to write the OmniVision bin file to the camera.

```
$ ./src/release/sxpf-IMI-RDACM24-Control 0 0 -w
    OV10640_OV490_combine_general_v330_20150926_common201604181028_withSPIHeader.bin
erase: done
write size: 29534
write: 0%
...
write: 98%
write: 99%
write: done
```

#### 12.1.1.3 Host commands

The *sxpf-IMI-RDACM24-Control* tool can also be used to send host commands to the camera. After initializing the grabber system and the camera successfully use the following command to set e.g. the Hue level to 5. A full list of supported host commands can be listed using the *-h* command line parameter.

```
$ ./src/release/sxpf-IMI-RDACM24-Control 0 0 -c CMD_HUE_SET,5
```

# 13 FAQ

## 13.1 [sxpf0] Could only allocate XX of YY DMA buffers

If the following error messages are shown when loading the kernel modules, the coherent memory configuration is too small for all required image buffers. There are two ways to resolve this issue.

```
$ sudo ./src/release/load_sxpf.sh framesize=$((0x800000))
chgrp: cannot access '/dev/sxpf[0-9]': No such file or directory
chmod: cannot access '/dev/sxpf[0-9]': No such file or directory
```

```
[ 118.024862] [sxpf0] allocate_buffer: out of memory.
[ 118.024864] [sxpf0] Could only allocate 0 of 31 DMA buffers.
[ 118.024867] [sxpf0] DMA buffer allocation failed for sxpf0
```

- Increase the coherent memory (CMA) in the kernel configuration
- Reduce number of image buffers, e.g.,

  ```
  $ sudo ./src/release/load_sxpf.sh buffers=10
  ```

## 13.2 flash init error

If this error message is shown when trying to update the grabber card using the tool `sxpfupdate`, the SPI flash of the grabber card hardware is not supported by the grabber card firmware. This means that maybe an old firmware was flashed on the grabber card before or the firmware file in `/lib/firmware/sxpf.fw` is not valid for the grabber card. To solve this issue run the following:
Copy the latest firmware file `sxpf.fw` from the software library into the directory `/lib/firmware/`

- Copy the latest firmware file `sxpf.fw` from the software library into the directory `/lib/firmware/`

  ```
  $ sudo cp sxpf.fw /lib/firmware/
  ```

- Reload the grabber card driver. This will replace the firmware in the grabber card by the content of the `sxpf.fw`.

  ```
  $ sudo ./src/release/load_sxpf.sh
  ```

- Run `sxpfupdate` to flash the new firmware file into the grabber card

  ```
  $ ./src/release/sxpfupdate CARD_UPDATE.bit /dev/sxpf0
  ```

- Remove the firmware file `/lib/firmware/sxpf.fw` because, in general, it is not recommended to use this feature.

  ```
  $ sudo rm /lib/firmware/sxpf.fw
  ```

## 13.3 Error - content of `xxx.bit` is no valid bit file

If this error message is shown when trying to update a camera adapter using the tool `sxpfxo2`, the selected bit file is not valid for the camera adapter. Maybe it is a bit file for the video grabber card or another camera adapter.

## 13.4 [sxpf0] Plasma command timed out on arg slot

If this error message is shown when trying to send an initialization sequence it is possible that the interrupts from the grabber card are not forwarded by the PC hardware. This has been seen with server mainboards like an ASUS Z10PE-D8 WS. One workaround is to enable the interrupt polling in the kernel driver.

## 13.5 Uninstalling Windows driver

To uninstall and fully remove the Windows driver e.g. to install an older driver version you must manually remove the file 'C:\Windows\System32\drivers\sx_pciex4.sys'. Else installation of an older driver version does not work.

## 13.6 No valid license

If you get an error output in the Windows service or the sample applications similar to

```
Error: 0x20000009 - No valid license
```

the version of the installed drivers does not match to the used Windows service or API revision. To solve this issue install the matching drivers found in the API release you are using.

## 13.7 uyvy2rgb.vert not found

If you get the following message when calling sxpfapp to show a YUV image you can simply ignore this message because the shader only requires the uyvy2rgb.frag file to work properly

```
File uyvy2rgb.vert not found - not used in shader program
```

## 13.8 [sxpf0] pci_iomap of BAR 4 failed

If you get the following message when loading the driver then the firmware of the proFRAME base board is too new for the software release you are using. Software releases before r1928 try to map BAR4 of the proFRAME. In newer firmware versions this BAR4 has been removed. To solve the issue use the latest software release, load the driver and if you want downgrade the firmware to an older version. Anyway it is recommended to use the latest software and firmware versions.

```
[sxpf0] pci_iomap of BAR 4 failed
sxpf: probe of 0000:09:00.0 failed with error -12
```

## 13.9 proFRAME 2.0 - LED Status Information

The LED1001 and LED4003 shows the status of the proFRAME 2.0 base board.

| LED status | Description |
|---|---|
| static red | Board power supply damaged |
| static green | Board power supply ok |

Table 20: proFRAME 2.0 LED1001 status

| LED status | Description |
|---|---|
| static red | FPGA firmware not loaded (maybe an invalid firmware was flashed or the flash memory is corrupted, please contact Solectrix support) |
| flashing orange/green | FPGA firmware is loaded, power supplies are ok, PLLs are locked and DDR memory is calibrated |

Table 21: proFRAME 2.0 LED4003 status

## 13.10 proFRAME 3.0 - LED Status Information

The LEDs on the slot bracked show the status of the proFRAME 3.0 base board.

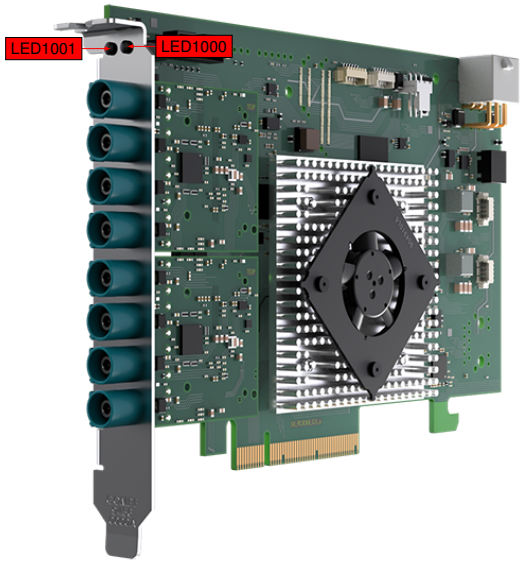



Figure 19: proFRAME 3.0 LED Status Information

| LED status | Description |
|---|---|
| LED1000 static green | Power supply is ok |
| LED1000 static red | Power supply is not ok |
| LED1001 static orange | Firmware is not loaded. Maybe a previous firmware update did fail or the SPI flash memory is defect. The hardware must be flashed using a Xilinx programmer. |
| LED1001 static red | The firmware is in the reset state. Reasons for this could be that the MMCM/PLL components are not started properly or the PCIe reset from the PC mainboard is active. Try the proFRAME in another PC. |
| LED1001 flashing red | Firmware is in operational state. MMCM/PLL components have locked properly, and no PCIe reset is active. |

Table 22: proFRAME 3.0 LED status information

## 13.11 Framesize of RAW files

Because the SX proFRAME system captures RAW image data that is received directly from the deserializers it is not trivial to calculate the frame size of the RAW file.
Based on the header informations of the RAW frame the frame size can be calculated in the following way

$$frame_size = (columns * bpp * rows)/8 + payload_offset \tag{9}$$

Take care the columns value is in most cases not the real image width. This value specifies the number of clock cycles where the FPGA did capture data. The number of bits captured per clock cycle is defined in the bpp value. Especially for image data captured in CSI-2 format the columns value does not match the read image width.
The following example shows how to calculate the frame size

```
$ ./sxpfrawinfo img_0000000000.raw
frame_size: 10735984
version: 0
```

```
type: 1
cam_id: 0
card_id: 0
payload_offset: 64
bpp: 32
sample_size: 12
rows: 1860
columns: 1443 [clock cycles]
ts_start_lo: 0xada066c4 [25nsec ticks]
ts_start_hi: 0x00000001 [25nsec ticks]
ts_start: 180.198 [sec since power on]
ts_end_lo: 0xadb39761 [25nsec ticks]
ts_end_hi: 0x00000001 [25nsec ticks]
ts_end: 180.230 [sec since power on]
frame_counter: 291
ilg: 1271 [clock cycles]
ifg: 908484661
ts_second_start_lo: 0x4db42697
ts_second_start_hi: 0x00000000
ts_second_end_lo: 0x4dc75733
ts_second_end_hi: 0x00000000

image payload seems to be RAW CSI-2 data. Trying to decode it...
    0: vc=0x0 dt=0x1e word count=5760 ecc=valid crc=valid
    1: vc=0x0 dt=0x1e word count=5760 ecc=valid crc=valid
        ...
```

$$frame_size = columns * bpp * rows + payload_offset = (1443 * 32 * 1860)/8 + 64 = 10735984 \quad (10)$$

For image data captured in CSI-2 format the columns value and the line size can be calulated in the following way.

$$line_size = dphy_sot + csi2_header + word_count + csi2_footer + padding = 4 + 4 + 5760 + 2 + x = 5770 + x \quad (11)$$

Because in this example the FPGA captures 32 bit per clock cycle $bpp = 32$ the line_size must be a multiple of 32bit. So in this case the line padding is 2 bytes.

$$line_size = dphy_sot + csi2_header + word_count + csi2_footer + padding = 4 + 4 + 5760 + 2 + 2 = 5772 \quad (12)$$

$$frame_size = line_size * rows + payload_offset = 5772 * 1860 + 64 = 10735984 \quad (13)$$

$$columns = line_size/(bpp/8) = 5772/(32/8) = 1443 \quad (14)$$

## 13.12 Open CSI-2 RAW file in ImageJ

The following shows how to open a capture RAW8 CSI-2 image using ImageJ. Because of the 64byte image header added by the FPGA and the CSI-2 header/footer the parameters for ImageJ is not only the expected image size.

```
$ ./sxpfrawinfo img_0000000000.raw
frame_size: 2095664
version: 0
type: 1
cam_id: 0
card_id: 0
payload_offset: 64
```

```
bpp: 32
sample_size: 12
rows: 1300
columns: 403 [clock cycles]
ts_start_lo: 0x6f89f3cf [25nsec ticks]
ts_start_hi: 0x0000008c [25nsec ticks]
ts_start: 15079.168 [sec since power on]
ts_end_lo: 0x6f93cc29 [25nsec ticks]
ts_end_hi: 0x0000008c [25nsec ticks]
ts_end: 15079.184 [sec since power on]
frame_counter: 22714
ilg: 1553 [clock cycles]
ifg: 517102187
ts_second_start_lo: 0x6f98ee31
ts_second_start_hi: 0x00000001
ts_second_end_lo: 0x6fa2c68b
ts_second_end_hi: 0x00000001

Image payload seems to be RAW CSI-2 data. Trying to decode it...
    0: vc=0x0 dt=0x2a (RAW8) ecc=valid crc=valid word count=1600
    1: vc=0x0 dt=0x2a (RAW8) ecc=valid crc=valid word count=1600
    ...
 1299: vc=0x0 dt=0x2a (RAW8) ecc=valid crc=valid word count=1600
ecc error count = 0
crc error count = 0
```

The output of sxpfinfo shows an image size of 1600px image width and 1300px image height. For ImageJ the following parameters must be used

- ”Image type” is 8-bit because of the RAW8 datatype
- ”Width” are the word count (1600) plus CSI-2 header/footer and padding (see 13.11)
- ”Height” are the number of lanes/CSI-2 packages
- ”Offset to first image” must be set to 64byte (see 8)

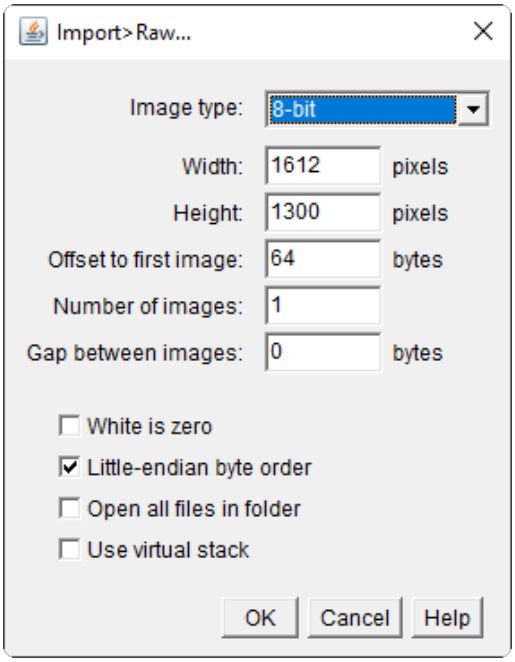



Figure 20: ImageJ Load Parameters

## 13.13 I2C bus scan

In some cases it helps to do an I2C bus scan to check which I2C devices in the camera are available. Use the tool sxpfi2cscan to run an I2C bus scan using acknowledge polling. Before this make sure you selected the external I2C bus.

## 13.14 unrecognized command-line option '-ftrivial-auto-var-init=zero'

If you get the following message when compiling the sxpf kernel module you face a problem of Ubuntu and their latest kernel version (6.5.0-14-generic and maybe newer).

```
gcc: error: unrecognized command-line option '-ftrivial-auto-var-init=zero'
```

The latest generic kernel binary from Ubuntu was compiled using gcc-12 but the latest default version is still gcc-11. To successfully build the sxpf kernel module your compiler needs the same version as the kernel itself was build. To solve this issue first you need a sxpf release later than revision 7138 and then do the following steps.

```
sudo apt install g++-12
KCC=gcc-12 make
```

## 13.15 Ubuntu 18.04

Ubuntu 18.04 is not officially supported, anymore.
But with a few extra steps one can at least build the base set of proFRAME libraries and utils.
One reason for getting compiler errors is the old release of libSDL2. Compile it from source like so:

```
sudo apt update
sudo apt upgrade
sudo apt install build-essential git make autoconf automake libtool pkg-config \
    cmake ninja-build gnome-desktop-testing libasound2-dev libpulse-dev libaudio-dev \
    libjack-dev libsndio-dev libsamplerate0-dev libx11-dev libxext-dev libxrandr-dev \
    libxcursor-dev libxfixes-dev libxi-dev libxss-dev libwayland-dev libxkbcommon-dev \
    libdrm-dev libgbm-dev libgl1-mesa-dev libgles2-mesa-dev libegl1-mesa-dev \
    libdbus-1-dev libibus-1.0-dev libudev-dev fcitx-libs-dev
cd ~
git clone https://github.com/libsdl-org/SDL.git sdl.git
cd sdl.git
git checkout SDL2
./configure
make -j4
sudo make install
```

The proFRAME tools and libraries depend on newer C++ features. Thus, a newer gcc must be installed than the one used by Ubuntu 18.04:

```
sudo add-apt-repository ppa:ubuntu-toolchain-r/test
sudo apt install g++-9
```

The proFRAME software needs some additional development libraries:

```
sudo apt install libavcodec-dev libavformat-dev libavutil-dev libfreetype6-dev \
    libglm-dev libharfbuzz-dev libhidapi-dev libjpeg-turbo8-dev libopencv-dev \
    libpng-dev libswscale-dev python3-dev
```

As a final step, in the included text rendering library libsdl-ttf2 the use of the harfbuzz text shaping library must be disabled:

```
cd <video-app-dir>
sed -i 's/TTF_USE_HARFBUZZ=1/TTF_USE_HARFBUZZ=0/' src/libs/sdl-ttf2.mk
cd src
make -j4 CC=gcc-9
```

Now the build should run successfully.

☞ All following make commands must now add the option `CC=gcc-9`. The easiest way to do this is by changing to the output directory `./src/release/` and just calling `make` from there, because in there is a generated `Makefile` that contains cached variable definitions for all important build settings.

# 14 Appendix

## 14.1 proFRAME 2 - FPGA Register Description

This section describes some FPGA registers.

☞ Nearly all registers are controlled using the SX proFRAME software API. The API takes care that the registers are controlled in the correct way. So do not modify the register values unless you really know what you are doing because it can cause undefined behaviour.

### 14.1.1 FPGA System Register

Table 23: FPGA System Register - Address Map

| Offset | Description |
|---|---|
| 0x000 | PROJECT |
| 0x004 | DATE |
| 0x008 | VERSION |

#### 14.1.1.1 PROJECT

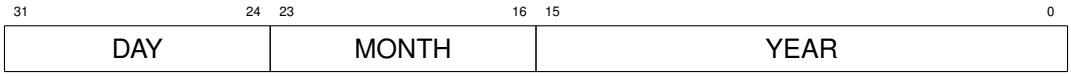
Register 14.1: PROJECT (0x00000000)




| Bits | Field Name | Description | Type | Reset |
|---|---|---|---|---|
| 31:0 | PROJECT | Project Name | RO | ”HIL2” |

#### 14.1.1.2 DATE

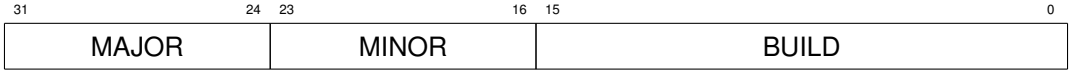
Register 14.2: DATE (0x00000004)




| Bits | Field Name | Description | Type | Reset |
|---|---|---|---|---|
| 31:24 | DAY | Day (in Hex) | RO | - |
| 23:16 | MONTH | Month (in Hex) | RO | - |
| 15:0 | YEAR | Year (in Hex) | RO | - |

#### 14.1.1.3 VERSION

Register 14.3: VERSION (0x00000008)




| Bits | Field Name | Description | Type | Reset |
|---|---|---|---|---|
| 31:24 | MAJOR | Major Version | RO | - |
| 23:16 | MINOR | Minor Version | RO | - |
| 15:0 | BUILD | Build Version | RO | - |

#### 14.1.1.4 Video Processing Register

Table 24: Video Processing Register - Address Map

| Offset | Description |
|---|---|
| 0x480 | PKT_PARSE_CFG_0 |
| 0x484 | PKT_PARSE_CFG_1 |
| 0x488 | PKT_PARSE_CFG_2 |
| 0x48C | PKT_PARSE_CFG_3 |

#### 14.1.1.4.1 CSI-2 PACKET PARSER CONFIGURATION

Register 14.4: PKT_PARSE_CFG_x (0x480 — 0x484 — 0x488 — 0x48C)




| Bits | Field Name | Description | Type | Reset |
|---|---|---|---|---|
| 31:30 | FE_VC | Frame End Virtual Channel<br>Default: VC=0 | RW | 0x0 |
| 29:24 | FE_DT | Frame End Data Type<br>Default: DT= Frame End Packet | RW | 0x01 |
| 23:22 | FS_VC | Frame Start Virtual Channel<br>Default: VC=0 | RW | 0x0 |
| 21:16 | FS_DT | Frame Start Data Type<br>Default: DT= Frame Start Packet | RW | 0x00 |
| 15:14 | PAUSE_VC | Pause Packet Virtual Channel<br>Default: VC=0 | RW | 0x0 |
| 13: 8 | PAUSE_DT | Pause Packet Data Type<br>Default: DT= 0x17, VC = 0<br>**NOTE:**<br>The defined data type and virtual channel for pause packets is used for replay mode only. | RW | 0x17 |
| 7:4 | RES | reserved | RES | - |
| 3 | VC_FILTER | Virtual Channel Filter<br>Enable Virtual Channel filtering for Frame Start/End<br>0: only check Data Types (Legacy Mode)<br>1: additionally include VC in Frame Start/End Check | RW | 1 |
| 2 | CAPT_ALL | Capture All<br>0: only Long Packets are recorded (Legacy Mode)<br>1: Short & Long Packets are recorded | RW | 0 |
| 1 | MODE | Packet Parsing Mode<br>0 : Frame Mode<br>1 : Packet Mode | RW | 0 |
| 0 | RES | reserved | RES | - |

#### 14.1.1.4.2 REPLAY CONTROL

Register 14.5: REPLAY_CTRL_x (0x4A0 — 0x4A4 — 0x4A8 — 0x4AC)




| Bits | Field Name | Description | Type | Reset |
|---|---|---|---|---|
| 31:8 | RES | reserved | RES | - |
| 7 | PAUSE_PKT_EN | Pause Packet Enable<br>Use embedded pause packets for inter-line gap (ILG)<br>0: use ILG from image header<br>1: use ILG from special CSI-2 generic long packet<br>(Pause Packet: DT = 0x17 (default), defined in CSI-2 PACKET PARSER CONFIGURATION)<br>NOTE:<br>If in this mode no pause packets are embedded in the image, the ILG from the image header is used instead. | RW | 0 |
| 6 | SP_ENABLE | Enable replay of embedded short packets<br>0: start/end short packets are generated automatically<br>1: no start/end short packets generation; short packets are also embbeded in replay data | RW | 0 |
| 5 | USE_PKT_WC | Use line length from header or WC from CSI-2 packet<br>**ATTENTION: experimental feature!!!**<br>0: use line length from image header (COL_CNT)<br>1: use WC from CSI-2 long packet | RW | 0 |
| 4 | CSI2_CALC_EN | ECC/CRC calculation enable<br>enable the on-the-fly calculation & insertion of CSI-2 ECC/CRC (only supported in 32 bit mode)<br>**ATTENTION:**<br>in this mode the SyncPreamble (4x 0xB8), header and footer MUST be included in the CSI-2 raw packet | RW | 0 |
| 3: 0 | RMODE | Replay Mode<br>0 : Primary Timestamp based Replay (Async./Sync)<br>1 : Secondary Timestamp based Replay (Async./Sync)<br>2 : Trigger based Replay (start @ rising edge)<br>3 : Trigger based Replay (start @ falling edge)<br>OTHERS : future modes | RW | 0 |

#### 14.1.1.5 PLASMA Control Register

Table 25: PLASMA Control Register - Address Map

| Offset | Description |
|---|---|
| 0x00020 | I2C_CONTROL_0 |
| 0x00024 | I2C_CONTROL_1 |
| 0x00028 | I2C_CONTROL_2 |
| 0x0002C | I2C_CONTROL_3 |

##### 14.1.1.5.1 I2C BUS CONTROL - PORT 0...3

Register 14.6: I2C_CTRL_x (0x00000020 — 0x00000024 — 0x00000028 — 0x0000002c)

| 31 | 0 |
|---|---|
| I2C_CTRL_x (x=0...3) | |

| Bits | Field Name | Description | Type | Reset |
|---|---|---|---|---|
| 31:4 | RES | Reserved | RES | - |
| 3:0 | I2C_BUS_EN | I2C Bus Enable Port<br>0: I2C Bus disabled,<br>1: I2C Bus enabled<br><br>[0]: I2C Bus - Logging (Serialiser, ...)<br>[1]: I2C Bus - Conf.(EEPROM, Oscillator, ...)<br>[2]: I2C Bus - Comm. (I2C Slave@Adapter)<br>[3]: I2C Bus - EXT1 (!Camera Adapter Gen3 only!) | RW | 0x7 |

#### 14.1.1.5.2 Trigger Module

#### 14.1.1.5.3 Trigger Module Architecture

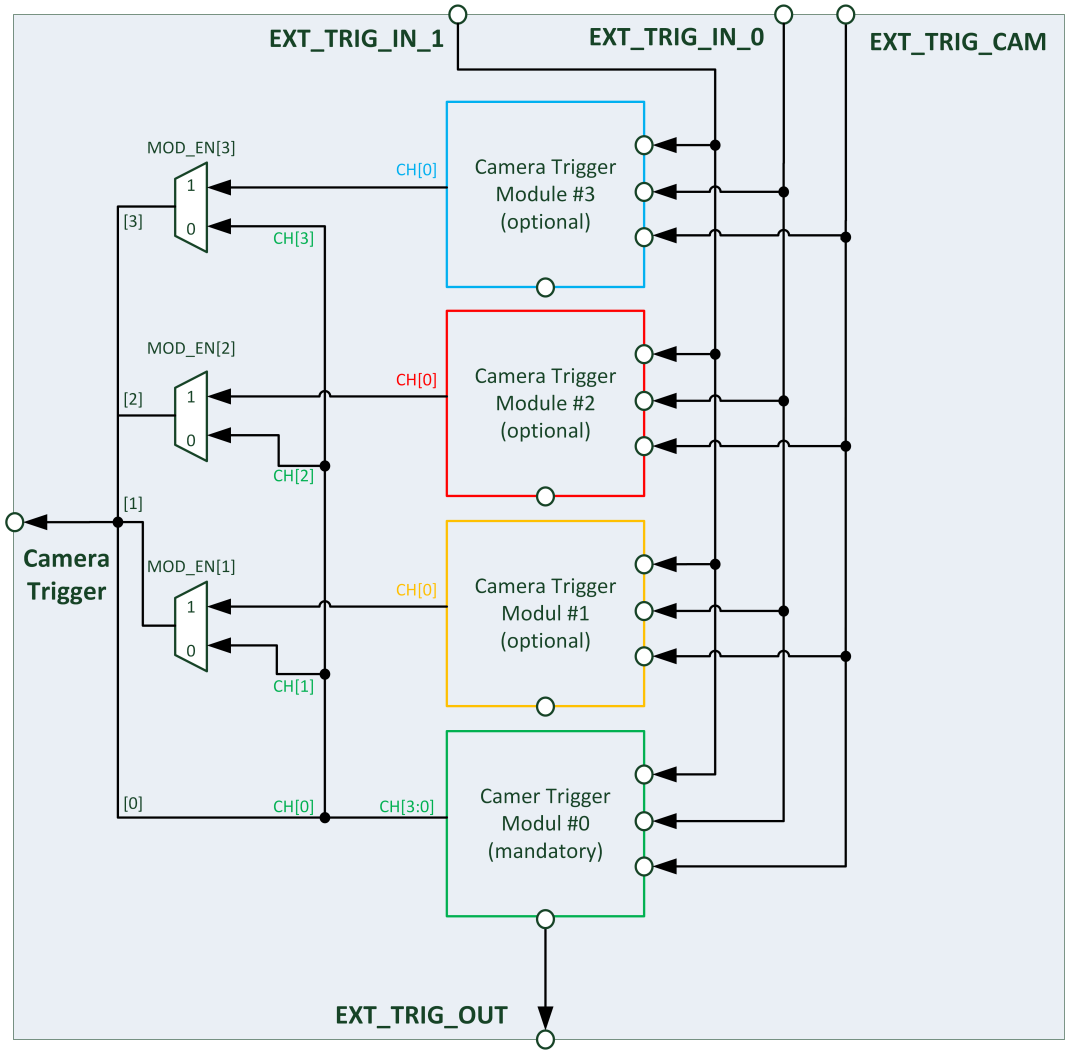



Figure 21: Trigger Module Architecture

#### 14.1.1.5.4 Trigger Module Overview

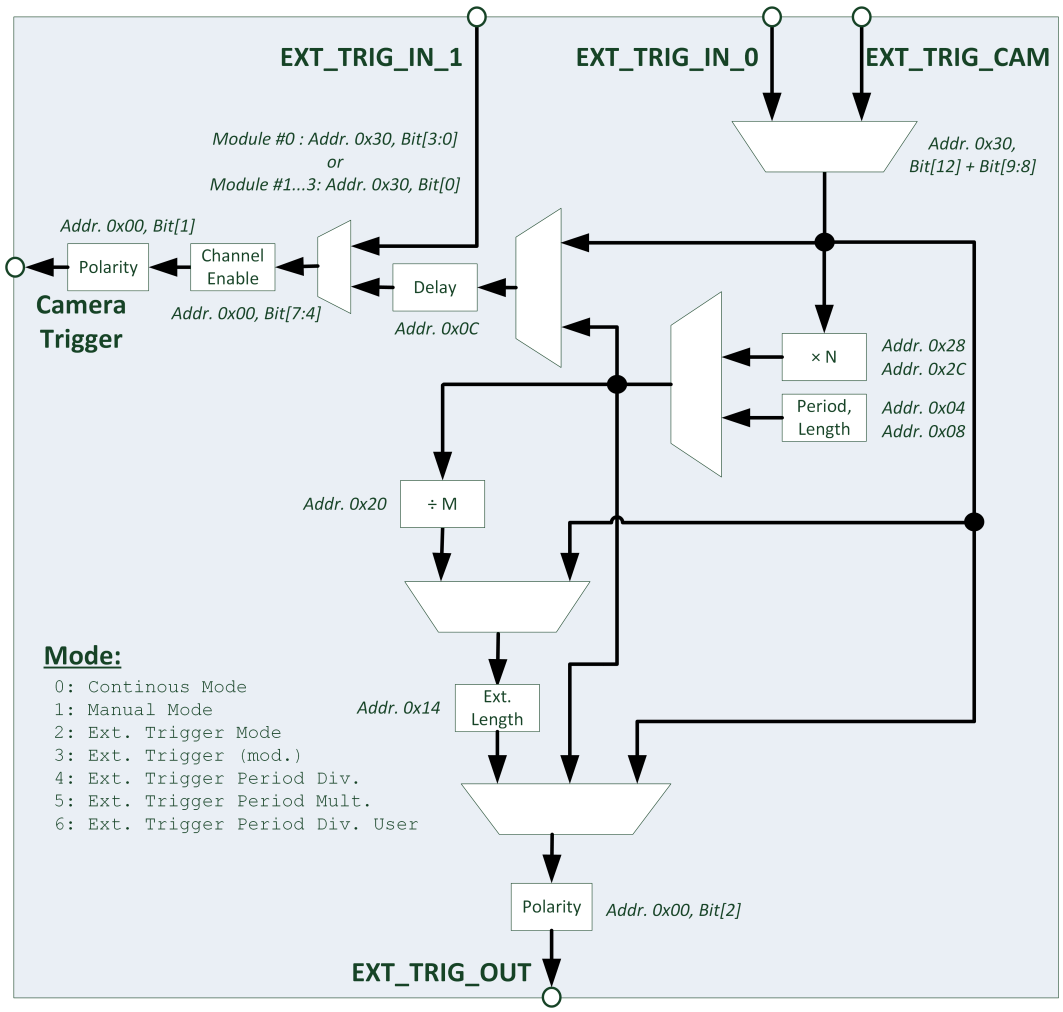



Figure 22: Trigger Module Overview

#### 14.1.1.5.5 Trigger Module Modes

The Trigger Module provides different selectable trigger modes.

- Continuous Mode
- Manual Mode
- External Trigger Mode
- External Trigger Mode (modified pulse length)
- External Trigger Input Frequency Multiplied
- External Trigger Output Frequency Divided
- External Trigger Output Frequency Multiplied (User value)

A graphical overview of all supported trigger modes is provided below.

#### 14.1.1.5.6 Continuous Mode

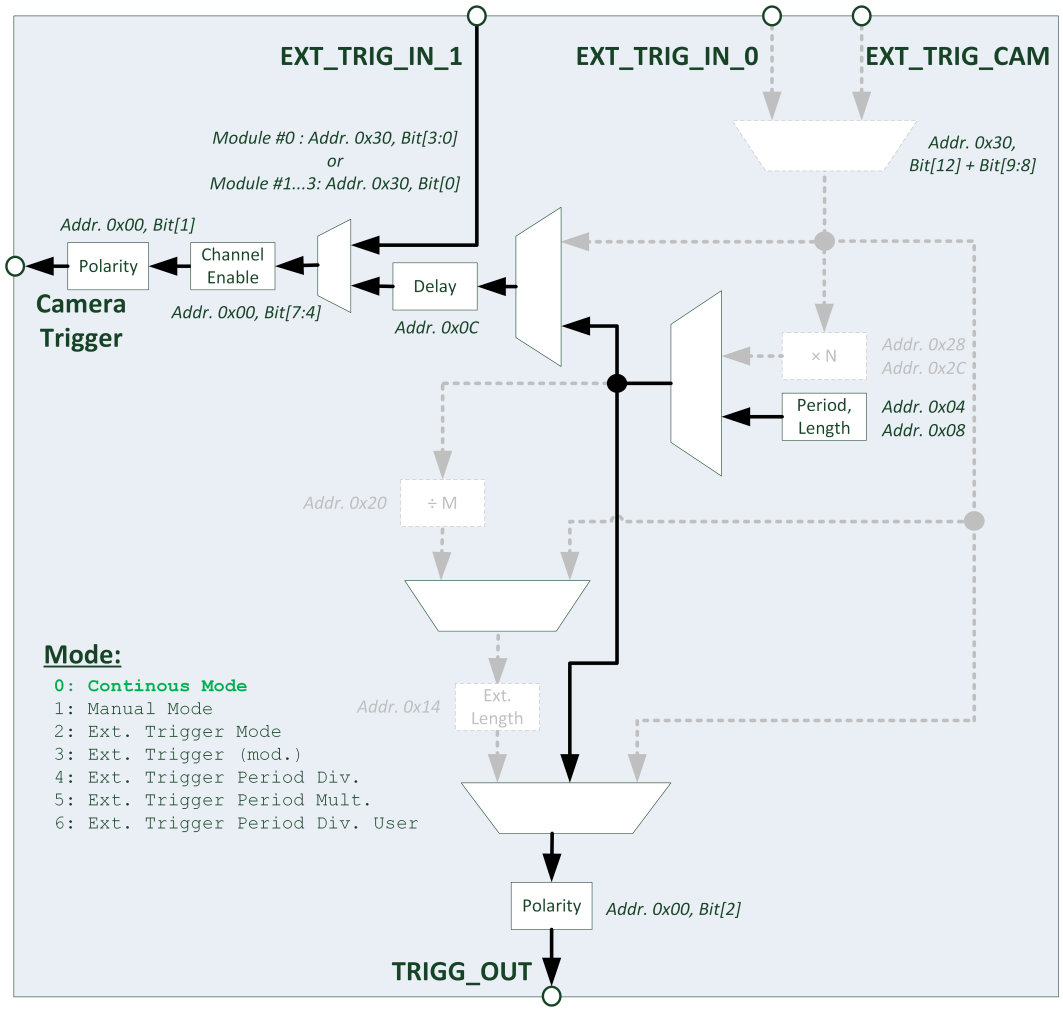



Figure 23: Continuous Mode

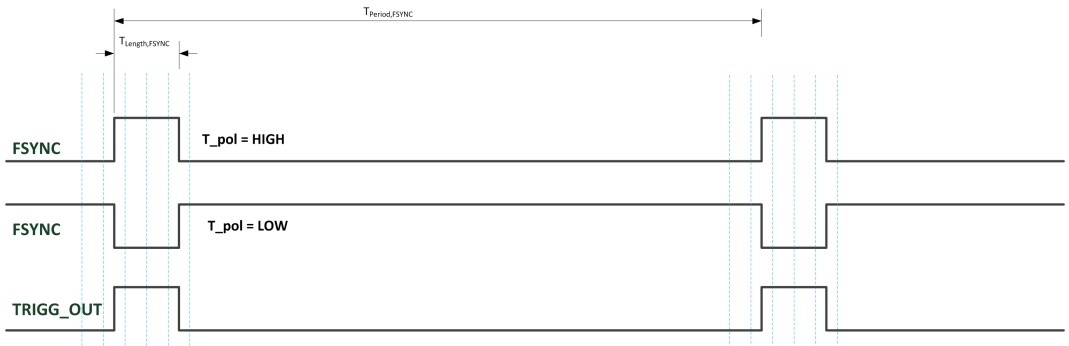



Figure 24: Waveform Continuous Mode

#### 14.1.1.5.7 Manual Mode

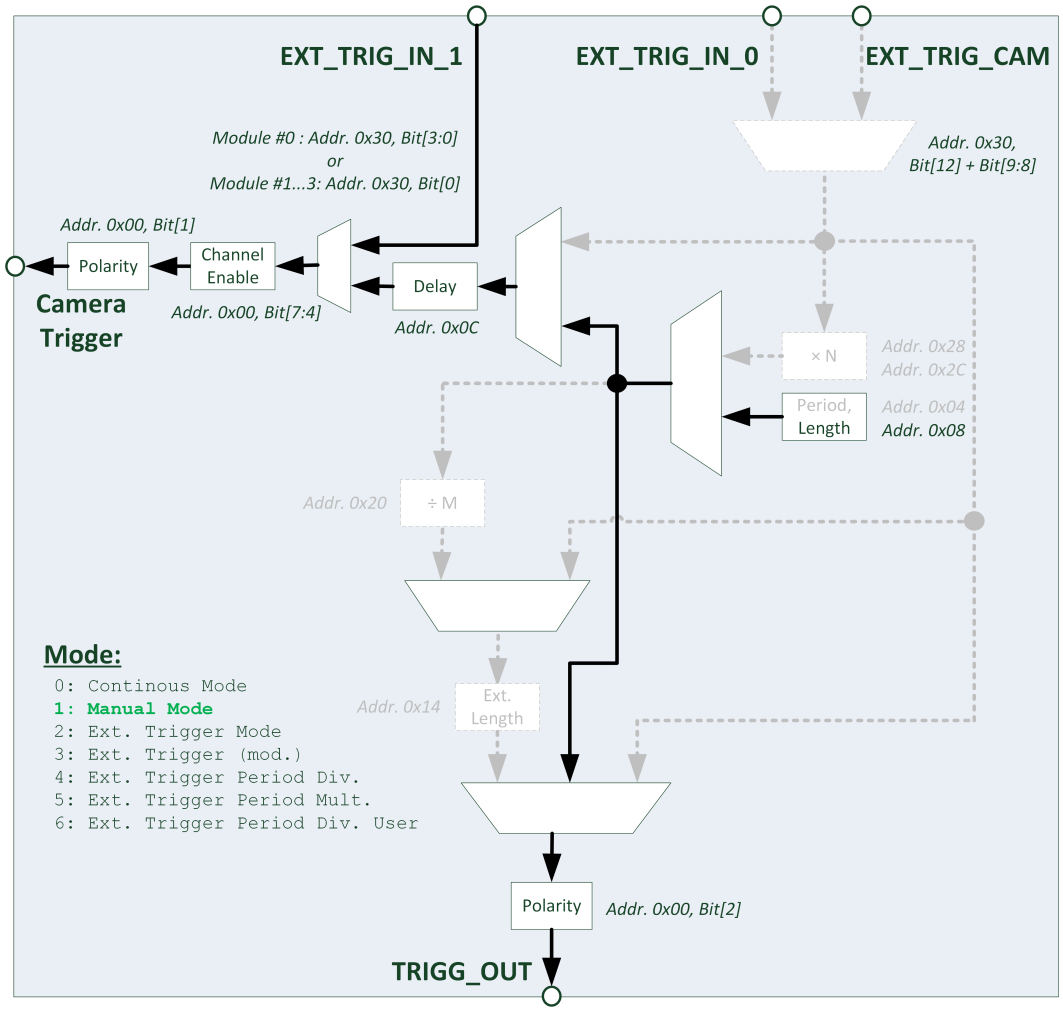



Figure 25: Manual Mode

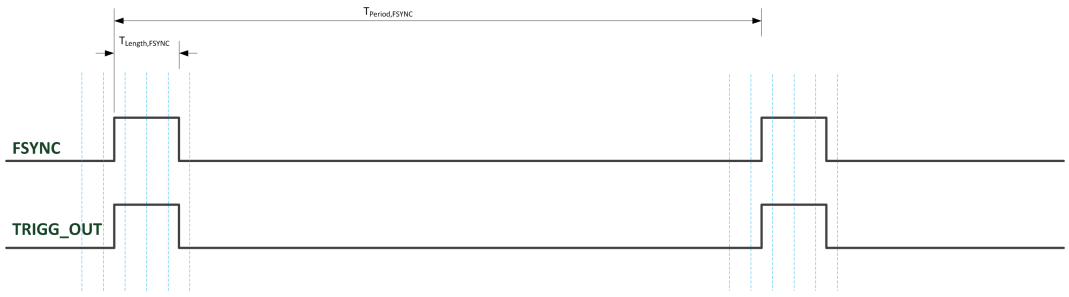



Figure 26: Waveform Manual Mode

#### 14.1.1.5.8 External Trigger Mode

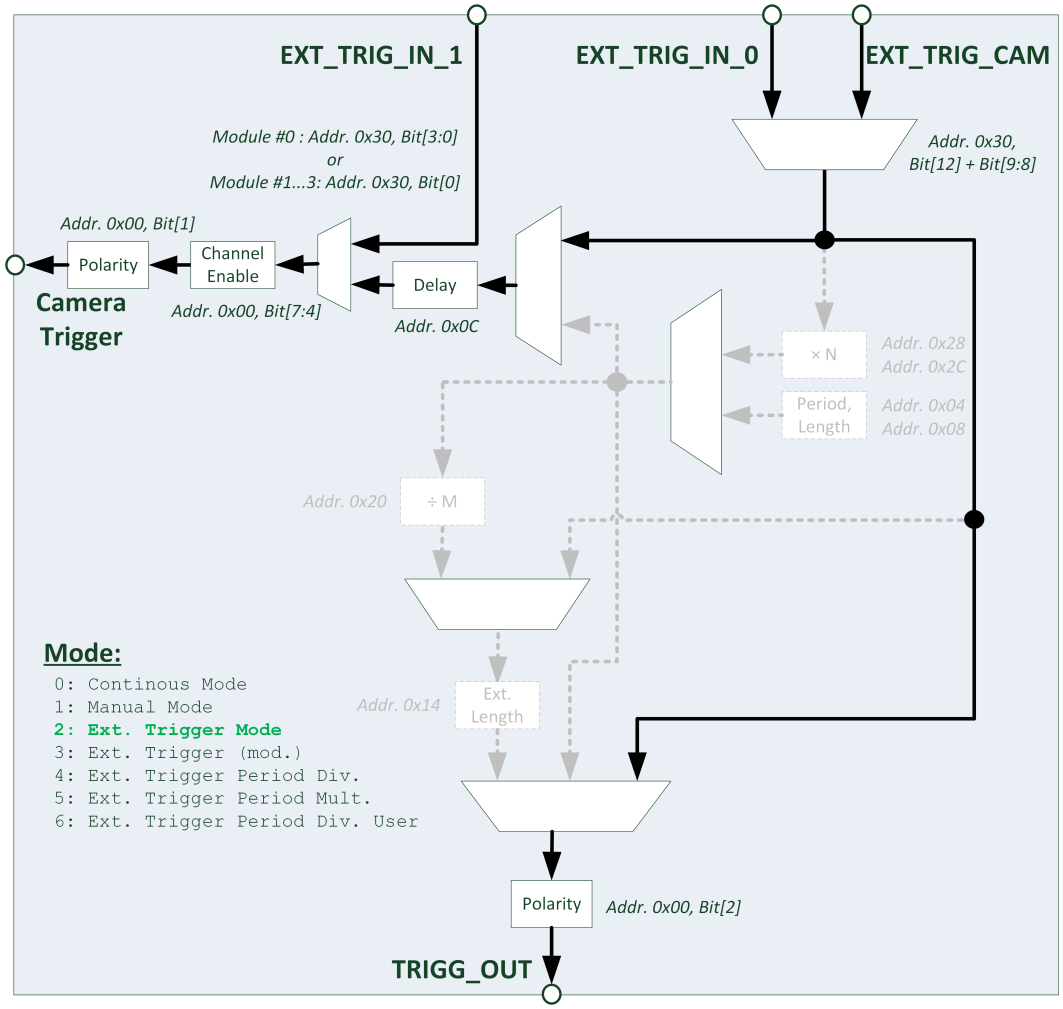



Figure 27: External Trigger Mode

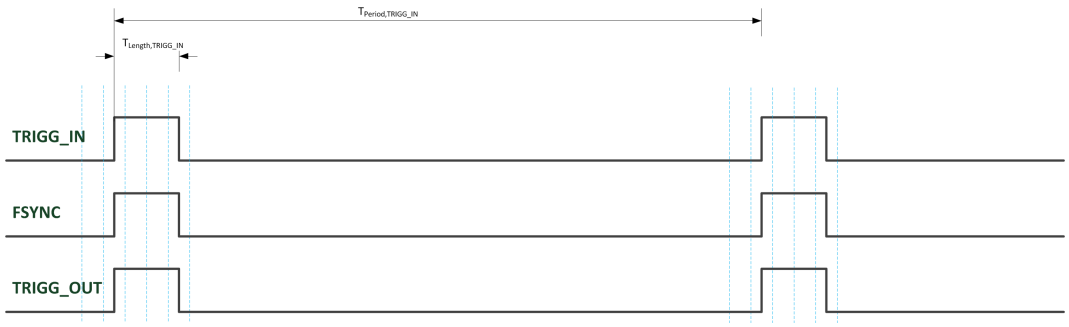



Figure 28: Waveform External Trigger

#### 14.1.1.5.9 External Trigger Mode (modified)

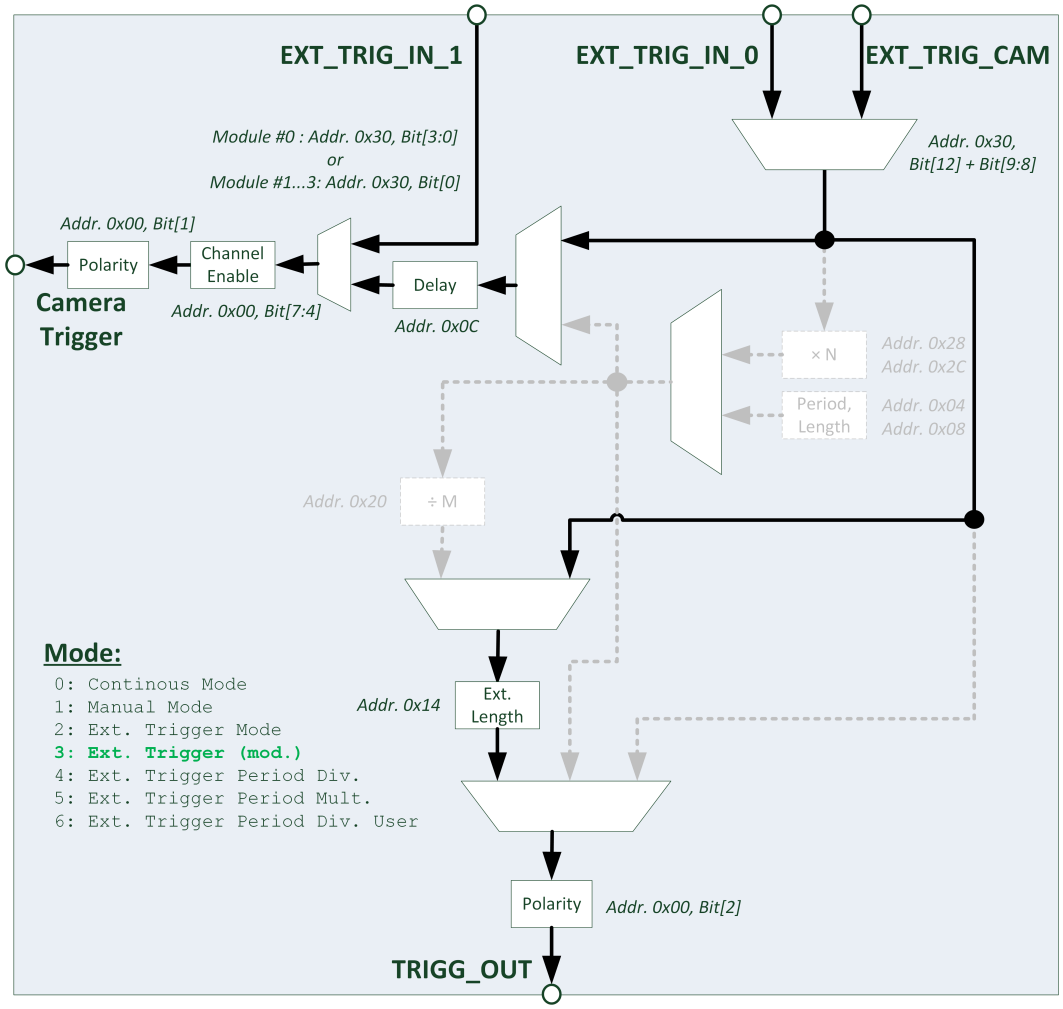



Figure 29: External Trigger Mode (modified)

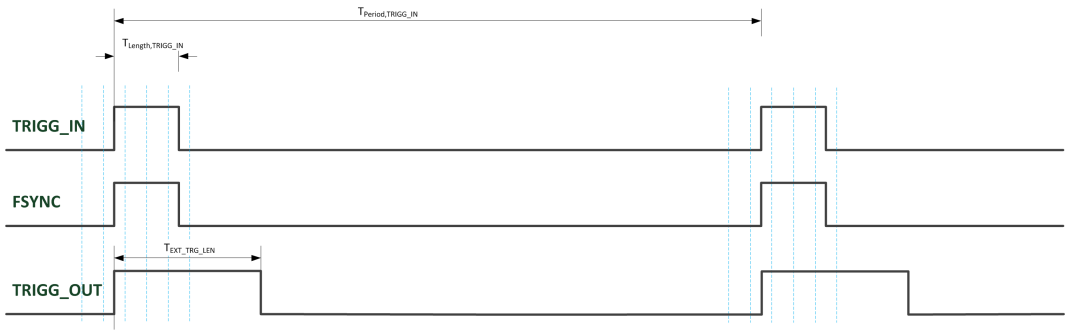



Figure 30: Waveform External Trigger (modified)

#### 14.1.1.5.10 External Trigger Input Frequency Multiplied

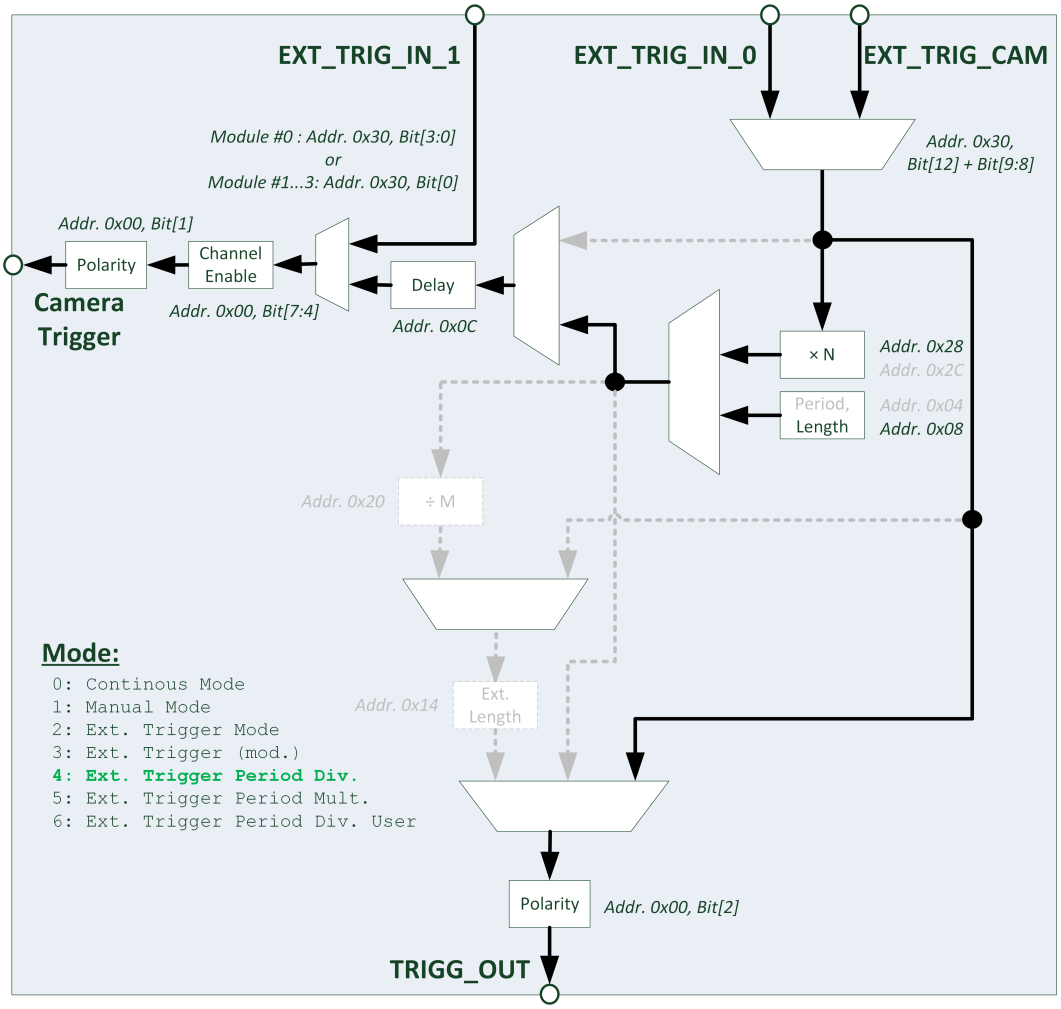



Figure 31: External Trigger Input Frequency Multiplied

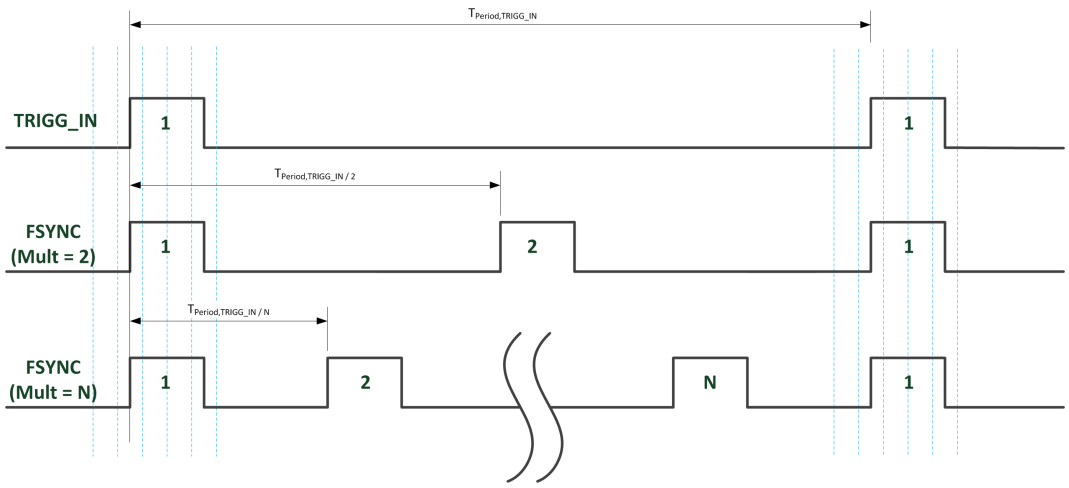



Figure 32: Different Multiplier Values

#### 14.1.1.5.11 External Trigger Output Frequency Divided

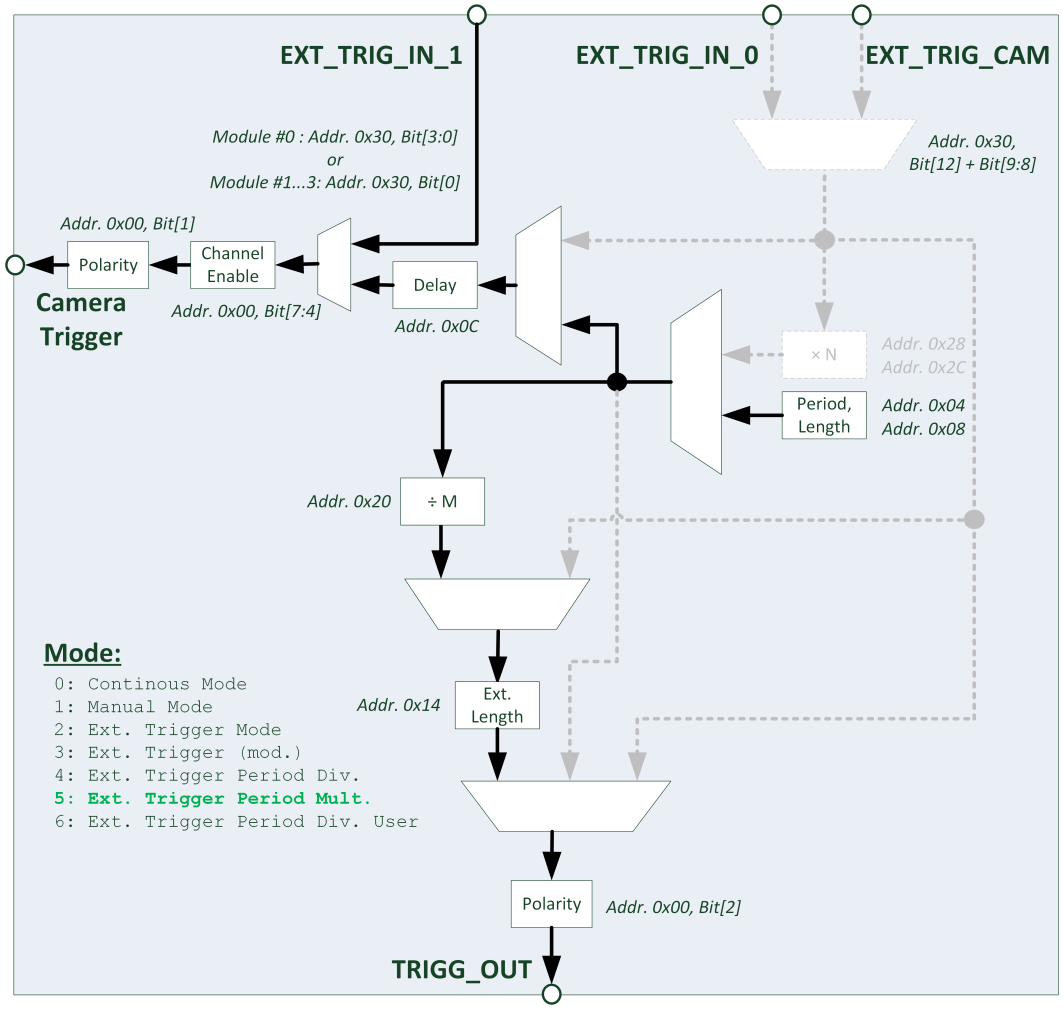



Figure 33: External Trigger Output Frequency Divided

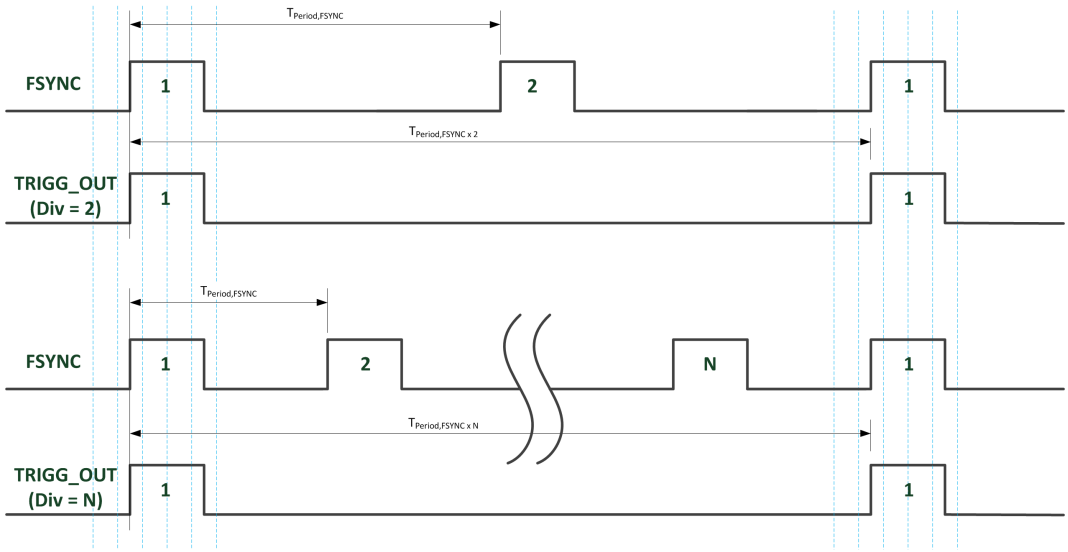



Figure 34: Different Divider Values

#### 14.1.1.5.12 External Trigger Input Frequency Multiplied (User Mode)

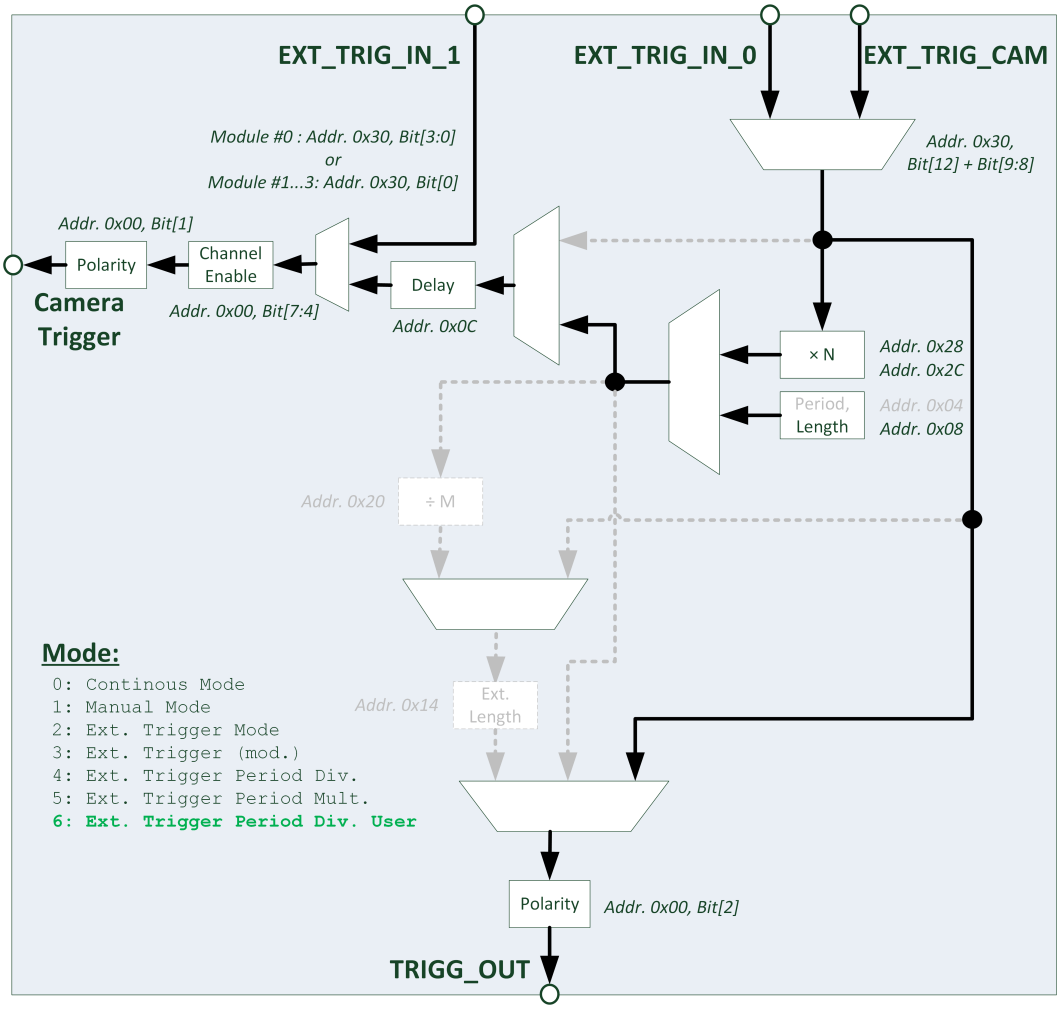



Figure 35: External Trigger Input Frequency Multiplied (User Mode)

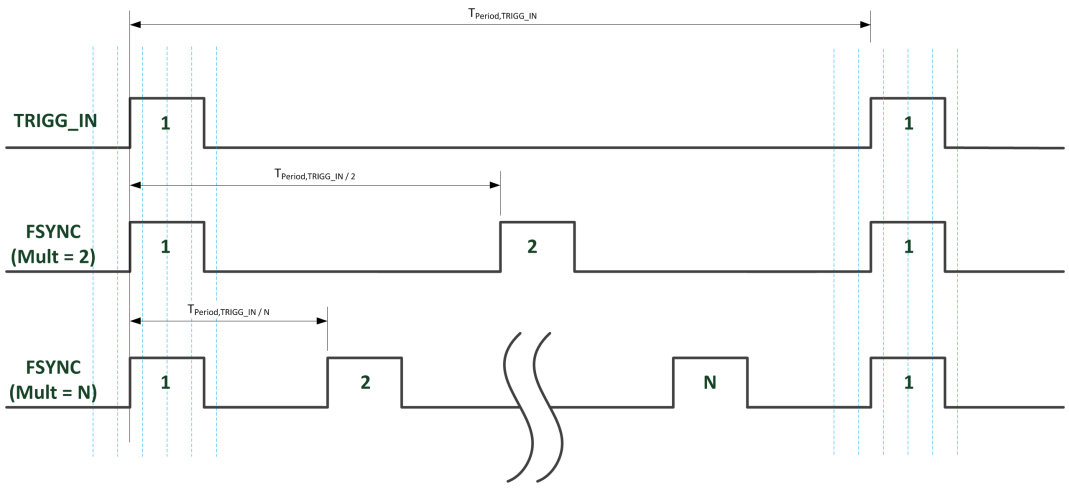



Figure 36: Different Multiplier Values

#### 14.1.1.6 Register

Table 26: Trigger Register - Overview

| Offset | Module |
|---|---|
| 0x600 | Trigger Module #0 |
| 0x680 | Trigger Event |
| 0x700 | Trigger Module #0 (mirrored) |
| 0x740 | Trigger Module #1 |
| 0x740 | Trigger Module #2 |
| 0x7C0 | Trigger Module #3 |

Table 27: Trigger Register - Address Map

| Offset | Description |
|---|---|
| Trigger Module #0...#3 | |
| 0x600 — 0x700 — 0x740 — 0x780 — 0x7C0 | TRG_CONTROL |
| 0x604 — 0x704 — 0x744 — 0x784 — 0x7C4 | TRG_PERIOD |
| 0x608 — 0x708 — 0x748 — 0x788 — 0x7C8 | TRG_LENGTH |
| 0x60C — 0x70C — 0x74C — 0x78C — 0x7CC | TRG_DELAY |
| 0x610 — 0x710 — 0x750 — 0x790 — 0x7D0 | EXT_TRG_PERIOD |
| 0x614 — 0x714 — 0x754 — 0x794 — 0x7D4 | EXT_TRG_LENGTH |
| 0x620 — 0x720 — 0x760 — 0x7A0 — 0x7E0 | TRIG_OUT_DIV |
| 0x628 — 0x728 — 0x768 — 0x7A8 — 0x7E8 | TRIG_IN_MULT |
| 0x62C — 0x72C — 0x76C — 0x7AC — 0x7EC | TRIG_IN_MULT_USR |
| 0x630 — 0x730 — 0x770 — 0x7B0 — 0x7F0 | EXT_TRIG_SRC_SEL |
| Trigger Event Channel x = 0...3 | |
| 0x680 — 0x688 — 0x690 — 0x698 | CHx_TRIGGER_TIME_P_LOW |
| 0x684 — 0x68C — 0x694 — 0x69C | CHx_TRIGGER_TIME_P_HIGH |
| 0x6C0 — 0x6C8 — 0x6D0 — 0x6D8 | CHx_TRIGGER_TIME_S_LOW |
| 0x6C4 — 0x6CC — 0x6D4 — 0x6DC | CHx_TRIGGER_TIME_S_HIGH |

#### 14.1.1.6.1 TRIGGER CONTROL

Register 14.7: TRG_CONTROL (0x600 — 0x700 — 0x740 — 0x780 — 0x7C00)

| 31 | 0 |
|---|---|
| TRG_CTRL | |

| Bits | Field Name | Description | Type | Reset |
|---|---|---|---|---|
| 31 | MOD_EN | Trigger Module #n Enable<br>0: Trigger Module is inactive<br>1: Trigger Module is activate<br><br>NOTE: Trigger Module #0 is mandatory and therefore is permanently activated. If Trigger Module #1...#3 is not active the corresponding Trigger Channel is controlled by Trigger Module #0 (Legacy Mode). | RW | 0 |
| 30:11 | RES | Reserved | RES | - |
| 10:8 | MODE | Trigger Mode<br>0: Continuous Trigger Mode<br>1: Manual Trigger Mode<br>2: External Trigger Mode<br>3: External Trigger (modified) Mode<br>4: Ext. Trigger (EXT_TRIG_IN) Frequency Multiplied<br>5: Ext. Trigger (EXT_TRIG_OUT) Frequency Divided<br>6: Ext. Trigger (EXT_TRIG_IN) Frequency Multiplied (User Mode)<br><br>NOTE: The External Sync Output is exclusively controlled by Trigger Module #0 | RW | 000 |
| 7:4 | CH_EN | Channel Enable<br>0: disable trigger<br>1: enable trigger<br>[4]: Trigger Channel 0/#n Enable<br>[5]: Trigger Channel 1 Enable<br>[6]: Trigger Channel 2 Enable<br>[7]: Trigger Channel 3 Enable<br><br>NOTE: Only Trigger Module #0 is able to control all available Trigger Channels (Legacy Mode). If Trigger Module #1...#3 is enabled, only Bit[4] is used to control its own Trigger Channel.<br>Bit[7:5] are not available in the Trigger Module #1...#3. | RW | 0x0 |
| 3 | RES | Reserved | RES | - |

Register 14.8: TRG_CONTROL (CONTINUED)

| Bits | Field Name | Description | Type | Reset |
|---|---|---|---|---|
| 2 | EXT_POL | Ext. Trigger Pulse Polarity | RW | 0 |
| 1 | POL | Pulse Polarity | RW | 0 |
| 0 | EN | Global Enable<br><br>NOTE: Trigger Module #0 is used to globally enable/disable all available trigger channels. This bit should be set last to ensure synchronous startup of all enabled trigger modules.<br>Bit[0] is not available in the Trigger Module #1...#3. | RW | 0 |

#### 14.1.1.6.2 TRIGGER PERIOD

Register 14.9: TRG_PERIOD (0x604 — 0x704 — 0x744 — 0x784 — 0x7C4)




| Bits | Field Name | Description | Type | Reset |
|---|---|---|---|---|
| 31:0 | TRG_PERIOD | Trigger Pulse Period<br>set the Trigger Pulse Period [in System Ticks] | RW | 0x0000 0000 |

#### 14.1.1.6.3 TRIGGER PULSE LENGTH

Register 14.10: TRG_LENGTH (0x604 — 0x708 — 0x748 — 0x788 — 0x7C8)




| Bits | Field Name | Description | Type | Reset |
|---|---|---|---|---|
| 31:0 | TRG_LEN | Trigger Pulse Length<br>set the Trigger Pulse Length [in System Ticks]<br><br>In Manual Trigger Mode:<br>Writing any value to this register generates a single trigger pulse with the configured pulse length | RW | 0x0000 0000 |

#### 14.1.1.6.4 TRIGGER DELAY

Register 14.11: TRG_DELAY (0x60C — 0x70C — 0x74C — 0x78C — 0x7CC)




| Bits | Field Name | Description | Type | Reset |
|---|---|---|---|---|
| 31:0 | TRG_DLY | Trigger Pulse Delay<br>set the Trigger Pulse Delay Time [in System Ticks]<br><br>*The Camera Trigger Pulse is delayed relative to the External Trigger Pulse* | RW | 0x0000 0000 |

#### 14.1.1.6.5 EXTERNAL TRIGGER PERIOD

Register 14.12: EXT_TRG_PERIOD (0x610 — 0x710 — 0x750 — 0x790 — 0x7B0)




| Bits | Field Name | Description | Type | Reset |
|---|---|---|---|---|
| 31:0 | EXT_TRG_PERIOD | External Trigger Pulse Period<br>External Trigger (EXT_TRIG_IN )Pulse Period [in System Ticks] | R | - |

#### 14.1.1.6.6 EXTRNAL TRIGGER PULSE LENGTH

Register 14.13: EXT_TRG_LENGTH (0x614 — 0x714 — 0x754 — 0x794 — 0x7B4)




| Bits | Field Name | Description | Type | Reset |
|---|---|---|---|---|
| 31:0 | EXT_TRG_LEN | External Trigger Pulse Length<br>set the External Trigger Pulse Length [in System Ticks]<br><br>*Note: not available in all modes* | RW | 0x0000 0000 |

#### 14.1.1.6.7 TRIG_OUT DIVIDER

Register 14.14: EXT_TRIG_OUT_DIV (0x620 — 0x720 — 0x760 — 0x7A0 — 0x7D0)

| 31:8 | 7:0 |
|---|---|
| RES | TRIG_OUT_DIV |

| Bits | Field Name | Description | Type | Reset |
|---|---|---|---|---|
| 31:8 | RES | Reserved | RES | - |
| 7:0 | TRIG_OUT_DIV | Ext. Trigger Output Frequency Divider<br>The external trigger (EXT_TRIG_OUT) signal frequency can be divided by an integer value from 1-8.<br><br>The EXT_TRIG_OUT frequency is the divided Camera Trigger Pulse frequency (see TRIGGER PERIOD). | RW | 0x01 |

#### 14.1.1.6.8 EXT_TRIG_IN MULTIPLIER

Register 14.15: TRIG_IN_MULT (0x628 — 0x728 — 0x768 — 0x7A8 — 0x7D8)

| 31:0 |
|---|
| TRIG_IN_MULT |

| Bits | Field Name | Description | Type | Reset |
|---|---|---|---|---|
| 31 | TRIG_IN_MODE | Ext. Trigger Input Mode<br>0: Automatic Mode<br>1: User Mode | RW | 0 |
| 30:0 | TRIG_IN_MULT | Ext. Trigger Input Frequency Multiplier<br>In **Automatic Mode** the external trigger (EXT_TRIG_IN) frequency can be multiplied by an integer value from 1 to 7.<br>In **User Mode** the multiplier can be set to any integer value from 1 to $2^{32}$-1.<br>In addition, the inverse divisor value in the EXT_TRIG_IN MULTIPLIER USER register must be set according to the desired multiplier value.<br><br>The Camera Trigger Pulse frequency is a multiple of the measured EXT_TRIG_IN frequency (see EXTERNAL TRIGGER PERIOD). | RW | 0x1 |

#### 14.1.1.6.9 EXT_TRIG_IN MULTIPLIER USER

Register 14.16: EXT_TRIG_IN_MULT_USR (0x62C — 0x72C — 0x76C — 0x7AC — 0x7DC)

| 31 … 0 |
|---|
| USER_DIV |

| Bits | Field Name | Description | Type | Reset |
|---|---|---|---|---|
| 31:0 | USER_DIV | Ext. Trigger Input Period User Divisor<br>User defined inverse fixed-point divisor of the measured external trigger (EXT_TRIG_IN) frequency.<br><br>The register value is calculated as follows:<br><br>$\frac{1}{EXT_TRIG_IN_MULT} \cdot 2^{31} = USER_DIV$<br><br>The value is presented in UQ1.31 notation.<br>The valid USER_DIV range is [0.0, 1.0]. | RW | 0x80000000 |

#### 14.1.1.6.10 EXT_TRIG SOURCE SELECT

31 Register 14.17: EXT_TRIG_SRC_SEL (0x630 — 0x730 — 0x770 — 0x7B0 — 0x7F0) 0

| EXT_TRIG_SRC_SEL |
|---|

| | | | | |
|---|---|---|---|---|
| 31:13 | RES | Reserved | RES | - |
| 12 | EXT_TRIG_CAM_EN | External Trigger Camera Adapter Enable<br>0: External Trigger from onboard connector<br>1: External TRigger from camera adapter<br><br>Select between the External Trigger Signal from the camera adapters or from the onboard connector with its trigger signals **EXT_TRIG_IN_0** (default mode) or **EXT_TRIG_IN_1**. | RW | 0 |
| 11:10 | RES | Reserved | RES | - |
| 9:8 | EXT_TRIG_CAM_SEL | External Trigger Camera Adapter Select<br>Select the Camera Adapter from which the External Trigger Signal shall be used<br>0: External Trigger from Camera Adapter #0<br>1: External Trigger from Camera Adapter #1<br>2: External Trigger from Camera Adapter #2<br>3: External Trigger from Camera Adapter #3 | RW | 0 |
| 7:4 | RES | Reserved | RES | - |
| 3:0 | EXT_TRIG_SEL | External Trigger Select<br>0: EXT_TRIG_IN_0<br>1: EXT_TRIG_IN_1<br>[0]: Channel 0 - External Trigger In Source Select<br>[1]: Channel 1 - External Trigger In Source Select<br>[2]: Channel 2 - External Trigger In Source Select<br>[3]: Channel 3 - External Trigger In Source Select<br><br>NOTE:<br>If any bit in this register is set, the second trigger (**EXT_TRIG_IN_1**) automatically becomes an input pin. Otherwise the second trigger pin is per default defined as an output pin. | RW | 0x0 |

#### 14.1.1.6.11 TRIGGER TIME - PRIMARY TIMESTAMP

Register 14.18: CHx_TRIGGER_TIME_P (0x680...0x69C)

| 31 | | 0 |
|---|---|---|
| TRG_TIME_P[63:32] / TRG_TIME_P[31:0]] | | |

| | | | | |
|---|---|---|---|---|
| 31:0 | TRG_TIME_P | Lower/upper part of the Primary Timestamp @ Trigger Event | R | - |

#### 14.1.1.6.12 TRIGGER TIME - SECONDARY TIMESTAMP

Register 14.19: CHx_TRIGGER_TIME_S (0x6C0...0x6DC)

| 31 | | 0 |
|---|---|---|
| TRG_TIME_S[63:32] / TRG_TIME_S[31:0]] | | |

| | | | | |
|---|---|---|---|---|
| 31:0 | TRG_TIME_S | Lower/upper part of the Secondary Timestamp @ Trigger Event | R | - |

## 14.2 proFRAME 3 - FPGA Register Description

This section describes some FPGA registers.
The documented register addresses are valid when using the low level register access mode e.g. by sxpf_read_register or sxpf_write_register function in the API or when using sxpfrw tool. When accessing the registers through the Plasma microcontroller in the proFRAME e.g. when using sxpf_plasma_readRegister or sxpf_plasma_writeRegister function the offset 0xA000 must be added. Same is true for a register write in an INI file.
So the following command line call is the same like the INI line below

```
$ sxpfrw 0 0x464 0x50
```

```
CMD_WRITE_REGISTER 8 0x0000A464 0x00000050
```

☞ Nearly all registers are controlled using the SX proFRAME software API. The API takes care that the registers are controlled in the correct way. So do not modify the register values unless you really know what you are doing because it can cause undefined behaviour.

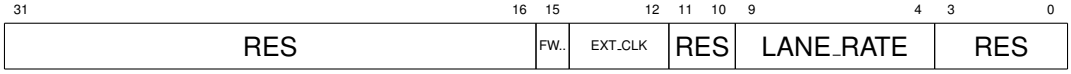
### 14.2.1 Video Processing Register

Table 28: Video Processing Register - Address Map

| Offset | Description |
|---|---|
| 0x464 — 0xC64 | MIPI_TX_CONTROL |
| 0x468 — 0xC68 | MIPI_RX_CONTROL |
| 0x47C — 0xC7C | PIXEL_FORMAT_CFG |
| 0x480 — 0xC80 | PKT_PARSE_CFG_0_4 |
| 0x484 — 0xC84 | PKT_PARSE_CFG_1_5 |
| 0x488 — 0xC88 | PKT_PARSE_CFG_2_6 |
| 0x48C — 0xC8C | PKT_PARSE_CFG_3_7 |
| 0x4C0 — 0xCC0 | PKT_PARSE_PACKET_CNT_0_4 |
| 0x4C4 — 0xCC0 | PKT_PARSE_PACKET_CNT_1_5 |
| 0x4C8 — 0xCC0 | PKT_PARSE_PACKET_CNT_2_6 |
| 0x4CC — 0xCC0 | PKT_PARSE_PACKET_CNT_3_7 |

#### 14.2.1.1 MIPI TRANSMITTER CONTROL

Register 14.20: MIPI_TX_CONTROL (0x464 — 0xC64)



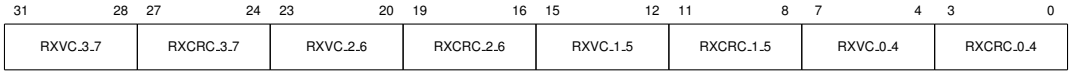

| Bits | Field Name | Description | Type | Reset |
|---|---|---|---|---|
| 31:16 | RES | Reserved | RES | - |
| 15 | FWD_EN | Enable Clock Fowarding Mode | RW | 0 |
| 14:12 | EXT_CLK | External Clock Source (for Clock Fowarding Mode)<br>**Val** / **External Clock Source**<br>0 — MIPI D-PHY RX Interface 0<br>2 — MIPI D-PHY RX Interface 2<br>4 — MIPI D-PHY RX Interface 3<br>6 — MIPI D-PHY RX Interface 4 | RW | 0 |
| 11:10 | RES | Reserved | RES | - |
| 9:4 | LANE_RATE | MIPI Tx Lane Rate<br>Set the MIPI Tx lane rate per bank.<br>Note:<br>• 0x464: MIPI D-PHY TX Interface 1/3<br>• 0xC64: MIPI D-PHY TX Interface 5/7 | RW | 4 |
| 3:0 | RES | Reserved | RES | 0 |

| Fixed Lane Rate | | | |
|---|---|---|---|
| **Val** | **Lane Rate [Mbps]** | **Val** | **Lane Rate [Mbps]** |
| 0 | 800 | 11 | 1900 |
| 1 | 900 | 12 | 2000 |
| 2 | 1000 | 13 | 2100 |
| 3 | 1100 | 14 | 2200 |
| 4 | 1200 | 15 | 2300 |
| 5 | 1300 | 16 | 2400 |
| 6 | 1400 | 17 | 2500 |
| 7 | 1500 | 27 | 400 |
| 8 | 1600 | 28 | 500 |
| 9 | 1700 | 29 | 600 |
| 10 | 1800 | 30 | 700 |
| **Variable Lane Rate (Clock Forwarding)** | | | |
| **Val** | **Lane Rate Range** | **Val** | **Lane Rate Range** |
| 32 | 325 ... 749 Mbps | 33 | 750 ... 1599 Mbps |
| 34 | 1600 ... 2500 Mbps | | |

Note: DO NOT perform a Bank Reset when switching to Clock Forward mode within the same bank.

#### 14.2.1.2 MIPI RECEIVER CONTROL

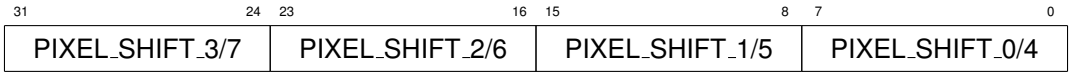
Register 14.21: MIPI_RX_CONTROL (0x468 — 0xC68)




| Bits | Field Name | Description | Type | Reset |
|---|---|---|---|---|
| 31:28 | RXVC_3_7 | MIPI VC Mask Ch3/Ch7 | RW | 0xF |
| 27:24 | RXCRC_3_7 | MIPI Rx CRC Control Ch3/Ch7<br>[ 27] : Force ECC Error C3/Ch7<br>[ 26] : ECC Counter Reset Ch3/Ch7<br>[ 25] : Force CRC Error Ch3/Ch7<br>[ 24] : CRC Counter Reset Ch3/Ch7 | WO | 0 |
| 23:20 | RXVC_2_6 | MIPI VC Mask Ch2/Ch6 | RW | 0xF |
| 19:16 | RXCRC_2_6 | MIPI Rx CRC Control Ch2/Ch6<br>[ 19] : Force ECC Error Ch2/Ch6<br>[ 18] : ECC Counter Reset Ch2/Ch6<br>[ 17] : Force CRC Error Ch2/Ch6<br>[ 16] : CRC Counter Reset Ch2/Ch6 | WO | 0 |
| 15:12 | RXVC_1_5 | MIPI VC Mask Ch1/Ch6 | RW | 0xF |
| 11:8 | RXCRC_1_5 | MIPI Rx CRC Control Ch1/Ch5<br>[ 11] : Force ECC Error Ch1/Ch5<br>[ 10] : ECC Counter Reset Ch1/Ch5<br>[ 9] : Force CRC Error Ch1/Ch5<br>[ 8] : CRC Counter Reset Ch1/Ch5 | WO | 0 |
| 7:4 | RXVC_0_4 | MIPI VC Mask Ch0/Ch4<br>let only pass data of enabled virtual channel(s)<br>Bit4 : VC==0 ... Bit7 : VC==3 | RW | 0xF |
| 3:0 | RXCRC_0_4 | MIPI Rx CRC Control Ch0/Ch4<br>[ 3] : Force ECC Error Ch0/Ch4<br>[ 2] : ECC Counter Reset Ch0/Ch4<br>[ 1] : Force CRC Error Ch0/Ch4<br>[ 0] : CRC Counter Reset Ch0/Ch4 | WO | 0 |

#### 14.2.1.3 PIXEL FORMATTER CONFIGURATION

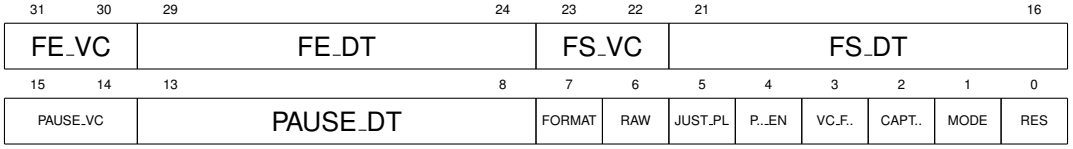
Register 14.22: PIXEL_FORMAT_CFG (0x47C — 0xC7C)




| Bits | Field Name | Description | Type | Reset |
|---|---|---|---|---|
| 31:24 | PIX_SHIFT_3_7 | Pixel Shift Value Ch.3/7 | RW | 0 |
| 23:16 | PIX_SHIFT_2_6 | Pixel Shift Value Ch.2/6 | RW | 0 |
| 15:8 | PIX_SHIFT_1_5 | Pixel Shift Value Ch.1/5 | RW | 0 |
| 7:0 | PIX_SHIFT_0_4 | Pixel Shift Value Ch.0/4<br>Shift the formatted pixel left by this value (max. 16). | RW | 0 |

#### 14.2.1.4 PACKET PARSER CONFIGURATION

Register 14.23: PKT_PARSE_CFG_x (0x480 ... 0x48C — 0xC80 ... 0xC8C)




| Bits | Field Name | Description | Type | Reset |
|---|---|---|---|---|
| 31:30 | FE_VC | Frame End Virtual Channel<br>Default: VC=0 | RW | 0x0 |
| 29:24 | FE_DT | Frame End Data Type<br>Default: DT= Frame End Packet | RW | 0x01 |
| 23:22 | FS_VC | Frame Start Virtual Channel<br>Default: VC=0 | RW | 0x0 |
| 21:16 | FS_DT | Frame Start Data Type<br>Default: DT= Frame Start Packet | RW | 0x00 |
| 15:14 | PAUSE_VC | Pause Packet Virtual Channel<br>Default: VC=0 | RW | 0x0 |
| 13: 8 | PAUSE_DT | Pause Packet Data Type<br>Default: DT= 0x17, VC = 0<br>**NOTE:**<br>The defined data type for pause packets is used for **both** modes, record and replay. | RW | 0x17 |
| 7 | FORMAT | Data Formatter Enable<br>0: CSI-2 packets are unprocessed<br>1: The pixels in the CSI-2 packets are formatted depending on the data type (all RAW and 8-bit formats are supported).<br>**NOTE**:<br>In this mode, only the payload is processed, so you have to activate the **JUST_PL** bit and must not enable the **CAPT_ALL** and **RAW_PKT** bit. It is also recommended to use the **Frame Mode** so the captured image starts with the first line within a video frame. | RW | 0 |

Register 14.24: PKT_PARSE_CFG_X (CONTINUED) (0x480 ... 0x48C — 0xC80 ... 0xC8C)

| Bits | Field Name | Description | Type | Reset |
|---|---|---|---|---|
| 6 | RAW_PKT | Raw Packet Mode<br>0 : Parse packets for CSI-2 short and long packets<br>1 : Use the (”raw”) data packets directly from the receiver without parsing the packet header.<br><br>This function is useful for checking traffic on at least one data lane, even if other lanes are not working. | RW | 0 |
| 5 | JUST_PL | Just Payload<br>0 : capture the complete frame (Preamble, Header, Payload & Footer)<br>1 : just capture Payload | RW | 0 |
| 4 | PAUSE_EN | Enable Pause Packet Inserter<br>0 : Pause Packet disabled<br>1 : Pause Packet enabled | RW | 0 |
| 3 | VC_FILTER | Virtual Channel Filter<br>Enable Virtual Channel filtering of Frame Start/End Packet<br>If virtual channel of frame start/end packets is NOT(!) VC=0 (default), you must adapt it to your setup. | RW | 1 |
| 2 | CAPT_ALL | Capture All<br>0: only Long Packets are recorded (Legacy Mode)<br>1: Short & Long Packets are recorded | RW | 0 |
| 1 | MODE | Packet Parsing Mode<br>**Frame Mode (0)**: the video will be recorded according to the settings in the registers.<br>**Packet Mode (1)**: data packets are recorded directly from the MIPI interfaces - without any filters. Only the number of recorded packets can be controlled (see **??**). | RW | 0 |
| 0 | RES | reserved | RES | - |

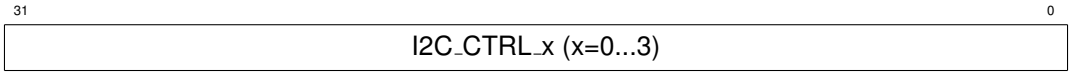
### 14.2.2 PLASMA Control Register

Table 29: PLASMA Control Register - Address Map

| Offset | Description |
|---|---|
| 0x20 | I2C_CONTROL_0 |
| 0x24 | I2C_CONTROL_1 |
| 0x28 | I2C_CONTROL_2 |
| 0x2C | I2C_CONTROL_3 |
| 0x30 | CAM_AD_CONTROL_0 |
| 0x34 | CAM_AD_CONTROL_1 |
| 0x44 | DEB_COUNTER_VAL |
| 0x50 | BASE_CONTROL |
| 0x54 | MIPI_CTL_STAT |
| 0x58 | MIPI_CTL_STAT_SET |
| 0x5C | MIPI_CTL_STAT_CLR |

#### 14.2.2.1 I2C BUS CONTROL - PORT 0...3

Register 14.25: I2C_CTRL_x (0x20 — 0x24 — 0x28 — 0x2c)




| Bits | Field Name | Description | Type | Reset |
|---|---|---|---|---|
| 31:7 | RES | Reserved | RES | - |
| 6:0 | I2C_BUS_EN | I2C Bus Enable Port<br>0: I2C Bus disabled,<br>1: I2C Bus enabled<br><br>[0]: reserved<br>[1]: I2C Bus #0 (Int) - MachXO2/3, EEPROM<br>[2]: I2C Bus Baseboard (Sensor TMP102,<br>SiLab Si5345 & Si549, Poti MAX5479, PoC)<br>[3] : I2C Bus #1 (Ext0) - Deserialiser #0<br>[4] : I2C Bus #2 (Ext1) - Deserialiser #0<br>[5] : I2C Bus #3 (Ext2) - Deserialiser #1<br>[6] : I2C Bus #4 (Ext3) - Deserialiser #1 | RW | 0x7 |

#### 14.2.2.2 CAMERA ADAPTER CONTROL

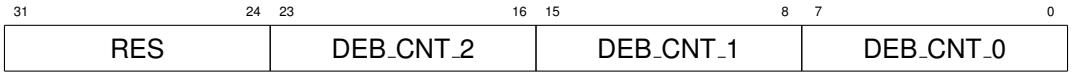
Register 14.26: CAM_AD_CTRL_x (0x30 — 0x34)




| Bits | Field Name | Description | Type | Reset |
|---|---|---|---|---|
| 31:3 | RES | Reserved | RES | - |
| 2 | MIPI_BANK_RESET | MIPI D-PHY Bank Reset<br>Reset all MIPI D-PHY Cores within the bank<br>0: MIPI D-PHY Bank Reset is inactive,<br>1: MIPI D-PHY Bank Reset is active | RW | 0 |
| 1 | POC_EN | Baseboard PoC Enable<br>0: PoC is disabled,<br>1: PoC is enabled | RW | 0 |
| 0 | CBRESET | Crosslink FPGA CRESETB<br>Sets Crosslink FPGA in reset state for programming<br>0: CRESETB is inactive,<br>1: CRESETB is active | RW | 0 |

#### 14.2.2.3 DEBOUNCE COUNTER VALUE

Register 14.27: DEB_COUNTER_VAL (0x44)




| Bits | Field Name | Description | Type | Reset |
|---|---|---|---|---|
| 31:24 | RES | Reserved | RES | - |
| 23:16 | DEB_CNT_2 | Debounce Counter Value - FRAME_EXT_SYNC1 | RW | 0xFF |
| 15:8 | DEB_CNT_1 | Debounce Counter Value - FRAME_EXT_SYNC0 | RW | 0xFF |
| 7:0 | DEB_CNT_0 | Debounce Counter Value - SD_SYNC_INT | RW | 0xFF |

#### 14.2.2.4 DEBOUNCE CONTROL

Register 14.28: DEB_CONTROL (0x48)




| Bits | Field Name | Description | Type | Reset |
|---|---|---|---|---|
| 31:9 | RES | Reserved | RES | - |
| 7:3 | RES | Reserved | RES | - |
| 8 | DEB_BYPASS_TRIGGER | Debounce BYPASS TRIGGER | RW | 0 |
| 2:0 | DEB_INVERT | Debounce Invert Output | RW | 0x0 |

#### 14.2.2.5 BASEBOARD CONTROL

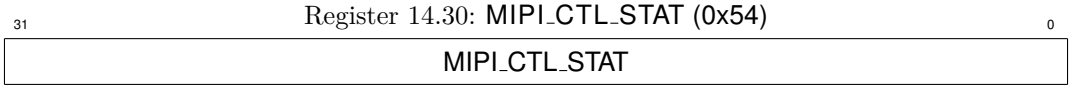
Register 14.29: BASE_CTL (0x50)




| Bits | Field Name | Description | Type | Reset |
|---|---|---|---|---|
| 31:2 | RES | Reserved | RES | - |
| 1 | I2C_BYPASS1_EN | I2C Bypass 1 Enable<br>Connect I2C bus **CAM1_EXT2** with I2C bus **CAM2_EXT3**.<br>0: I2C buses are not connected,<br>1: I2C buses are connected | RW | 0 |
| 0 | I2C_BYPASS0_EN | I2C Bypass 0 Enable<br>Connect I2C bus **CAM1_EXT0** with I2C bus **CAM2_EXT13**.<br>0: I2C buses are not connected,<br>1: I2C buses are connected | RW | 0 |

#### 14.2.2.6 MIPI CONTROL/STATUS

Register 14.30: MIPI_CTL_STAT (0x54)




| Bits | Field Name | Description | Type | Reset |
|---|---|---|---|---|
| 31:24 | RES | Reserved | RES | - |
| 23:16 | MIPI_INIT_DONE | MIPI D-PHY Core Init Done<br>[16] : MIPI D-PHY Core #0<br>...<br>[23] : MIPI D-PHY Core #7 | RO | - |
| 15:8 | RES | Reserved | RES | - |
| 7:0 | MIPI_DPHY_RST | MIPI D-PHY Core Reset<br>[0] : MIPI D-PHY Core #0<br>...<br>[7] : MIPI D-PHY Core #7 | RO | - |

#### 14.2.2.7 MIPI CONTROL/STATUS SET

Register 14.31: MIPI_CTL_STAT_SET (0x58)




| Bits | Field Name | Description | Type | Reset |
|---|---|---|---|---|
| 31:8 | RES | Reserved | RES | - |
| 7:0 | MIPI_DPHY_RST_SET | MIPI D-PHY Core Set Reset<br>[0] : MIPI D-PHY Core #0<br>...<br>[7] : MIPI D-PHY Core #7 | WO | - |

#### 14.2.2.8 MIPI CONTROL/STATUS CLEAR

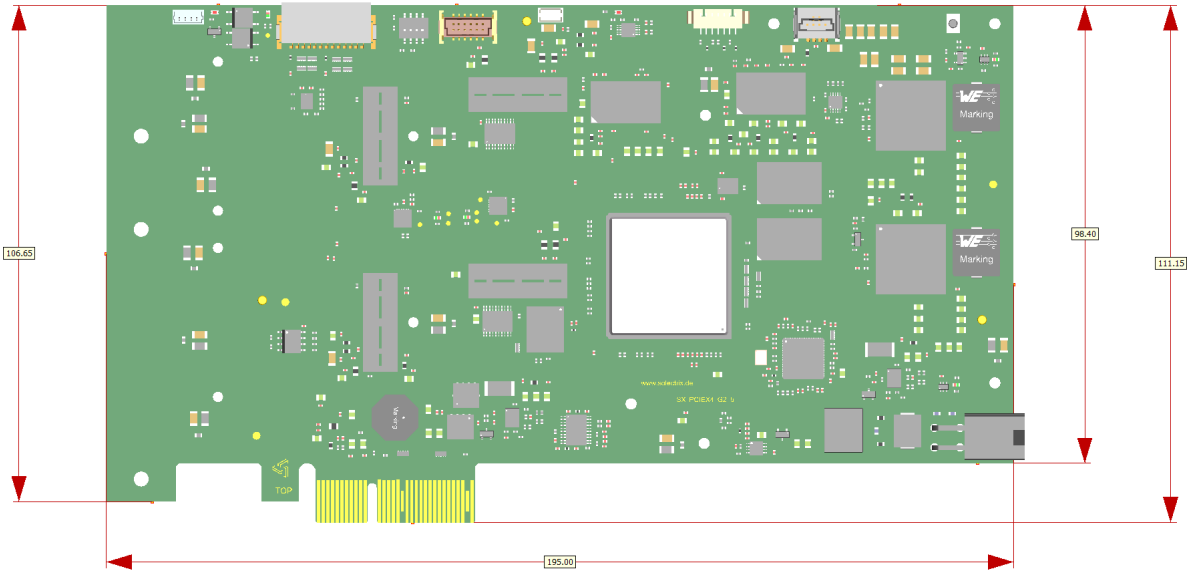
Register 14.32: MIPI_CTL_STAT_CLR (0x5C)




| Bits | Field Name | Description | Type | Reset |
|---|---|---|---|---|
| 31:8 | RES | Reserved | RES | - |
| 7:0 | MIPI_DPHY_RST_CLR | MIPI D-PHY Core Clear Reset<br>[0] : MIPI D-PHY Core #0<br>...<br>[7] : MIPI D-PHY Core #7 | WO | - |

## 14.3 proFRAME V2 dimensions

proFRAME V2 board dimensions in mm.




## 14.4 proFRAME V2 component overview

